



**Universidad
de Cartagena**
Fundada en 1827

ESTIMACIÓN DE LA SATISFACCIÓN DEL USUARIO EN PRUEBAS DE USABILIDAD DE SOFTWARE MEDIANTE EL ANÁLISIS DE SENTIMIENTOS

Investigador:

DAVID JOSÉ GARCÉS CONDE

LinkedIn: [@garconde](#)

UNIVERSIDAD DE CARTAGENA
FACULTAD DE INGENIERÍA
PROGRAMA DE INGENIERÍA DE SISTEMAS
CARTAGENA DE INDIAS

2025



**Universidad
de Cartagena**
Fundada en 1827

ESTIMACIÓN DE LA SATISFACCIÓN DEL USUARIO EN PRUEBAS DE
USABILIDAD DE SOFTWARE MEDIANTE EL ANÁLISIS DE SENTIMIENTOS

PROPUESTA DE TRABAJO DE GRADO

Grupo de investigación:

DaToS

Línea de investigación:

INTERACCIÓN HUMANO-COMPUTADOR

Investigador:

DAVID JOSÉ GARCÉS CONDE

LinkedIn: [@garconde](#)

Director:

ING. GABRIEL CHANCHÍ GOLONDRINO, PHD

UNIVERSIDAD DE CARTAGENA
FACULTAD DE INGENIERÍA
PROGRAMA DE INGENIERÍA DE SISTEMAS
CARTAGENA DE INDIAS

2025

DEDICATORIA Y AGRADECIMIENTO

Dedico esta obra:

A mi Creador, por la más alta prueba de amor que jamás se podrá igualar.

A mi madre Dennis, quien se fue a la eternidad habiéndome enseñado a temer a Dios.

A mi padre José, el fiel amigo, sabio maestro, honrado trabajador.

A mi esposa Lilia, mi luna, por la dulzura de su compañía.

A mi hermana Saray, mi ejemplo de vida, por su tenacidad que inspira.

A mis hermanos, mis tías y mis abuelos, impulsores de vida.

A mi amigo Deimer, entre guitarras y proyectos, siempre presente.

A mi iglesia Antorcha Encendida, refugio de paz.

Al Ing. Gabriel, tutor dedicado, crítico y paciente.

A mis profesores de toda la vida.

Y a mi Universidad de Cartagena.

A todos ellos: ¡Gracias, muchas gracias!

LISTA DE CONTENIDO

RESUMEN.....	11
ABSTRACT.....	12
1. INTRODUCCIÓN.....	13
1.1. Planteamiento del problema.....	13
1.2. Formulación del problema.....	14
1.3. Justificación.....	14
1.4. Contexto de la investigación.....	16
1.5. Objetivos y alcance.....	17
1.5.1. Objetivo general.....	17
1.5.2. Objetivos específicos.....	17
2. MARCO DE REFERENCIA.....	18
2.1. Estado del arte.....	18
2.1.1. Antecedentes bibliográficos.....	18
2.1.2. Antecedentes internacionales.....	19
2.1.3. Antecedentes nacionales y regionales.....	20
2.1.4. Antecedentes a nivel de software comercial.....	22
2.2. Marco teórico.....	24
2.2.1. Ingeniería de Software.....	24
2.2.2. Calidad del software.....	24
2.2.3. Usabilidad.....	27
2.2.4. Satisfacción del usuario.....	29
2.2.5. Estimación de la satisfacción del usuario.....	30
2.2.6. Laboratorio de usabilidad.....	31
2.2.7. Análisis de sentimientos.....	32
2.2.8. Relación entre emociones, usabilidad y experiencia de usuario.....	32
3. METODOLOGÍA.....	34
3.1. Tipo de investigación, diseño y enfoque.....	34
3.2. Técnicas de recolección de información.....	35
3.3. Análisis de los datos.....	35
3.4. Metodología de gestión del proyecto.....	36
3.4.1. PT1 - Caracterización conceptual.....	36
3.4.2. PT2 - Diseño e implementación del sistema.....	37
3.4.3. PT3 - Pruebas del sistema.....	37
4. RESULTADOS Y DISCUSIÓN.....	39
4.1. Paquete de trabajo 1: Caracterización conceptual.....	39

4.1.1. Usabilidad.....	39
4.1.1.1. Usabilidad como producto.....	42
4.1.1.2. Usabilidad como proceso.....	43
4.1.1.3. Medición de la usabilidad.....	43
4.1.1.4. Plan para el estudio de la usabilidad.....	44
4.1.2. Test con usuarios.....	46
4.1.2.1. Acuerdo de confidencialidad.....	47
4.1.2.2. Cuestionario pre-test.....	47
4.1.2.3. Listado de tareas.....	47
4.1.2.4. Cuestionario post-test.....	48
4.1.3. Preguntas abiertas.....	48
4.1.4. Análisis de sentimientos.....	50
4.2. Paquete de trabajo 2: Diseño e implementación del sistema.....	51
4.2.1. Especificación de requisitos.....	52
4.2.1.1. Roles.....	52
4.2.1.2. Restricciones.....	52
4.2.1.3. Requisitos funcionales.....	53
4.2.1.4. Requisitos no funcionales.....	55
4.2.2. Modelo de dominio.....	56
4.2.3. Vista de escenarios del sistema.....	58
4.2.4. Vista lógica del sistema.....	65
4.2.5. Modelo de base de datos.....	66
4.2.6. Mockups de las vistas del sistema.....	71
4.2.7. Vista de desarrollo del sistema.....	76
4.2.8. Elección de tecnologías de desarrollo.....	78
4.2.9. Desarrollo del lado del servidor.....	83
4.2.10. Desarrollo del lado del cliente.....	98
4.2.11. Vistas del aplicativo.....	106
4.3. Paquete de trabajo 3: Pruebas del sistema.....	117
4.3.1. Etapas de la prueba de usabilidad realizada.....	117
4.3.1.1. Acuerdo de confidencialidad.....	118
4.3.1.2. Etapa 1: cuestionario pre-test.....	118
4.3.1.3. Etapa 2: listado de tareas.....	119
4.3.1.4. Etapa 3: cuestionario post-test.....	121
4.3.2. Participantes de la prueba.....	123
4.3.3. Ejecución de la prueba.....	124
4.3.4. Resultados de la prueba de usabilidad.....	127

4.3.4.1. Eficacia.....	127
4.3.4.2. Eficiencia.....	129
4.3.4.3. Satisfacción de los usuarios a partir de preguntas cerradas.....	130
4.3.4.4. Satisfacción de los usuarios a partir de preguntas abiertas.....	131
4.3.4.5. Grado de usabilidad del software evaluado.....	133
4.3.5. Cálculo de la usabilidad mediante el uso del aplicativo.....	134
4.3.5.1. Eficacia.....	136
4.3.5.2. Eficiencia.....	140
4.3.5.3. Satisfacción de los usuarios a partir de preguntas cerradas.....	143
4.3.5.4. Satisfacción de los usuarios a partir de preguntas abiertas.....	144
4.3.5.5. Estimación del atributo Satisfacción general.....	146
4.3.5.6. Grado de usabilidad del software evaluado.....	148
4.4. Resultados inesperados.....	150
4.5. Discusión de los resultados.....	152
4.6. Cumplimiento de los objetivos.....	154
5. CONCLUSIONES Y RECOMENDACIONES.....	156
6. REFERENCIAS BIBLIOGRÁFICAS.....	159

LISTA DE TABLAS

Tabla 1. Definiciones del concepto de usabilidad en los últimos años.....	40
Tabla 2. Requisitos funcionales del sistema.....	53
Tabla 3. Descripción del caso de uso Gestionar evaluaciones.....	60
Tabla 4. Descripción del caso de uso Cargar valores.....	61
Tabla 5. Descripción caso de uso Obtener nivel de usabilidad.....	63
Tabla 6. Descripción caso de uso Obtener reporte resultados.....	64
Tabla 7. Descripción de las claves y los valores de la colección Software_evaluado.....	67
Tabla 8. Descripción de las claves y los valores de la colección Evaluacion.....	69
Tabla 9. Tecnologías implementadas en el backend.....	78
Tabla 10. Tecnologías implementadas en el frontend.....	80
Tabla 11. Etapa 1: preguntas del cuestionario pre-test.....	118
Tabla 12. Etapa 2: listado de tareas para el participante.....	119
Tabla 13. Valores de referencia para cada tarea a criterio del evaluador.....	121
Tabla 14. Etapa 3: preguntas del cuestionario post-test.....	122
Tabla 15. Personal seleccionado para la prueba de usabilidad.....	123
Tabla 16. Cantidad de subtareas totales y cantidad de subtareas completadas por usuario.....	127
Tabla 17. Porcentajes de eficacia.....	128
Tabla 18. Tiempos estimados para cada tarea y duración en completarlas por usuario.....	129
Tabla 19. Porcentajes de eficiencia.....	130
Tabla 20. Respuesta de cada usuario a las preguntas cerradas.....	130
Tabla 21. Porcentaje de satisfacción por usuarios en cada pregunta cerrada.....	131
Tabla 22. Resumen de los comentarios de los usuarios en la prueba de usabilidad.....	132
Tabla 23. Resultados para el software evaluado.....	134
Tabla 24. Resultados del evaluador de usabilidad para el software evaluado.....	148

LISTA DE FIGURAS

Figura 1. Paquetes de trabajo como metodología de gestión del proyecto.....	36
Figura 2. Etapas de una prueba de usabilidad.....	47
Figura 3. Modelo de dominio del mundo real.....	57
Figura 4. Modelo de dominio del sistema desarrollado.....	58
Figura 5. Diagrama de casos de uso.....	59
Figura 6. Diagrama de clases del sistema.....	66
Figura 7. Estructuras de las colecciones de datos.....	67
Figura 8. Mockup de la vista principal.....	72
Figura 9. Mockup de la vista al crear una evaluación.....	73
Figura 10. Mockup de la vista al cargar los datos.....	74
Figura 11. Mockup de la vista al analizar los datos.....	75
Figura 12. Mockup de la vista al graficar la usabilidad.....	76
Figura 13. Diagrama de paquetes del sistema.....	77
Figura 14. Fragmento del código fuente en la parte del servidor.....	83
Figura 15. Fragmento del código fuente del lado del servidor.....	84
Figura 16. Código fuente para crear nuevo software.....	85
Figura 17. Código fuente para listar softwares.....	86
Figura 18. Código fuente para la eliminación de softwares.....	87
Figura 19. Código fuente para guardar tareas de un usuario.....	88
Figura 20. Código fuente para almacenar datos de evaluación.....	89
Figura 21. Código fuente para calcular la eficacia.....	90
Figura 22. Código fuente para calcular la eficiencia.....	91
Figura 23. Código fuente de calcular la satisfacción por puntajes.....	92
Figura 24. Cálculo de la satisfacción por comentarios.....	94
Figura 25. Código fuente para el cálculo de la satisfacción.....	95
Figura 26. Método para validar software analizado.....	95
Figura 27. Rutas y métodos para obtener datos de la BD.....	97
Figura 28. Código para ejecutar la aplicación Flask.....	98
Figura 29. Estructura del proyecto del lado del cliente.....	99
Figura 30. Código del componente App.js del lado del cliente.....	104
Figura 31. Código para crear un nuevo software a evaluar.....	105
Figura 32. Agregar nuevo software a evaluar, CU1.....	106
Figura 33. Notificación del software creado y listado, CU1.....	107
Figura 34. Eliminación del software y su evaluación, CU1.....	107
Figura 35. Notificación eliminación, listado actualizado, CU1.....	108

Figura 36. Panel de bienvenida, opciones e instrucciones para cada software, CU1.....	108
Figura 37. Creación de tabla para cargar datos, CU1.....	109
Figura 38. Carga y edición de valores en la tabla, CU2.....	109
Figura 39. Validación de los datos ingresados, CU2.....	110
Figura 40. Tabla editable con funciones de copiar, pegar, cortar, mover, etc, CU2.....	110
Figura 41. Datos guardados correctamente, CU2.....	111
Figura 42. Descarga de la plantilla de ejemplo, CU2.....	111
Figura 43. Cargue de datos en formato csv o tipo Excel, CU2.....	112
Figura 44. Datos guardados correctamente, CU2.....	112
Figura 45. Reporte con gráficas, CU3.....	113
Figura 46. Conclusiones en el reporte, CU3.....	113
Figura 47. Cargue de los comentarios de los usuarios en las preguntas abiertas, CU2.....	114
Figura 48. Reporte del nivel de sentimiento encontrado en los comentarios, CU3.....	115
Figura 49. Descripción detallada de los valores al pasar el cursor sobre la gráfica, CU3..	115
Figura 50. Reporte final de la usabilidad total obtenida del software evaluado, CU4.....	116
Figura 51. Vista inicial cuando no hay ningún software a evaluar, CU1.....	117
Figura 52. Entorno de la prueba de usabilidad del software con la participante 1.....	124
Figura 53. Entorno de la prueba de usabilidad del software con el participante 2.....	125
Figura 54. Entorno de la prueba de usabilidad del software con el participante 3.....	125
Figura 55. Entorno de la prueba de usabilidad del software con la participante 4.....	126
Figura 56. Entorno de la prueba de usabilidad del software con el participante 5.....	126
Figura 57. Creación del software a evaluar.....	135
Figura 58. Carga de datos en el aplicativo evaluador de usabilidad.....	136
Figura 59. Reporte de la eficacia por usuarios y por tareas.....	137
Figura 60. Reporte de la eficacia del software evaluado.....	138
Figura 61. Interfaz de OpenShot con el menú contextual sobre una transición.....	139
Figura 62. Interfaz de OpenShot para la exportación de videos.....	140
Figura 63. Reporte de la eficiencia por usuarios y por tareas.....	141
Figura 64. Reporte de la eficiencia del software evaluado.....	142
Figura 65. Reporte satisfacción en preguntas cerradas por usuarios y por preguntas.....	143
Figura 66. Reporte de la satisfacción basada en puntajes.....	144
Figura 67. Reporte del atributo satisfacción basada en comentarios.....	146
Figura 68. Reporte de usabilidad del software evaluado.....	147
Figura 69. Reporte satisfacción basado en comentarios netamente en español.....	151

LISTA DE ECUACIONES

Ecuación 1. Atributo Efectividad de la usabilidad.....	44
Ecuación 2. Atributo eficiencia de la usabilidad.....	44
Ecuación 3. Atributo satisfacción de la usabilidad.....	44

LISTA DE ANEXOS

ANEXOS.....	169
Anexo A. Enlaces de acceso al repositorio del aplicativo software desarrollado y Manual Del Usuario.....	169
Anexo B. Acuerdo de confidencialidad.....	170
Anexo C. Cuestionario pre-test.....	171
Anexo D. Cuestionario post-test.....	173
Anexo E. Respuestas del cuestionario pre-test.....	177
Anexo F. Respuestas del cuestionario post-test.....	178

RESUMEN

Este proyecto se enmarca en una investigación cualitativa, aplicada y descriptiva, cuyo objetivo fue mejorar la estimación de la satisfacción del usuario en pruebas de usabilidad mediante análisis de sentimientos aplicado a comentarios abiertos. Se desarrolló un sistema web con arquitectura cliente-servidor, utilizando tecnologías de software libre (ReactJS, Flask, VADER, TinyDB), lo que permitió una solución accesible, flexible y de bajo costo, adaptable a entornos académicos.

La metodología de gestión siguió la Estructura de Desglose de Trabajo (WBS) del PMBOK, organizada en tres paquetes: caracterización conceptual, diseño e implementación del sistema, y validación funcional. El sistema automatiza la gestión completa de pruebas de usabilidad: permite registrar software, cargar datos de usuarios, calcular automáticamente eficacia, eficiencia y satisfacción, analizar comentarios mediante técnicas de procesamiento de lenguaje natural, y generar reportes detallados en tiempo real.

Durante la prueba aplicada al software OpenShot, el sistema estimó un 73 % de eficacia, 81 % de eficiencia y 70 % de satisfacción global, esta última calculada como promedio entre respuestas cerradas (65 %) y análisis de sentimientos (76 %). Estos resultados superaron el 72,9 % obtenido con métodos tradicionales, evidenciando que la incorporación del análisis de sentimientos mejora la precisión de la estimación al cuantificar percepciones subjetivas de forma objetiva. Un hallazgo clave en este proceso fue que el modelo VADER, al analizar comentarios en español, arrojó inicialmente un 46 % de satisfacción; tras aplicar una traducción automática al inglés con Argos Translate, el valor aumentó a 76 %, validando la confiabilidad del método cuando se adapta al idioma del modelo.

Además de validar el modelo propuesto, el proyecto entregó un aplicativo que cumple con los requisitos funcionales, implementado según la arquitectura prevista y validado con datos reales. Se concluye que el análisis de sentimientos mejora la calidad y objetividad de la evaluación de usabilidad, permitiendo una estimación más rica, automatizada y reproducible.

ABSTRACT

This project is framed within a qualitative, applied and descriptive research, aimed at improving the estimation of user satisfaction in usability testing through sentiment analysis applied to open-ended comments. A web-based system with a client-server architecture was developed using free and open-source technologies (ReactJS, Flask, VADER, TinyDB), resulting in an accessible, flexible, and low-cost solution, adaptable to academic settings.

The project management methodology followed the Work Breakdown Structure (WBS) approach of the PMBOK, organized into three work packages: conceptual characterization, system design and implementation, and functional validation. The system automates the entire usability testing process: it allows for registering software, uploading user data, automatically calculating effectiveness, efficiency, and satisfaction, analyzing comments using natural language processing techniques, and generating detailed real-time reports.

In the usability test conducted on OpenShot software, the system estimated 73% effectiveness, 81% efficiency, and 70% overall satisfaction—the latter calculated as the average between closed-ended responses (65%) and sentiment analysis results (76%). These outcomes exceeded the 72.9% obtained through traditional methods, demonstrating that incorporating sentiment analysis enhances the accuracy of satisfaction estimation by objectively quantifying users' subjective perceptions. A key finding in this process was that the VADER model initially produced a 46% satisfaction rate when analyzing comments in Spanish. After applying automatic translation to English using Argos Translate, the result increased to 76%, validating the reliability of the approach when aligned with the model's language capabilities.

In addition to validating the proposed evaluation model, the project delivered a functional software application, implemented according to the defined architecture and tested with real data. It is concluded that sentiment analysis not only automates but also enriches the estimation of user satisfaction, contributing to more accurate, objective, and reproducible usability evaluations.

1. INTRODUCCIÓN

La Ingeniería de Software tiene como principio fundamental la creación de software de calidad. En este contexto, la calidad se define como el conjunto de características y propiedades de un producto software orientadas a satisfacer requisitos específicos previamente definidos (Cuervo et al., 2017). Para su aseguramiento, el estándar internacional ISO/IEC 25010 (2011) establece atributos clave de calidad, entre ellos la usabilidad, entendida como el grado en que un producto software puede ser utilizado por usuarios específicos para alcanzar objetivos definidos con eficacia, eficiencia y satisfacción en un contexto determinado (ISO 9241-11, 2018).

De los tres componentes de la usabilidad, la satisfacción es el único considerado subjetivo, ya que se basa en percepciones del usuario (Castro, 2016; Bowman et al., 2002). Esta se define como la ausencia de incomodidad y la presencia de actitudes positivas frente al uso del sistema (ISO 9241-11, 2018). En laboratorios de usabilidad, la satisfacción se mide mediante cuestionarios que incluyen preguntas cerradas y abiertas (Cancio y Bergues, 2013). Las preguntas abiertas permiten al usuario expresar valoraciones subjetivas mediante comentarios con adjetivos calificativos (Castro, 2016), aunque su análisis implica un reto debido a la dificultad de cuantificarlas y reducir el sesgo del evaluador (Hone y Graham, 2001).

Frente a estas limitaciones, técnicas como el análisis de sentimientos permiten procesar datos cualitativos mediante la extracción de polaridades textuales (Mishev et al., 2020; Vilares et al., 2013). Esta técnica automatiza el análisis de opiniones expresadas en texto, generando valoraciones positivas, neutras o negativas (Balahur et al., 2013).

1.1. Planteamiento del problema

El proceso de estimación de la satisfacción del usuario en pruebas de usabilidad presenta retos significativos, especialmente al interpretar comentarios abiertos de forma cuantitativa. Esta tarea es compleja, propensa al sesgo y difícil de escalar cuando se realiza manualmente (Retnani et al., 2017; Romero et al., 2021). Asimismo, se reconoce que la

dimensión emocional del usuario influye en su satisfacción, por lo que el análisis de sentimientos emerge como un enfoque complementario para identificar actitudes positivas o negativas (Montero, 2006; Schmidt et al., 2020).

A pesar de propuestas como el monitoreo de expresiones faciales (Delgado et al., 2018), marcos de evaluación automatizada (Dingli y Cassar, 2014), o estudios metodológicos sobre procesamiento de lenguaje natural en pruebas de usabilidad (Mendes et al., 2013), no se evidencia la existencia de una herramienta capaz de clasificar automáticamente las opiniones de los usuarios en pruebas de usabilidad. Por ello, este proyecto desarrolló una herramienta para categorizar dichas opiniones utilizando análisis de sentimientos, facilitando así la estimación automatizada de la satisfacción.

1.2. Formulación del problema

Frente a la necesidad de optimizar y automatizar la medición de la satisfacción del usuario, se formuló la siguiente pregunta de investigación:

¿Cómo mejorar el proceso de estimación de la satisfacción del usuario en las pruebas de usabilidad mediante el uso del análisis de sentimientos sobre los datos cualitativos de los cuestionarios de usabilidad para contribuir a la calidad del software?

Asimismo, se propuso la siguiente hipótesis:

El desarrollo de un aplicativo software basado en algoritmos de análisis de sentimientos para valorar opiniones cualitativas de los cuestionarios de usabilidad permite mejorar el proceso de estimación de la satisfacción del usuario en las pruebas de usabilidad para contribuir a la calidad del software.

1.3. Justificación

Existen diversas propuestas de herramientas para evaluar la usabilidad, como las sugeridas por El-Halees et al. (2017) para análisis de texto en sitios web, por Dingli y Cassar (2014) en la automatización de pruebas, o por Delgado et al. (2018) para interpretar expresiones

faciales. Sin embargo, no se encuentra evidencia de herramientas diseñadas para determinar automáticamente la polaridad de comentarios abiertos en contextos de evaluación de usabilidad.

Con el fin de contribuir a ese vacío, este proyecto propuso una herramienta web que automatiza el análisis de comentarios mediante la técnica de análisis de sentimientos, reduciendo el sesgo humano y facilitando el trabajo del evaluador. Su implementación permitió avanzar en la automatización de pruebas de usabilidad y optimizar su escalabilidad mediante la posibilidad de replicar evaluaciones en entornos virtuales.

En el entorno empresarial, las pruebas automatizadas aportan al análisis e identificación temprana de fallas, aumentando la calidad del software entregado (Liu et al., 2020). Además, al mejorar la medición de la satisfacción, se fortalece la confianza del usuario final (Poth y Sunyaev, 2013; Shen et al., 2018). En el campo académico, el proyecto representa un avance metodológico que permite caracterizar procesos de evaluación y generar una herramienta documentada con aplicación potencial en la formación de ingenieros de software.

Por otro lado, para el escenario profesional, este proyecto representa un aporte relevante a la disciplina de la Ingeniería de Software, ya que permite medir de forma objetiva y automatizada un atributo subjetivo como la satisfacción del usuario. Al integrar una técnica de análisis de sentimientos, se fortalece el alineamiento con estándares internacionales como la ISO 9241-11 (2018), promoviendo mejores prácticas en la validación de productos software.

Como parte del beneficio comunitario local, la herramienta propuesta puede ser utilizada por estudiantes e investigadores del programa de Ingeniería de Sistemas de la Universidad de Cartagena, así como por grupos de investigación enfocados en el desarrollo de software, como una alternativa económica, reproducible y centrada en la usabilidad.

1.4. Contexto de la investigación

La investigación se llevó a cabo en la Universidad de Cartagena, departamento de Bolívar – Colombia, como trabajo de grado en el marco del grupo de investigación DaToS, dentro de la línea de Interacción Humano-Computador. El objetivo fue desarrollar una herramienta web para automatizar la estimación de la satisfacción del usuario en pruebas de usabilidad, mediante análisis de sentimientos.

El proyecto, de enfoque aplicado, se fundamentó en los principios establecidos por la norma ISO 9241-11:2018, relacionada con la ergonomía de la interacción humano-sistema y la evaluación de la usabilidad. Se desarrolló en un periodo de siete meses utilizando tecnologías de software libre. La implementación se estructuró en tres fases: caracterización de los procesos de evaluación de usabilidad, desarrollo del sistema e integración tecnológica, y validación funcional en un entorno virtual. Las pruebas se realizaron de forma remota, con cinco usuarios ubicados en sus respectivos domicilios, bajo la supervisión de un único evaluador.

La herramienta demostró ser útil para complementar la medición de la satisfacción del usuario mediante análisis de sentimientos, con aplicaciones potenciales en distintos contextos de evaluación de experiencia de usuario. En el ámbito académico, puede ser incorporada como recurso práctico en la asignatura de Interacción Humano-Computador (HCI), fortaleciendo el enfoque aplicado en la formación de estudiantes. Se identificaron como limitación algunas dependencias tecnológicas ligadas al uso de bibliotecas de código abierto.

1.5. Objetivos y alcance

1.5.1. Objetivo general

Desarrollar un aplicativo web que contribuya a la automatización del proceso de estimación del atributo satisfacción del usuario en las pruebas de usabilidad de software, mediante el uso de la técnica de análisis de sentimientos.

1.5.2. Objetivos específicos

1. Caracterizar los procesos de evaluación de usabilidad desarrollados en un laboratorio de usabilidad.
2. Diseñar e implementar los módulos funcionales que componían el aplicativo web, a partir de la caracterización realizada.
3. Verificar la funcionalidad del aplicativo construido mediante el desarrollo de una prueba de usabilidad sobre un software determinado.

2. MARCO DE REFERENCIA

2.1. Estado del arte

2.1.1. Antecedentes bibliográficos

Entre los aportes a nivel investigativo en cuanto a la medición de la satisfacción en las pruebas de usabilidad encontramos los aportes de Nielsen (1990), quien crea patrones de usabilidad con la publicación de lo que llama Heurísticas de Nielsen, sobre cómo favorecer el nivel de usabilidad. Por otro lado, Dray y Siegel (2004) proponen un marco metodológico para la aplicación de pruebas de usabilidad de manera remota mediante uso de herramientas web. Y, por su parte, Mendes y otros (2013) en el que se limitan a presentar un estudio conceptual del uso beneficioso del procesamiento del lenguaje natural en las pruebas de usabilidad en las redes sociales.

Más recientemente, Hussain y Mohamed (2020) hicieron un estudio para determinar las dimensiones que tienen un impacto significativo en la satisfacción de los usuarios con discapacidad visual moderada y severa que utilizan aplicaciones móviles. En dicho estudio propusieron un modelo de evaluación de la usabilidad de aplicaciones móviles. En ese mismo año, Davis y otros (2020), hicieron una revisión bibliográfica sistemática y encontraron que los métodos de evaluación de la usabilidad de los sistemas software para una determinada patología incluyeron: seguimiento ocular, cuestionarios, entrevistas semiestructuradas, entrevistas contextuales, protocolos de pensamiento en voz alta, recorridos cognitivos, evaluaciones heurísticas y revisiones de expertos, grupos focales y escenarios.

Sobre la medición automatizada de la usabilidad en sitios web, recientemente, Namoun y otros (2021) revelaron numerosos problemas críticos: la usabilidad parece ser ignorada o distorsionada por la mayoría de las herramientas automatizadas, las cuales exhibieron puntajes variables y contradictorios con respecto al análisis de los mismos sitios web seleccionados para el estudio. Entre otras cosas, hallaron que el funcionamiento interno de las herramientas no parece incorporar los fundamentos teóricos de la usabilidad, lo que

exige una colaboración urgente entre los expertos de la industria y los profesionales e investigadores de HCI.

Sin embargo, a pesar de las bondades conceptuales y metodológicas propuestas, la mayoría de las investigaciones recientes carecen de un marco metodológico específico para la evaluación de la usabilidad mediante análisis de sentimientos.

2.1.2. Antecedentes internacionales

Actualmente, como parte de la solución ofrecida evidenciaron algunos aportes como alternativas para la evaluación de la satisfacción de usuario en las pruebas de usabilidad de software, por ejemplo, en el 2013 Mascheroni y otros, desarrollaron una herramienta para facilitar la evaluación de la usabilidad durante el proceso de desarrollo de software, basada en la aplicación de los estándares y criterios heurísticos sobre usabilidad, vigentes hasta ese año. No obstante, el prototipo sólo ofrece una elección votación numérica ante ciertas preguntas hechas por el evaluador, dejando de lado la posibilidad de analizar las opiniones subjetivas de los usuarios.

Después, los académicos Dingli y Cassar (2014) propusieron un marco de trabajo para la evaluación automatizada de la usabilidad en sitios web en que se propone automatizar el proceso de evaluación de la usabilidad mediante el empleo de una técnica de evaluación heurística de manera inteligente mediante la adopción de varios métodos de inteligencia artificial como la regresión lineal o algoritmos clasificatorios basados en la investigación. Sin embargo, aunque los resultados experimentales mostraron que existe una alta correlación entre la herramienta y los anotadores humanos al identificar las violaciones de usabilidad consideradas, estas mediciones carecen de la percepción subjetiva del usuario final por lo que no ofrece una completa valoración de la satisfacción del usuario en otros contextos de software.

En otro escenario, Díaz y Valderrama (2018) propusieron un método basado en la lógica difusa para cuantificar de forma más exacta la valoración de la experiencia de usuario en la medición de la usabilidad de los entornos virtuales de aprendizaje. No obstante, el estudio

llevado a cabo no presentó una herramienta que automatice el proceso de evaluación, pues se limita a presentar un modelo matemático que toma como base los datos cualitativos para procesarlos de manera cuantitativa.

Más recientemente, Pal y Vanijja (2020) hicieron un estudio de la usabilidad percibida de la plataforma Microsoft Teams por más de 1500 estudiantes jóvenes de diversas universidades y perfiles académicos en el contexto de la pandemia del COVID-19. Para ello usaron la Escala de Usabilidad del Sistema (SUS, por sus siglas en inglés) la cual se basa en un listado de 10 preguntas con rangos calificativos numéricos del 1 al 5. Obtuvieron, entre otros hallazgos, que la usabilidad de la plataforma es alta en diferentes dispositivos. A pesar de las bondades del estudio, se evidencia que las opiniones abiertas no fueron consideradas en el cuestionario de usabilidad, por ende, se refleja una carencia de las valoraciones subjetivas de los usuarios.

2.1.3. Antecedentes nacionales y regionales

En Colombia, los investigadores Claro y Collazos (2006) hicieron un estudio en el que evaluaron sitios web oficiales de algunas entidades públicas, basándose en una jerarquía de tres niveles para la evaluación de la usabilidad, es decir, en criterios, métricas y atributos. Luego de hacer mediciones basadas en procesos no automatizados hallaron que el nivel de usabilidad presentado por las páginas web del Gobierno Colombiano no es apropiado para la ciudadanía objetivo. Entre otros resultados encontraron falencias en la presentación de contenidos, manejo de errores y aspectos de la navegación por los sitios, además no se evidenciaron estrategias de recordación o pedagogía que facilitara a los usuarios entender y aprovechar los recursos dispuestos por estas instituciones. En ese sentido, los autores concluyeron que es mucho el trabajo por desarrollar en el área de usabilidad para Colombia. Sin embargo, a pesar de los esfuerzos y conclusiones como aportes, no se evidenció el uso de una herramienta para la automatización del proceso de evaluación de la usabilidad de sitios web, y tampoco se tuvieron en cuenta las opiniones de los usuarios.

Otra contribución en el ámbito de la usabilidad fue hecha por Ocampo y otros (2014), quienes, desde una examinación de la página de la Universidad de Nacional de Colombia

Sede Manizales, establecieron una valoración numérica de ciertas métricas de usabilidad haciendo uso de árboles de decisión y basado en criterios de expertos. Una desventaja de este método, cómo ellos mismos lo exponen, es la complejidad de hallar la ponderación para diferentes sitios web puesto que cada uno tiene un enfoque específico. Por otro lado, los pesos de las aristas corresponden a calificativos basados en datos cualitativos que no tomaban en cuenta las valoraciones de los usuarios del sitio web analizado.

Por otra parte, Delgado y otros (2018) propusieron un método automatizado para monitorear el comportamiento emocional de un usuario durante una prueba de usabilidad en que la expresión facial del usuario es una variable de seguimiento. No obstante, la estimación de la satisfacción del usuario carece de un componente que tenga en cuenta el uso de las opiniones de los usuarios.

Por último, se pueden destacar las recientes investigaciones en el área de la evaluación de la usabilidad de software por parte de investigadores de la Universidad de Cartagena y otras universidades colombianas. En primera instancia, Chanchí, Campo y Sierra (2019) propusieron el uso de la computación afectiva al incorporar un aplicativo software para el análisis de sentimientos que obtiene la polaridad y la subjetividad de un estudio de satisfacción de usabilidad de los usuarios de una aplicación, teniendo como resultados indicadores más objetivos para la estimación de la satisfacción. También, Chanchí y otros (2020) presentaron una herramienta que ofrece la posibilidad de evaluar cada uno de los criterios asociados a los principios heurísticos para videojuegos de Pinelle, así como generar de manera automática gráficas explicativas del cumplimiento de los objetivos del software. Y, asimismo, Chanchí y otros (2022) con el fin de determinar el nivel de criticidad de los diferentes problemas identificados dentro de una inspección de usabilidad, propusieron una herramienta basada en lógica difusa para la determinación del porcentaje de criticidad, a partir de los valores de entrada de severidad y frecuencia de cada problema, basándose en el lenguaje FCL (Fuzzy Control Language), en el cual obtuvieron el nivel de criticidad porcentual de los problemas de usabilidad identificados en una evaluación heurística.

A pesar de las bondades de las herramientas propuestas por los autores mencionados, se hace necesario mencionar que ninguna de estas abarca la automatización de pruebas de usabilidad ni tampoco permiten hacer pruebas de usabilidad remota, puesto que se hacen especificación a software de determinados enfoques como los videojuegos, cerrando la posibilidad de adaptarse a la medición de la usabilidad en otros contextos.

2.1.4. Antecedentes a nivel de software comercial

Diversas herramientas comerciales para la evaluación de la usabilidad del software fueron encontradas en la investigación:

- ❖ **Maze:** Esta es una herramienta web que ofrece la administración de pruebas de usabilidad mediante diversos tipos de preguntas, permite conectarse con las plataformas de creación de prototipos más conocidos, y además ofrece reportes . En cuanto al costo, ofrece una versión mínima gratis con muy pocas funcionalidades. La opción menos costosa está en USD \$25 por mes, con algunas opciones como 10 proyectos activos, 250 respuestas por mes, protección por contraseña, exportar formato CSV (archivo con valores separados por comas), etc. (Maze, 2021).
- ❖ **Userback:** Es una herramienta de comentarios de sitios web para recopilar comentarios visuales con videos y capturas de pantalla anotadas de un sitio web o diseño. Ofrece administrar los comentarios de los clientes, informes de errores, exámenes de aceptación (UAT, por sus siglas en inglés), registro de usuarios, integración con herramientas de gestión productiva, entre otras funciones. El plan estándar está en el valor de USD \$39 por mes, con las funciones de colaborar en múltiples proyectos, 5 usuarios, reportes ilimitados, integración con apps de diseño, entre otras (Userback, 2021).
- ❖ **Usabilla:** Plataforma lanzada en 2009 y adquirida por SurveyMonkey en 2019. Está fraccionada en diversas herramientas, APIs y soluciones para recibir retroalimentación de usuarios de sitios web, aplicaciones móviles, email y diseños. El proceso de contratación de la herramienta incluye llenar un formulario y esperar

respuesta mediante correo electrónico en el cual ofrecen planes, precios, tutoriales y soluciones a la medida.

- ❖ **Useberry:** Herramienta web que permite crear cuestionarios de usabilidad con la posibilidad de diversos métodos de prueba como prueba de cinco segundos, prueba de referencia, prueba de árbol, tarea única, tareas múltiples y preguntas de satisfacción mediante escala numérica. Ofrece una versión gratuita con mínimas posibilidades, y tiene a disposición planes básico, Pro y Equipo. La versión básica tiene un costo de USD \$33 por mes con las ventajas o limitaciones de 100 respuestas por mes, 3 proyectos, colaboradores ilimitados, exportar a CSV, idiomas personalizados, manejo de versiones de proyecto.
- ❖ **TryMyUI:** Portal soportado por prueba de usuarios desconocidos quienes opinan sobre un software en específico, ofreciendo la posibilidad de escoger el perfil de los encuestados. La herramienta ofrece las siguientes funciones: prueba de usuario remoto, prueba de usuario móvil, pruebas de prototipos, prueba de impresiones, encuesta escrita, entre otros, con un precio de USD \$100 la mensualidad para el plan más básico.
- ❖ **GetFeedback:** Herramienta digital para la recopilación de comentarios de usuarios. Se centra más en la experiencia del cliente en general, pero bien se puede aplicar al contexto de pruebas de usabilidad en el diseño y desarrollo de software. Ofrece integración guiada con software de manejo de clientes como Salesforce y otros. Entre otras funcionalidades proporciona obtener frases clave, temas y opiniones, sin embargo, se basa en una previa calificación mediante una escala de cinco de puntos para hacer la clasificación de la polaridad de la opinión del usuario. Los costos de suscripción a cada uno de sus planes son entregados mediante correo electrónico.

A pesar de las utilidades de cada uno de los portales examinados, se evidenció en cada uno la ausencia de la medición de la satisfacción mediante la conversión de los comentarios de los usuarios a valores cuantitativos, impidiendo la polarización de dichos comentarios de acuerdo al contenido sintáctico que poseen.

2.2. Marco teórico

2.2.1. Ingeniería de Software

La Ingeniería de Software ha evolucionado durante los últimos 50 años, inicialmente como respuesta a la llamada crisis del software (los problemas que tenían las organizaciones para producir sistemas de software de calidad a tiempo y dentro del presupuesto) de las décadas de 1960 y 1970 (Arndt, 2018).

La Ingeniería de Software se ha definido como “la aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación y mantenimiento de software” (IEEE, 1990).

A lo largo de la historia de la Ingeniería de Software se han planteado diversas metodologías para el desarrollo de productos software. Básicamente, la mayoría de dichos enfoques metodológicos conllevan una serie de etapas que conforman el proceso de desarrollo: ingeniería de requerimientos, diseño arquitectónico, desarrollo, validación e implementación (Ojeda y Fuentes, 2012). Estas metodologías determinan los pasos a seguir y cómo realizarlos para crear prototipos del producto software. Algunas de las metodologías de desarrollo son Cascada, Espiral, RUP, Programación Extrema, SCRUM, entre otras, los cuales se organizan en modelos tradicionales, evolutivos, iterativos y ágiles (Cadavid, Martínez y Vélez, 2013).

Uno de los aspectos más importantes de la Ingeniería de Software es el tema de la calidad del software.

2.2.2. Calidad del software

La calidad del software se define como el conjunto de atributos presentes en un producto y, en su caso, el nivel del atributo para el cual el usuario de software (cliente) tiene un valor positivo (Planning, 2002).

La industria del software reconoce principalmente la importancia del software de alta calidad, robusto, confiable y fácil de mantener. La industria siempre exige soluciones eficientes que pueden mejorar la calidad del software (Malhotra, 2019).

En ese sentido, se han elaborado diversos estándares de calidad los cuales contienen una lista de atributos de calidad. La siguiente lista es elaborada por Castro, Solarte y Muñoz (2019):

- ❖ ISO 9001: Es el encargado de medir la importancia del software y los procesos productivos de la organización. Por ejemplo, en la entrega y mantenimiento del software.
- ❖ ISO/IEC 9003: Es un conjunto de tareas y procedimientos que han tenido éxito en el concepto de software sobre los procesos de la organización. Este modelo está orientado a la Ingeniería de Software, sirve como indicador de la norma ISO 9001:2000.
- ❖ ISO/IEC 12207 (Information Technology / Software LifeCycle Processes): son normas que se utilizan para el ciclo de vida del proceso de software de una organización, además sirve de apoyo para ISO 15504-SPICE.
- ❖ ISO/IEC 15504 SPICE (Software Process Improvement and Assurance Standards Capability Determination). Este estándar está conformado por 7 modelos que sirven de base para la implantación, mejoramiento (capacidad) y madurez de los procesos de las organizaciones, se utilizan para la evaluación de la calidad de los procesos. La descripción de los métodos se realiza teniendo en cuenta la norma ISO/IEC 12207.
- ❖ ISO/IEC 9126 (Software engineering Product quality): creada entre 1991 y 2001. Esta norma está dividida en 4 etapas, la primera consiste en un conjunto de normas ISO/IEC 9126 que tiene que ver con la calidad del producto de software, la segunda y tercera etapa trabaja con la medición (métricas internas y externas), la cuarta está dedicada hacia la calidad de uso, de acuerdo con condiciones particulares.

- ❖ ISO/IEC 14598 (Software product evaluation): Creada entre 1999 y 2001. Está conformada por 6 partes que están interrelacionadas con la familia ISO 9126, esta es la encargada de la parte de evaluación de producto de software.
- ❖ ISO 25000. Hace parte del modelo de calidad para el producto de software, está conformado por 5 fases que se encuentran en desarrollo, fue creada para reemplazar la ISO 9126 e ISO 14598 ya que desde 2001 no se publicaron nuevas versiones.

Paralelamente, en torno a la calidad de software se han establecido modelos de calidad de software los cuales son una serie de atributos estructurados que conforman una serie de metodologías a seguir (Castro y otros 2019). Se pueden mencionar los siguientes, basándose en la recopilación hecha por los anteriores investigadores, como algunos de los ejemplares de muchos modelos de calidad:

- ❖ Modelo de McCall. Este modelo se dio a conocer en el año de 1977 y está dirigido hacia el producto final, se dedica a la identificación de atributos claves a partir de la perspectiva del usuario, estos atributos se conocen como factores de calidad, estos comúnmente son factores externos, pero en algunos casos también se tienen en cuenta factores internos.
- ❖ Modelo de Boehm. Se dio a conocer por Barry Boehm en el año 1978, uno de los aspectos más importantes es que introduce características de alto nivel, intermedio y primitivas donde cada uno aporta al nivel general de calidad.
- ❖ Modelo FURPS. Tiene las características de los modelos anteriores, pero incluye otros aspectos como funcionalidad, usabilidad, confianza, rendimiento, soportabilidad.
- ❖ Modelos ad-hoc. Este modelo sirve para el monitoreo de la calidad del software, existen dos caminos: Adoptar un camino fijo, se supone que todos los factores de calidad importantes son un subconjunto de los de un modelo publicado, se acepta el conjunto de criterios y métricas asociados al modelo. Desarrollar un modelo propio de calidad, en el cual se acepta que esté compuesta por varios atributos, pero no se

adopta lo impuesto por modelos existentes. En este último caso, se debe consensuar el modelo con los clientes antes de empezar el proyecto, se deciden cuáles atributos son importantes para el producto y las medidas específicas que los compone.

Por otro lado, para el aseguramiento de la calidad del software se hace necesario aplicar el concepto prueba de la calidad del software. Según Dijkstra (1970), la prueba de software puede ser usada para mostrar la presencia de errores, pero nunca su ausencia. En ese sentido los autores de la IEEE Bourque, Dupuis, Abran, Moore, y Tripp (1999) sostienen que la prueba de la calidad es una actividad realizada para evaluar la calidad del producto y mejorarla, identificando defectos y problemas. Por otro lado, Mera (2016) afirma que la prueba de la calidad de software es la verificación dinámica del comportamiento de un programa contra el comportamiento esperado, usando un conjunto finito de casos de prueba, seleccionados de manera adecuada.

2.2.3. Usabilidad

En la actualidad, la usabilidad y el diseño centrado en el usuario se han convertido en cuestiones fundamentales para garantizar la calidad y el éxito de un proyecto de software (Cayola, 2018).

La usabilidad determina qué tan fácil es lograr una tarea por parte del usuario que usa una aplicación. La usabilidad se conoce como la facilidad de uso e idoneidad de un sistema para una clase específica de usuarios que realizan tareas específicas en un lugar preciso (Zahra, 2017).

Como se puede comprobar, en esta definición se liga la usabilidad de un sistema a usuarios, necesidades y condiciones específicas. Por tanto, la usabilidad del sistema no es un atributo inherente al software, no puede especificarse independientemente del entorno de uso y de los usuarios concretos que vayan a utilizar el sistema (Alarcón, Hurtado, Pardo, Collazos, y Pino (2007).

La usabilidad es una cualidad demasiado abstracta como para ser medida directamente. Para poder estudiarla se descompone habitualmente en los siguientes cinco atributos básicos (Nielsen, 1990):

- ❖ **Facilidad de aprendizaje:** Cuán fácil es aprender la funcionalidad básica del sistema como para ser capaz de realizar correctamente la tarea que desea realizar el usuario. Se mide normalmente por el tiempo empleado con el sistema hasta ser capaz de realizar ciertas tareas en menos de un tiempo dado.
- ❖ **Eficiencia:** El número de transacciones por unidad de tiempo que el usuario puede realizar usando el sistema. Lo que se busca es la máxima velocidad de realización de tareas del usuario. Cuanto mayor es la usabilidad de un sistema, más rápido es el usuario al utilizarlo, y el trabajo se realiza con mayor rapidez. Nótese que eficiencia del software en cuanto su velocidad de proceso no implica necesariamente eficiencia del usuario en el sentido en el que aquí se ha descrito.
- ❖ **Recuerdo en el tiempo:** Para usuarios intermitentes (que no utilizan el sistema regularmente) es vital ser capaces de usar el sistema sin tener que aprender cómo funciona partiendo de cero cada vez. Este atributo refleja el recuerdo acerca de cómo funciona el sistema que mantiene el usuario, cuando vuelve a utilizarlo tras un periodo de no utilización.
- ❖ **Tasa de errores:** Este atributo contribuye de forma negativa a la usabilidad de un sistema. Se refiere al número de errores cometidos por el usuario mientras realiza una determinada tarea. Un buen nivel de usabilidad implica una tasa de errores baja. Los errores reducen la eficiencia y satisfacción del usuario, y pueden verse como un fracaso en la transmisión al usuario del modo de hacer las cosas con el sistema.
- ❖ **Satisfacción:** Éste es el atributo más subjetivo. Muestra la impresión subjetiva que el usuario obtiene del sistema.

Algunos de estos atributos no contribuyen a la usabilidad del sistema en la misma dirección, pudiendo ocurrir que el aumento de uno de ellos tenga como efecto la

disminución de otro. La usabilidad del sistema no es una simple adición del valor de estos atributos, sino que se define para cada sistema como un nivel a alcanzar para algunos de ellos (Cancio y Bergues, 2013).

Con respecto a la evaluación de usabilidad, Ferré (2000) afirma que ésta permite conocer el nivel de usabilidad que alcanza el prototipo actual del sistema, e identificar los fallos de usabilidad existentes, y añade que la evaluación se realiza usualmente mediante test de usabilidad, complementados con evaluaciones heurísticas.

Los test de usabilidad son la práctica de usabilidad más extendida. Consisten en presentar al usuario una serie de tareas a realizar, y pedirle que las realice con el prototipo del sistema. Las acciones y comentarios del usuario se recopilan para un análisis posterior. Para conseguir unos test de usabilidad con resultados fiables, las condiciones del test y del lugar donde éste se realiza deben ser lo más parecidas posibles al entorno de uso previsto para el sistema (Ferré, 2000).

2.2.4. Satisfacción del usuario

La satisfacción del usuario es un factor clave para garantizar el éxito de cualquier servicio en línea. Entre los diferentes determinantes de la satisfacción del usuario, la calidad de experiencia apareció en la década de 2000 como una métrica para evaluar la calidad del servicio percibida por el usuario (Najjar, 2021). En la misma línea, Doll y Torkzadeh (1988), definieron la satisfacción del usuario como “la forma en que los usuarios se sienten con respecto al uso de ciertas aplicaciones informáticas. Estos sentimientos están estrechamente relacionados con la utilidad del sistema para abordar todas las necesidades del cliente.”

La satisfacción está asociada al confort y a la existencia de actitudes positivas durante la interacción, por ende tiene un carácter subjetivo (Enriquez y Casas, 2013). De este modo, la emocionalidad de un usuario se constituye como un factor relevante en el proceso de la interacción, estableciendo la percepción del usuario con respecto a un sistema interactivo (Delgado y otros, 2018).

Dentro de los atributos de la usabilidad, la satisfacción del usuario es el atributo más subjetivo. Éste muestra la impresión subjetiva que el usuario obtiene del sistema. Para ello se utilizan cuestionarios, encuestas y entrevistas, diseñados especialmente para recabar un cierto grado de satisfacción en función de aspectos predefinidos (Mascheroni, Greiner, Petris, Dapozo y Estayno, 2012).

2.2.5. Estimación de la satisfacción del usuario

La satisfacción del usuario se estima mediante técnicas como cuestionarios aplicados tras el uso del sistema. Según Monedero, Cebrián y Desenne (2015), esta medición puede aplicarse a sitios web o sistemas informáticos. Serrano y Cebrián (2014) destacan diversas escalas y tipos de cuestionarios utilizados comúnmente para este propósito:

- ❖ El cuestionario SUS (System Usability Scale): Quizás es uno de los cuestionarios más conocidos por conjugar el número reducido de preguntas y precisión. Mide la usabilidad de una herramienta, programa informático, instrumento, etc. Está compuesto por diez ítems a valorar en una escala Likert del 1 al 5, donde el 1 significa “totalmente en desacuerdo” y 5 “totalmente de acuerdo”. Al total de puntuaciones se le aplican las transformaciones necesarias para presentarlo en una escala del 1 al 100. Necesita poco tiempo para ser contestado, es simple de calificar y fácilmente es comparable con otros instrumentos. Se recomienda pasarlo una vez que los usuarios han trabajado con la aplicación o herramienta que se va a evaluar.
- ❖ El cuestionario QUIS (Questionnaire for User Interface Satisfaction): Este cuestionario permite la evaluación de la satisfacción de los usuarios. Se diferencia de los cuestionarios SUS en que se aplica mientras se está utilizando el software, instrumento, servicio, etc. Se desarrolló a finales de los años 80, y actualmente se están desarrollando versiones mejoradas del mismo. Los ítems se presentan en una escala de 0 al 9, siendo 0 confuso y el 9 claro. Consta de 5 categorías de preguntas: Reacciones generales sobre el software, las ventanas, la terminología e información del sistema, el aprendizaje y las capacidades del sistema.

- ❖ El cuestionario USE (Usefulness, Satisfaction and Ease of Use): Este cuestionario no sólo mide la usabilidad, sino también la utilidad y la satisfacción de los usuarios. Es uno de los más completos al evaluar la satisfacción, usabilidad y utilidad, además de ser muy simple de implementar al igual que SUS. Consta de 30 ítems en una escala Likert de siete puntos, desde muy fuertemente de acuerdo con la máxima puntuación, a muy fuertemente desacuerdo. Posee también la posibilidad de adaptar las preguntas del cuestionario a necesidades particulares.
- ❖ El cuestionario SUMI (Software Usability Measurement Inventory): Es un cuestionario de pago que mide la usabilidad de un software, o servicio, puede ser utilizado para evaluar nuevos productos, realizar comparaciones desde versiones previas para llegar a nuevos y mejores desarrollos, siempre y como todos los demás instrumentos, desde la opinión del usuario final. Consta de 50 ítems a valorar en una escala del 1 al 3 (“desacuerdo”, “no lo sé” y “de acuerdo”).

Cada uno de estos cuestionarios permiten recolectar información sobre la valoración de la usabilidad de un sistema software evaluado y, a partir de ello, se usan técnicas como el análisis de sentimientos aplicada a opiniones abiertas para obtener finalmente la polaridad de la opinión expresada. Esta polaridad indicaría el nivel de satisfacción del usuario (Chanchí y otros, 2019).

2.2.6. Laboratorio de usabilidad

Un laboratorio de usabilidad es definido como el espacio físico compuesto de un área para el usuario y otro para los evaluadores donde se realizan evaluaciones de usabilidad y accesibilidad a productos, sistemas y dispositivos por medio de software equipos hardware especializados en el monitoreo, entre otros que registren el accionar del sujeto de prueba con el aplicativo a evaluar. (Delgado y otros, 2018).

En adición a lo anterior, un laboratorio de usabilidad debe estar diseñado para obtener indicadores que permitan medir los atributos de eficiencia, eficacia, y satisfacción (Palinko y otros, 2015).

2.2.7. Análisis de sentimientos

La definición aportada por Kaur y Mangat (2017) concibe el análisis de sentimientos, o también llamado minería de opinión, como el proceso de extracción de emociones, actitudes u opiniones de un fragmento de texto sobre un tema determinado, mediante estudio computacional. Según Medhat, Hassan y Korashy (2014), el análisis de sentimientos también se puede aplicar a audio, imágenes y videos; y aportan que existe una ligera distinción entre la minería de opinión y el análisis de sentimientos. Mientras la primera extrae y analiza la opinión de las personas sobre una entidad, la siguiente identifica el sentimiento expresado en un texto y luego lo analiza. Por lo tanto, el objetivo del análisis de sentimientos es encontrar opiniones, identificar los sentimientos que expresan y luego clasificar su polaridad.

Gracias a la técnica análisis de sentimientos podemos clasificar frases y textos en objetivos o subjetivos; positivos, negativos o neutros, pero además obtener la polaridad, con este tipo de análisis podemos detectar emociones como la alegría, la tristeza, la ira, el miedo, etc. (García López, 2022).

2.2.8. Relación entre emociones, usabilidad y experiencia de usuario

En el campo de la Interacción Humano-Computador (HCI), la experiencia de usuario (UX) ha sido tradicionalmente evaluada a través de la usabilidad, entendida como la combinación de eficacia, eficiencia y satisfacción (Nielsen, 1990). Si bien este enfoque ha sido ampliamente adoptado, investigaciones recientes han evidenciado que las emociones influyen directamente en la percepción de calidad de la interacción y en el grado de satisfacción del usuario, por lo que deben considerarse parte integral de la evaluación de UX (Agarwal y Meyer, 2009).

Agarwal y Meyer (2009) demostraron que, aun cuando no se observan diferencias en métricas tradicionales de usabilidad, las respuestas emocionales pueden revelar contrastes importantes en la experiencia. Thüring y Mahlke (2007) propusieron el modelo CUE (Componentes de la Experiencia del Usuario), en el que la usabilidad, la estética y la

emoción interactúan como dimensiones fundamentales. Sauer y Sonderegger (2009) complementaron esta visión al evidenciar que el diseño estético puede mejorar el estado emocional del usuario, afectando positivamente su evaluación subjetiva, incluso en prototipos de baja fidelidad.

Champney y Stanney (2007) introdujeron el perfilado emocional como técnica para interpretar la carga afectiva de la interacción y sus implicaciones en la experiencia. Desde la psicología del usuario, Saariluoma y Jokinen (2014) propusieron el modelo competencia–frustración, que vincula el comportamiento emocional con el desempeño percibido. Hibbeln et al. (2017), por su parte, demostraron que es posible inferir emociones negativas durante el uso del sistema a partir del análisis de movimientos del cursor, permitiendo una evaluación continua y no invasiva.

Finalmente, Sauer, Sonderegger y Schmutz (2020) propusieron un modelo integrador entre usabilidad, UX y accesibilidad, en el que las emociones tienen un papel transversal dentro de la experiencia interactiva. En conjunto, estos estudios consolidan la idea de que la satisfacción no solo se deriva del cumplimiento funcional, sino también de las emociones que emergen durante la interacción.

3. METODOLOGÍA

3.1. Tipo de investigación, diseño y enfoque

Esta investigación se tipificó como cualitativa, aplicada y descriptiva. Cualitativa, debido a que no se fundamentó en la estadística (Leavy, 2017). Aplicada, porque se pretendió resolver un determinado problema y se enfocó en la consolidación del conocimiento, con el propósito de enriquecer el desarrollo científico (Bordens y Abbott, 2018). Descriptiva porque se analizaron y detallaron las propiedades relevantes de la variable de estudio (Hernández y otros, 2014).

La variable de estudio en la presente investigación fue la satisfacción del usuario en las pruebas de usabilidad, la cual fue medida mediante la ponderación cualitativa de la percepción del usuario, a través del análisis de sentimientos aplicado a los comentarios y opiniones dejadas en los cuestionarios de usabilidad.

El proyecto se desarrolló entre el 2023 y el 2024, en el marco del grupo de investigación DaToS de la Universidad de Cartagena, sede Piedra de Bolívar, ubicada en el departamento de Bolívar – Colombia. Las actividades de desarrollo y validación se llevaron a cabo en modalidad virtual, empleando entornos de trabajo colaborativos en línea. Las pruebas de usabilidad se realizaron de forma remota, con usuarios ubicados en sus respectivos domicilios, garantizando condiciones controladas mediante el uso de herramientas de videoconferencia y recolección digital de datos.

Para la validación del sistema se aplicó una prueba de usabilidad a cinco participantes seleccionados por conveniencia, siguiendo la recomendación de Nielsen (1994), quien sostiene que cinco usuarios suelen ser suficientes para detectar la mayoría de los problemas de usabilidad. Se establecieron criterios de inclusión como el manejo básico de computadores y la disponibilidad para participar en modalidad remota. Durante la prueba, los participantes completaron tareas específicas mientras se recolectaban datos mediante cuestionarios cerrados tipo SUS, formularios para comentarios abiertos y herramientas integradas al sistema, como el módulo de análisis de sentimientos con VADER. La eficacia

se evaluó a partir del porcentaje de tareas completadas por cada usuario, la eficiencia por el tiempo promedio de ejecución de dichas tareas, y la satisfacción mediante la combinación de las puntuaciones obtenidas en los cuestionarios cerrados y los resultados del análisis emocional automatizado. Como referencia, se consideró exitoso un desempeño igual o superior al 75 %, valor comúnmente aceptado en estudios de usabilidad (Tullis & Albert, 2013). Para validar el funcionamiento del sistema, se compararon los resultados obtenidos por medios tradicionales con los generados por el sistema automatizado, observándose consistencia entre ambos enfoques. Esto respaldó la fiabilidad del sistema para estimar con precisión la satisfacción del usuario.

3.2. Técnicas de recolección de información

Se utilizaron técnicas de recolección de información como entrevistas a docentes, la revisión documental y la experimentación. La primera se basó en consultas virtuales y entrevistas informales sobre los aspectos técnicos del proyecto, solicitud de correcciones y tutorías sobre la investigación; la segunda hace referencia a los aportes teóricos consultados en fuentes científicas en base de datos bibliográficas, y también la consulta en internet de material de referencia; por último, la experimentación trata de las diferentes etapas de prueba de los entregables del proyecto y el prototipo funcional del sistema propuesto.

3.3. Análisis de los datos

Para el análisis de los datos se aplicó el análisis documental, con el cual se pudo realizar una operación objetiva con el fin de conocer, detallar y organizar elementos, significados y forma de los documentos encontrados con relación a un tema específico. Los datos obtenidos de las fuentes de información fueron transcritos, agrupados, interpretados, comparados, procesados en formatos como audios, textos, vídeos, enlaces web, hojas escritas en físico, etc, y fueron almacenados en archivos en diferentes medios como la nube, marcadores web, documentos de edición de texto, presentaciones de diapositivas, hojas de cálculo, papeles en físico, etc. Para el análisis de los mismos, se hicieron filtros, lecturas, comparaciones, cálculos estadísticos sencillos, entre otras técnicas de análisis de

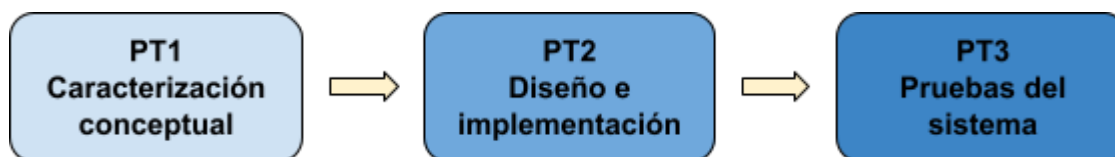
datos tanto cualitativos como cuantitativos, con el fin de obtener información relevante para el presente proyecto.

3.4. Metodología de gestión del proyecto

Para el desarrollo de este proyecto se tomó como referencia la Estructura de Desglose de Trabajo (WBS, por sus siglas en inglés) sugerida por el Project Management Institute (PMI), a través de su guía metodológica PMBOK (Guide to the Project Management Body of Knowledge), la cual facilitó la organización el proyecto mediante la subdivisión del mismo en bloques de trabajo (Devi y Reddy, 2012).

Según la WBS el trabajo planificado debe estar contenido en los niveles más bajos de sus componentes, denominados Paquetes de Trabajo, los cuales se pueden utilizar para agrupar actividades donde el trabajo es programado y estimado, seguido y controlado (Martínez, 2015). Por esto para llevar a cabo este proyecto, se propusieron tres paquetes de trabajo que derivan de los objetivos específicos, tal como se puede ver en la *figura 1*: PT1 (Caracterización conceptual), PT2 (Diseño e implementación del sistema) y PT3 (Pruebas del sistema). Éstos conllevan internamente una serie de actividades interrelacionadas.

Figura 1. Paquetes de trabajo como metodología de gestión del proyecto.



Fuente: elaboración propia.

3.4.1. PT1 - Caracterización conceptual

El primer paquete de trabajo abordó la caracterización de los procesos de evaluación de usabilidad en un laboratorio especializado, lo cual permitió cumplir el primer objetivo específico. Para ello, se desarrollaron tres actividades clave relacionadas con la obtención y análisis de información:

- ❖ **Actividad 1.1:** Definición e implementación de los métodos de recolección de información.
- ❖ **Actividad 1.2:** Documentación de los procesos de evaluación de la usabilidad, mediante la definición de los conceptos asociados y el diseño de flujos de procesos.
- ❖ **Actividad 1.3:** Selección de las tecnologías adecuadas para el desarrollo del producto software.

3.4.2. PT2 - Diseño e implementación del sistema

Por su parte, el segundo paquete de trabajo dio cumplimiento al segundo objetivo específico, el cual consistió en diseñar e implementar los módulos funcionales del aplicativo web a partir de la caracterización previa. A través de este paquete se consolidó una solución tanto conceptual como funcional del sistema, estructurada en una serie de actividades:

- ❖ **Actividad 2.1:** Establecimiento de los requisitos del sistema.
- ❖ **Actividad 2.2:** Diseño del modelo de negocios, el cual incluye diagramas de casos uso, diagrama de clase, esquema de base datos, y diagrama de paquete.
- ❖ **Actividad 2.3:** Desarrollo funcional del prototipo (back-end), diseño de interfaces de usuario (front-end) e integración.
- ❖ **Actividad 2.4:** Despliegue de prototipo del aplicativo en un servidor web.

3.4.3. PT3 - Pruebas del sistema

Finalmente, el tercer paquete de trabajo cumplió con el tercer objetivo específico, cuyo propósito fue verificar la funcionalidad del aplicativo desarrollado mediante la ejecución de una prueba de usabilidad sobre un software determinado. Para ello, se llevaron a cabo las siguientes actividades:

- ❖ **Actividad 3.1:** Diseño de los instrumentos a utilizar en la prueba de usabilidad.

- ❖ **Actividad 3.2:** Configuración del entorno controlado para pruebas de usabilidad.
- ❖ **Actividad 3.3:** Ejecución y documentación de pruebas sobre el software escogido al que se le hará una evaluación de usabilidad.
- ❖ **Actividad 3.5:** Análisis de los resultados obtenidos en la prueba de usabilidad.

Para la validación del sistema se aplicó una prueba de usabilidad a cinco participantes seleccionados por conveniencia, siguiendo la recomendación de Nielsen (1994), quien sostiene que cinco usuarios suelen ser suficientes para detectar la mayoría de los problemas de usabilidad. Se establecieron criterios de inclusión como el manejo básico de computadores y la disponibilidad para participar en modalidad remota. Durante la prueba, los participantes completaron tareas específicas mientras se recolectaban datos mediante cuestionarios cerrados tipo SUS, formularios para comentarios abiertos y herramientas integradas al sistema, como el módulo de análisis de sentimientos con VADER. La eficacia se evaluó a partir del porcentaje de tareas completadas por cada usuario, la eficiencia por el tiempo promedio de ejecución de dichas tareas, y la satisfacción mediante la combinación de las puntuaciones obtenidas en los cuestionarios cerrados y los resultados del análisis emocional automatizado. Como referencia, se consideró exitoso un desempeño igual o superior al 75 %, valor comúnmente aceptado en estudios de usabilidad (Tullis & Albert, 2013). Para validar el funcionamiento del sistema, se compararon los resultados obtenidos por medios tradicionales con los generados por el sistema automatizado, observándose consistencia entre ambos enfoques. Esto respaldó la fiabilidad del sistema para estimar con precisión la satisfacción del usuario.

4. RESULTADOS Y DISCUSIÓN

En esta sección de resultados y discusiones se presentan los procedimientos llevados a cabo para obtener los resultados de la presente investigación, siguiendo la metodología planteada la cual consta de tres paquetes de trabajos con actividades específicas para cada uno de ellos.

4.1. Paquete de trabajo 1: Caracterización conceptual

Una caracterización conceptual es definida como el acto de describir características distintivas o esenciales de un concepto (Aelenei y otros, 2016). En ésta se plasman definiciones y descripciones que pueden recurrir a la obtención de datos cualitativos y cuantitativos con el objetivo de aproximarse al conocimiento y comprensión de las estructuras, características, dinámicas, acontecimientos y experiencias asociadas a un objeto de interés (Sánchez Upegui, 2010).

Para el desarrollo de la caracterización se tomaron en cuenta los conceptos que representan el marco conceptual de la presente investigación y se consultaron fuentes de información. Los conceptos considerados pertinentes se caracterizan a continuación.

4.1.1. Usabilidad

A lo largo de la historia de la calidad del software, la usabilidad ha adquirido relevancia en la mayoría de los estándares de calidad y ha sido una preocupación constante de los desarrolladores enfocados en el usuario final (Downey, 2007). Las primeras definiciones de usabilidad surgieron en los inicios de la informática, cuando los investigadores comenzaron a estudiar cómo las personas interactúan con los ordenadores. En este contexto, la *tabla 1* presenta algunas definiciones del concepto de usabilidad, organizadas cronológicamente según la investigación de Grau (2007).

Tabla 1. Definiciones del concepto de usabilidad en los últimos años.

Autores	Año	Definición
Ronald A. Guillemette	1989	La usabilidad se refiere al grado de eficacia del probable uso de la documentación por parte de los usuarios finales durante la ejecución de tareas dentro de las restricciones y requerimientos del entorno real. Identifica los conceptos de eficacia y satisfacción del usuario, que se relacionan respectivamente con los conceptos de usabilidad y utilidad (Guillemette, 1989).
Jakob Nielsen	1994	Uno de los gurús de la usabilidad en el entorno web, comenta que las aplicaciones desarrolladas para uso educativo, a las que el autor denomina <i>courseware</i> , son muy útiles si sus usuarios aprenden por medio de ellas, y que los juegos u otros programas con fines de entretenimiento son útiles si sus usuarios disfrutan de ellos. Por lo tanto, la usabilidad también se asocia al grado de aceptación de un producto. Señala que la utilidad de un sistema en cuanto a medio para conseguir un objetivo, tiene un componente de funcionalidad (utilidad funcional) y otro basado en el modo en que los usuarios pueden usar la citada funcionalidad. (Nielsen, 1994).
Nigel Bevan y Miles Macleod	1994	La usabilidad no sólo depende del producto sino también del usuario. Por eso, un producto no es ningún caso intrínsecamente usable, sólo tendrá la capacidad de ser usado en un contexto particular y por usuarios particulares. La usabilidad no puede ser valorada estudiando un producto de manera aislada. (Bevan y Macleod, 1994).

Edward Kit y Susannah Finzi	1995	Considerando el ámbito ergonómico, el objetivo del test de usabilidad es “adaptar el software a los estilos de trabajo reales de los usuarios, en lugar de forzar a los usuarios a adaptar sus estilos de trabajo al software”. (Kit y Finzi, 1995).
Donald A. Norman	1998	Los sistemas que sean usables, seguros y funcionales, acercarán mutuamente al usuario y el ordenador y, por lo tanto, harán que el espacio entre la tecnología y las personas disminuya. Eventualmente podríamos conseguir que este espacio estuviera vacío y llegar al caso ideal en que el ordenador fuera invisible. (Norman, 1998).
ISO 9241	1998	La usabilidad se refiere a la efectividad, eficiencia y satisfacción con la que usuarios concretos pueden abarcar unos objetivos específicos en un entorno particular. (ISO 9241, 1998).
Sun Microsystems	2000	Para la compañía Sun Microsystems, la ingeniería de la usabilidad “es una aproximación al desarrollo del producto que está basado en los datos del cliente y el feedback. Se fundamenta en la observación directa y en las interacciones con los usuarios para conseguir datos más reales que los conseguidos por técnicas automatizadas. La ingeniería de la usabilidad empieza ya en la fase conceptual del desarrollo con estudios de campo e investigaciones contextuales para comprender la funcionalidad y los requisitos de diseño.” (Sun Microsystems, 2000).

Kristoffer Bohmann	2001	Especialista en usabilidad, en su página web explica su definición de usabilidad de la siguiente manera tan concisa: “que los usuarios puedan completar sus tareas. En particular, los usuarios y sus principales necesidades son conocidas y detalladas; los usuarios pueden completar sus tareas sin demora o errores, y pueden disfrutar de la experiencia.” (Bohmann, 2001).
ISO/IEC 9126	2001	La usabilidad se refiere a la capacidad de un software para ser entendido, aprendido, usado y para ser atractivo al usuario, en condiciones específicas de uso” Esta definición resalta los atributos internos y externos del producto, que contribuyen a su usabilidad, funcionalidad y eficiencia. (ISO/IEC 9126, 2001).

Fuente: Grau (2007).

El factor de calidad usabilidad se puede ver desde dos perspectivas, según lo proponen en su investigación Gonzáles y otros (2012), como producto o como proceso.

4.1.1.1. Usabilidad como producto

Se refiere a la capacidad de un software para ser comprendido, aprendido, utilizado y resultar atractivo para el usuario en condiciones específicas de uso. Esta definición se utiliza para evaluar la calidad de un software y se basa en métricas internas (inherentes a los elementos del producto software) y externas (cuando el sistema es utilizado por usuarios). La usabilidad es una medida genérica de calidad de los sistemas de información que se ha utilizado en diversos estudios y proyectos como un elemento identificador de la calidad del software. Algunos de estos estudios han sido publicados en revistas, como los de Ribera, Térmens y García-Martín (2008), Baeza-Yates, Rivera-Loaiza y Velasco-Martín (2004) o Marcos (2004) (González y otros, 2012).

4.1.1.2. Usabilidad como proceso

Se define como la eficacia, eficiencia y satisfacción con la que un sistema permite alcanzar objetivos específicos a usuarios concretos en un contexto de uso específico. Esta definición es adecuada para evaluar sistemas software o de información en un entorno profesional, pero puede tener limitaciones en productos de entretenimiento en los que la satisfacción es más importante que la eficacia o la eficiencia. Es importante tener en cuenta que la evaluación de la usabilidad puede variar dependiendo del contexto y objetivos del producto en cuestión (González y otros, 2012).

4.1.1.3. Medición de la usabilidad

La norma ISO 9241-11 proporciona una metodología de evaluación muy precisa para la usabilidad, desde la especificación de tareas, usuarios, objetivos, entorno o equipos, hasta las mediciones a realizar.

La prueba de usabilidad utiliza un acuerdo de confidencialidad el cual es un documento que informa al usuario sobre el alcance de la prueba y garantiza que la información de la prueba se utilizará únicamente con fines académicos. El cuestionario previo a la prueba es un instrumento que busca obtener el perfil del usuario, así como la experiencia previa en el uso de aplicaciones similares a las que el usuario evaluará en la prueba. La lista de tareas corresponde a las tareas definidas por los coordinadores de la prueba y que el usuario realizará dentro de la aplicación que se evaluará.

El cuestionario posterior a la prueba es un instrumento cuantitativo y cualitativo que busca indagar sobre la experiencia del usuario con la aplicación evaluada, lo cual incluye la percepción de satisfacción en el uso del software.

Con base en la supervisión realizada por los observadores sobre las tareas realizadas por el usuario, se calculan los atributos de efectividad, eficiencia y satisfacción de acuerdo con la norma ISO 9241-11. El porcentaje de efectividad por tarea se puede determinar mediante la *ecuación 1*, donde cada tarea en la prueba debe tener un conjunto de subtarefas:

Ecuación 1. Atributo Efectividad de la usabilidad.

$$\%efectividad = \frac{(\text{número de sub-tareas realizadas} \times 100)}{(\text{número de sub-tareas definidas})} \quad (1)$$

Fuente: ISO 9241-11.

Por otro lado, el porcentaje de eficiencia por tarea se puede determinar a partir de lo que se presenta en la *ecuación 2*, donde los coordinadores de la prueba han estimado una duración para cada tarea:

Ecuación 2. Atributo eficiencia de la usabilidad.

$$\%eficiencia = \frac{(\text{tiempo estimado por tarea} \times 100)}{(\text{tiempo empleado por tarea})} \quad (2)$$

Fuente: ISO 9241-11.

Finalmente, en cuanto a la satisfacción, se promedia la evaluación dada por los usuarios finales a las preguntas del cuestionario posterior a la prueba que involucran la percepción de satisfacción del usuario, como se presenta en la *ecuación 3*:

Ecuación 3. Atributo satisfacción de la usabilidad.

$$\%satisfacción = \frac{(\text{suma de las calificaciones de las preguntas})}{(\text{número de preguntas})} \quad (3)$$

Fuente: ISO 9241-11.

Una forma paralela de calcular el porcentaje de satisfacción es mediante el promedio ponderado. En este método, cada pregunta recibe un peso específico según su importancia. Luego, se multiplica la calificación de cada pregunta por su peso, y finalmente se obtiene el promedio total dividiendo la suma de esos resultados entre el número de preguntas.

4.1.1.4. Plan para el estudio de la usabilidad

El portal Nielsen Norman Group sugiere una lista de pasos para la medición de la usabilidad de software. Según la autora Loranger (2016), dicha lista está compuesta por 7

pasos subsecuentes que empiezan por la definición de objetivos hasta la observación de los usuarios en el campo de la medición. En resumen, Loranger propone el siguiente orden:

- a. Definir claramente los objetivos de la investigación, identificar preguntas y preocupaciones relevantes y elegir la metodología adecuada para recopilar datos cualitativos o cuantitativos.
- b. Determinar el formato y entorno de estudio considerando factores como el formato y el entorno del estudio, incluyendo si se realizará en un laboratorio o en el campo, si será moderado o no moderado, y si se llevará a cabo en persona o de forma remota.
- c. Determinar el número de participantes requeridos en función del tipo de investigación que se esté realizando. En términos generales, es recomendable contar con al menos 5 a 10 participantes en cada grupo de usuarios objetivo.
- d. Reclutar participantes representativos que se ajusten a las características demográficas y conductuales del público objetivo, evitando usuarios no representativos que puedan proporcionar resultados inválidos.
- e. Desarrollar tareas coherentes con los objetivos del estudio y escritas como escenarios, con dos tipos principales de tareas: exploratorias y específicas.
- f. Realizar un estudio piloto para refinar las instrucciones de las tareas, determinar el número de tareas que se pueden realizar por sesión y refinar los criterios de reclutamiento de los participantes.
- g. Medir la usabilidad en estudios cuantitativos, utilizando métricas como el tiempo de tarea, la tasa de éxito, la tasa de errores y la satisfacción. Cuando se utilizan medidas subjetivas, como preguntas de facilidad de uso o satisfacción del sistema/tarea, es importante utilizar instrumentos de encuesta validados y considerar cuidadosamente la redacción de las preguntas.

El permiso informado de los participantes antes de llevar a cabo una evaluación o estudio de usabilidad se denomina consentimiento del usuario. Este, se realiza mediante un formulario en el que el usuario manifiesta que entiende que la participación en el estudio de usabilidad es voluntaria y se compromete a comunicar de inmediato cualquier inquietud o incomodidad durante la sesión con el administrador del estudio. También, debe firmar para indicar que ha leído y comprende la información del formulario y que se han respondido todas las preguntas que pueda tener sobre la sesión. Por último, acepta participar en el estudio realizado por la agencia/organización.

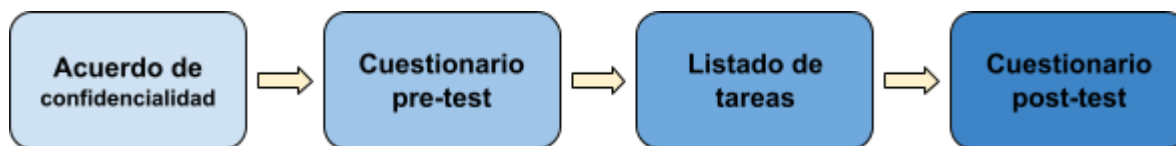
Por otro lado, para la grabación de las interacciones y comentarios durante el estudio de usabilidad, se debe diligenciar un formulario en el que el usuario acepte participar en una grabación de audio, video y/o digital durante el estudio realizado por la agencia/organización, este puede retirarse en cualquier momento. El usuario debe manifestar que entiende y es consciente del uso y la divulgación de la grabación por parte de la agencia/organización. También, el usuario renuncia a cualquier derecho sobre la grabación y entiende que la agencia/organización puede duplicar y utilizar la grabación sin necesidad de obtener más permisos. Finalmente, acepta comunicar de inmediato cualquier inquietud o sensación de incomodidad al administrador del estudio (Usability.gov, 2023).

4.1.2. Test con usuarios

En el contexto de la medición de la usabilidad de un software determinado se usan los cuestionarios de usabilidad. Éstos son instrumentos de recolección de información, previamente elaborados por el coordinador de la prueba o el evaluador, y son llevados a un laboratorio de usabilidad en el que un usuario diligencia de acuerdo a su experiencia luego de hacer uso del software a medir.

Un cuestionario de usabilidad con usuarios consta de varias etapas, como se puede ver en la *Figura 2*, que incluyen la selección de los usuarios y la definición de los instrumentos de la prueba. Estas etapas son: acuerdo de confidencialidad, cuestionario pre-test, listado de tareas y cuestionario post-test.

Figura 2. Etapas de una prueba de usabilidad.



Fuente: elaboración propia.

4.1.2.1. Acuerdo de confidencialidad

Antes de iniciar cualquier actividad relacionada con el cuestionario de usabilidad, es importante establecer un acuerdo de confidencialidad con los participantes. Esto garantiza que cualquier información recopilada durante el proceso se mantenga confidencial y se utilice únicamente con fines de evaluación. Se solicita la autorización del usuario para capturar datos durante la prueba y se le informa que sus datos serán utilizados únicamente con fines académicos.

4.1.2.2. Cuestionario pre-test

Este cuestionario busca recopilar información demográfica y de experiencia previa con el producto o servicio que se está evaluando. También puede incluir preguntas sobre las expectativas del participante con respecto a la usabilidad y su familiaridad con la tecnología en general. Se le pide al usuario que proporcione información relacionada con su perfil.

4.1.2.3. Listado de tareas

En esta etapa, se presenta a los participantes un conjunto de tareas específicas que deben realizar utilizando el producto o servicio. Estas tareas se eligen cuidadosamente para abordar los aspectos clave de la usabilidad que se desean evaluar. El listado de tareas debe ser claro y conciso, y cada tarea debe estar formulada de manera que no sugiera la forma de realizarla. Esto permite obtener una evaluación más realista de la usabilidad del producto. Los usuarios deben realizar las tareas definidas sin la intervención del coordinador o moderador.

4.1.2.4. *Cuestionario post-test*

Después de que los participantes hayan completado todas las tareas, se les puede pedir que completen un cuestionario post-test. Este cuestionario busca recopilar información sobre su experiencia general, su satisfacción con el producto o servicio, y cualquier problema o dificultad que hayan encontrado durante la realización de las tareas. También puede incluir preguntas abiertas para obtener comentarios más detallados sobre aspectos específicos de la usabilidad. Los usuarios responden un conjunto de preguntas, tanto cuantitativas como cualitativas, una vez que han completado las tareas.

4.1.3. Preguntas abiertas

En el contexto de los cuestionarios de usabilidad, las preguntas abiertas son aquellas que permiten a los participantes proporcionar comentarios y opiniones detalladas sobre su experiencia de uso de un producto, aplicación o sitio web. Estas preguntas no están limitadas a opciones de respuesta predefinidas y brindan a los participantes la libertad de expresarse de manera más amplia.

Las preguntas abiertas en los cuestionarios de usabilidad suelen solicitar a los usuarios que describan sus impresiones generales sobre la interfaz, los problemas que encontraron durante la navegación, las dificultades específicas que experimentaron o cualquier sugerencia que tengan para mejorar la usabilidad del producto. Estas preguntas permiten recopilar información cualitativa valiosa sobre la experiencia del usuario, sus frustraciones, necesidades y expectativas.

La ventaja de las preguntas abiertas en los cuestionarios de usabilidad es que brindan información más rica y detallada que puede ayudar a los diseñadores y desarrolladores a comprender mejor los desafíos que enfrentan los usuarios y a identificar áreas de mejora. Sin embargo, es importante tener en cuenta que el análisis de las respuestas abiertas puede ser más complejo y requerir más tiempo que el análisis de las preguntas cerradas, ya que implica examinar y categorizar las respuestas de manera cualitativa (Cough, 2021).

Estas preguntas abiertas se pueden hacer en el idioma que sea más apropiado y relevante para los participantes del estudio. El idioma utilizado dependerá del público objetivo y de las características demográficas de los usuarios a quienes va dirigido el producto o servicio en cuestión, en este caso, se utilizará el español pero se usará un algoritmo de traducción.

En general, los cuestionarios de usabilidad se adaptan al idioma predominante o preferido de los usuarios para asegurar que comprendan claramente las preguntas y puedan proporcionar respuestas significativas. Si el producto o servicio se dirige a un público multilingüe, es posible que se requieran versiones del cuestionario en diferentes idiomas para garantizar la participación de todos los usuarios relevantes. Es importante tener en cuenta la localización y adaptación lingüística al diseñar cuestionarios de usabilidad para garantizar que las preguntas sean claras, relevantes y comprensibles para los participantes, independientemente de su idioma o cultura.

Una pregunta abierta en una prueba de usabilidad busca información detallada y enriquecedora sobre la experiencia del usuario, sus opiniones, sugerencias, comentarios y perspectivas subjetivas sobre la experiencia del usuario con un producto o servicio. (Marcos y Rovira, 2005). Por eso, estas preguntas deben tener los siguientes elementos: claridad, enfoque en la experiencia del usuario, invitación a la reflexión, apertura y libertad de respuesta, y especificidad. Un ejemplo de pregunta abierta en una prueba de usabilidad es la siguiente, basado en la prueba de usabilidad hecha en la investigación de Llana, Martín, Pareja y Velázquez (2013):

"Después de usar el producto, por favor, comparte cualquier comentario adicional, sugerencia o aspecto específico que consideres relevante para mejorar la experiencia de uso."

Las preguntas abiertas brindan la oportunidad de obtener información cualitativa valiosa que complementa los datos cuantitativos recopilados a través de métricas y cuestionarios cerrados (Debdi y otros, 2013).

4.1.4. Análisis de sentimientos

El análisis de sentimientos en las pruebas de usabilidad es un enfoque utilizado para evaluar las respuestas emocionales y subjetivas de los usuarios mientras interactúan con un producto o servicio (Chanchi y otros, 2019). Consiste en recopilar datos sobre las emociones, actitudes y opiniones de los usuarios durante una prueba de usabilidad y analizarlos para comprender mejor su experiencia y percepción (Vilares y otros, 2013).

El análisis de sentimientos en las pruebas de usabilidad es útil para comprender cómo se sienten los usuarios mientras interactúan con un producto o servicio. Los resultados pueden ayudar a los diseñadores y desarrolladores a identificar aspectos problemáticos, áreas de mejora y fortalezas en la experiencia del usuario. Al comprender las emociones y actitudes de los usuarios, se pueden tomar decisiones informadas para optimizar la usabilidad y la satisfacción general del usuario (Clavel y Callejas, 2015).

Existen varias bibliotecas y algoritmos de análisis de sentimientos de código abierto disponibles actualmente. Algunas de las más populares son:

- ❖ **NLTK (Natural Language Toolkit):** Es una biblioteca de Python ampliamente utilizada en el procesamiento de lenguaje natural. Proporciona herramientas para la tokenización, análisis gramatical y análisis de sentimientos.
- ❖ **TextBlob:** Es una biblioteca de Python que se basa en NLTK y ofrece una interfaz sencilla para el análisis de sentimientos. Proporciona funciones predefinidas para calcular la polaridad y subjetividad de un texto.
- ❖ **VADER (Valence Aware Dictionary and Sentiment Reasoner):** Es un algoritmo de análisis de sentimientos diseñado específicamente para procesar texto de redes sociales. Está disponible como parte de NLTK y ofrece un enfoque basado en reglas para asignar puntuaciones de sentimiento. VaderSentiment devuelve un diccionario con cuatro puntajes de sentimiento para un fragmento de texto: neg (sentimiento negativo), neu (sentimiento neutral), pos (sentimiento positivo) y compound (puntaje global de sentimiento). Los valores de neg, neu y pos están en el rango de 0

a 1, mientras que compound varía entre -1 (totalmente negativo) y +1 (totalmente positivo), representando la polaridad general del texto (Hutto y Gilbert, 2014).

- ❖ **Sentiment Analysis Toolkit:** Es una biblioteca de Python que proporciona una variedad de algoritmos y modelos para el análisis de sentimientos. Incluye métodos basados en aprendizaje automático y enfoques basados en reglas.

Los resultados generados por el análisis de sentimientos en las pruebas de usabilidad permiten obtener información valiosa sobre la respuesta emocional de los usuarios. Estos resultados pueden incluir métricas como la polaridad promedio de los comentarios (positiva, negativa o neutral), la distribución de sentimientos a lo largo del tiempo o en diferentes secciones de la aplicación, y patrones de emociones específicas asociadas con ciertos aspectos del producto (Vinodhini y Chandrasekaran, 2012).

Estos resultados pueden ayudar a los diseñadores y desarrolladores a comprender mejor la experiencia del usuario, identificar problemas de usabilidad, evaluar la eficacia de las mejoras realizadas y tomar decisiones informadas para mejorar la calidad y satisfacción del usuario (Zvarevashe y Olugbara, 2018).

4.2. Paquete de trabajo 2: Diseño e implementación del sistema

En esta sección se abordaron el diseño e implementación del sistema propuesto. Se definieron los requisitos funcionales y se elaboraron los modelos de dominio, base de datos y los diagramas UML correspondientes: casos de uso, clases, secuencia y despliegue. Asimismo, se seleccionaron las tecnologías de desarrollo y se describieron las funcionalidades tanto del lado del cliente como del servidor. Finalmente, se incluyeron capturas de pantalla que ilustran la interfaz del sistema implementado.

En el [Anexo A](#) de este documento se incluyen los enlaces al repositorio que contiene los códigos fuente de ambos proyectos: el frontend y el backend. Esto permite acceder al desarrollo completo del sistema.

4.2.1. Especificación de requisitos

Para la especificación de requisitos se tomaron en cuenta algunos de los aspectos sugeridos por la IEEE en el estándar 830 (IEEE, 1998):

4.2.1.1. Roles

El usuario intencional que usó el sistema para la estimación de la usabilidad tiene las siguientes características: tiene el rol de coordinador de prueba de usabilidad, conoce los cuestionarios de usabilidad, la forma en que se evalúan, entiende e interpreta las gráficas de porcentaje de usabilidad del software y sabe manejar el computador (tiene nociones básicas acerca de su funcionamiento, tiene experiencia mínima en su uso).

4.2.1.2. Restricciones

Las restricciones concernientes al sistema para la estimación de la usabilidad corresponden a las limitaciones que se presentaron durante el diseño y desarrollo del software, especialmente en aspectos relacionados con la privacidad de los datos, la precisión de los resultados, el uso de tecnologías libres y la compatibilidad con otras plataformas. En primer lugar, el sistema debía garantizar la confidencialidad de la información, evitando la exposición de los datos recopilados durante el análisis de sentimientos, incluyendo tanto los comentarios de los usuarios como los resultados generados por la herramienta.

En cuanto a los resultados ofrecidos, se estableció como restricción el uso de bibliotecas de análisis de sentimientos confiables y precisas, asegurando una interpretación consistente y exenta de errores que respaldara la calidad de la evaluación de usabilidad. Además, se definió como requisito el empleo exclusivo de tecnologías de código abierto, evitando cualquier infracción a derechos de autor o licencias propietarias. Finalmente, se consideró esencial que el aplicativo web fuera compatible con los navegadores más utilizados, a fin de asegurar su accesibilidad, usabilidad y correcta visualización en múltiples plataformas.

4.2.1.3. *Requisitos funcionales*

La *Tabla 2*, a continuación, muestra el listado de los requisitos funcionales establecidos al sistema para la estimación de la usabilidad, según lo propuesto por la IEEE 830. El listado incluye un identificador, el nombre del requisito, la descripción, lo que recibe como entrada, lo que entrega y la prioridad:

Tabla 2. Requisitos funcionales del sistema.

ID	Nombre	Descripción	Prioridad
RF1	Gestionar los resultados de los software evaluados.	Crear, listar, editar y eliminar los registros de las pruebas de usabilidad realizadas a cada software. Recibe como entrada: el nombre y versión del software evaluado. Entrega como salida: una lista de los software evaluados con sus porcentajes de usabilidad.	Media
RF2	Importar los valores de las pruebas.	Cargar archivo separado por comas o llenar la tabla de valores manualmente y almacenarlos para su análisis. Recibe como entrada: los valores obtenidos en la prueba de usabilidad, la cantidad de usuarios evaluados y cantidad de tareas y preguntas. Entrega como salida: un mensaje que confirme que fueron almacenados los valores de la prueba	Alta
RF3	Importar los comentarios de los usuarios	Cargar archivo separado por comas o llenar la tabla con las respuestas a las preguntas abiertas de los usuarios.	Alta

		Recibe como entrada: las respuestas de los usuarios y la cantidad de usuarios. Entrega como salida: un mensaje que confirme que fueron almacenados los comentarios de los usuarios.	
RF4	Calcular el porcentaje de eficiencia, eficacia y satisfacción	Calcular el porcentaje de cada atributo de la usabilidad siguiendo las métricas de tiempos empleados, cantidad de tareas realizadas exitosamente, y evaluaciones del software basadas en escalas cuantitativas. Recibe como entrada: los valores cargados en los pasos anteriores. Entrega como salida: el nivel porcentual de cada atributo de la usabilidad.	Alta
RF5	Calcular el nivel de satisfacción de los comentarios	Calcular el porcentaje de satisfacción de los usuarios mediante el análisis de sentimientos. Recibe como entrada: las opiniones de los usuarios. Entrega como salida: el nivel porcentual de satisfacción.	Alta
RF6	Mostrar las gráficas de eficiencia, eficacia y satisfacción	Graficar los porcentajes de cada atributo. Recibe como entrada: los porcentajes calculados en los pasos anteriores. Entrega como salida: gráfica con el nivel porcentual de cada atributo de la usabilidad.	Media

RF7	Calcular el porcentaje de usabilidad	Calcular el porcentaje de la usabilidad siguiendo las mediciones de cada uno de los atributos. Recibe como entrada: los porcentajes ya calculados. Entrega como salida: el nivel porcentual de la usabilidad.	Alta
RF8	Graficar el porcentaje de usabilidad	Graficar el porcentaje de la usabilidad. Recibe como entrada: los porcentajes ya calculados. Entrega como salida: gráfica con el porcentaje de usabilidad.	Media
RF9	Reporte de la usabilidad del software evaluado	Entregar el porcentaje de usabilidad del software evaluado. Recibe como entrada: los porcentajes ya calculados. Entrega como salida: frase textual con el nombre del software evaluado, la versión y el porcentaje de usabilidad calculado.	Alta

Fuente: elaboración propia.

4.2.1.4. Requisitos no funcionales

Los requisitos no funcionales describen características y atributos que no están directamente relacionados con la funcionalidad del sistema, sino con aspectos como su rendimiento, seguridad, usabilidad, mantenibilidad y otros. Estos requisitos se centran en cómo el sistema debe comportarse y cumplir ciertos estándares y criterios de calidad (Glinz, 2007). Basándonos en la norma ISO 25010, a continuación se presenta una elección de los requisitos no funcionales relevantes para el aplicativo que gestiona las evaluaciones de usabilidad de software:

- ❖ Usabilidad: El aplicativo debe poseer una interfaz intuitiva que permita al coordinador de la prueba navegar y utilizar las funcionalidades sin dificultad. Los elementos de la interfaz deben estar organizados de manera clara y coherente.

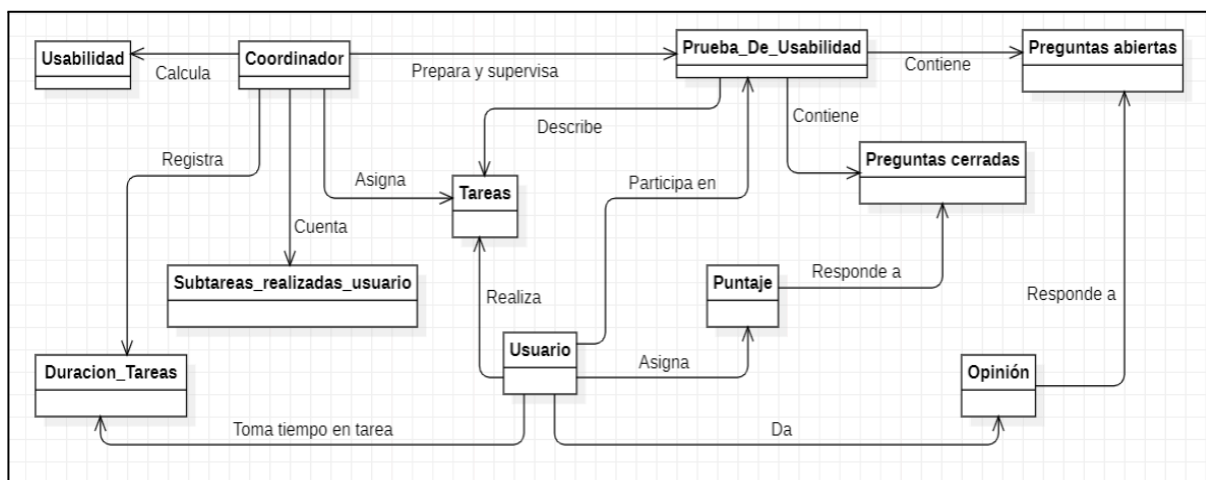
- ❖ Eficiencia de desempeño: El aplicativo debe tener un tiempo de respuesta rápido al procesar los valores tomados de las pruebas de usabilidad y generar los reportes con gráficas. La generación de los informes debe ser eficiente, evitando retrasos significativos en la entrega de resultados.
- ❖ Fiabilidad: El aplicativo debe ser confiable y consistente en su funcionamiento. Debe procesar y almacenar los datos de manera precisa, asegurando que los resultados de las evaluaciones de usabilidad sean correctos y consistentes.
- ❖ Mantenibilidad: El aplicativo debe ser modular y tener un código bien estructurado, lo que facilitará las futuras actualizaciones y mejoras. Se debe permitir una fácil incorporación de nuevas funcionalidades y corrección de errores sin afectar el funcionamiento general.
- ❖ Portabilidad: El aplicativo debe ser compatible con diferentes navegadores web y sistemas operativos comunes, garantizando una experiencia consistente para el coordinador de la prueba.
- ❖ Seguridad: Aunque no se maneje sesión de usuarios, es importante implementar medidas de seguridad para proteger la integridad de los datos almacenados y asegurar que no sean accesibles o modificables por personas no autorizadas, tales como incluir certificados de transferencia de hipertexto seguro y ocultamiento de identificadores de evaluaciones.

4.2.2. Modelo de dominio

El modelo de dominio es la representación conceptual que identifica las entidades que intervienen en el problema y las relaciones que existen entre estas, además proporciona una visión estructural (Puertos, Cerdán y De La Mora, 2020).

En la *figura 3* se muestra el modelo de dominio del contexto del problema en el mundo real, es decir, las entidades halladas y las relaciones entre estas, que existen en el contexto de la evaluación de la usabilidad mediante test con usuarios en un laboratorio de usabilidad.

Figura 3. Modelo de dominio del mundo real.

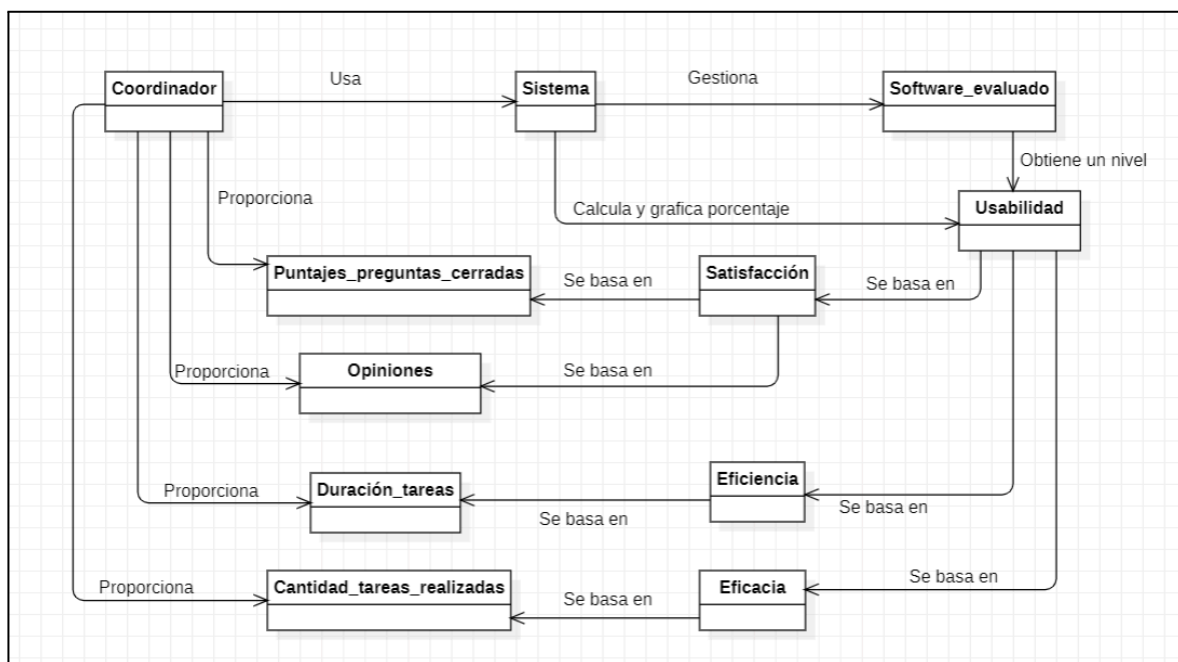


Fuente: elaboración propia.

En el modelo de dominio del mundo real se pudo observar, entre otras cosas, que un coordinador prepara y supervisa una prueba de usabilidad la cual contiene preguntas abiertas y cerradas, describe una lista de tareas y subtareas. A su vez, estas tareas son asignadas por el coordinador al usuario. El usuario, por su parte, realiza las tareas, luego asigna un puntaje a cada pregunta cerrada y da su valoración con una opinión respondiendo a las preguntas abiertas. Mientras cada usuario ejecuta las tareas, el coordinador registra los tiempos que toman los usuarios para completar las subtareas y cuenta aquellas que fueron finalizadas correctamente. Finalmente, el coordinador calcula los porcentajes de eficiencia, eficacia y satisfacción, con lo cual obtiene la estimación de la usabilidad del software evaluado.

Por otro lado, en la *figura 4* se presentan las entidades pertenecientes al contexto del sistema desarrollado y las relaciones entre ellas.

Figura 4. Modelo de dominio del sistema desarrollado.



Fuente: elaboración propia.

En este caso, el modelo de dominio del sistema desarrollado pone en contexto el uso de la herramienta software para estimación de la usabilidad de un software mediante el análisis de sentimientos.

Es así que, un coordinador usa la herramienta proporcionando los puntajes de las preguntas cerradas, las opiniones de las preguntas abiertas, los valores de duración por tarea y la cantidad de tareas realizadas de cada usuario, mediante llenado manual de tablas o la carga de archivos.

El sistema, por su lado, gestiona las evaluaciones y obtiene un nivel de usabilidad de cada una, basándose en los cálculos realizados para la eficiencia, eficacia y satisfacción, los cuales usan los datos proporcionados por el coordinador.

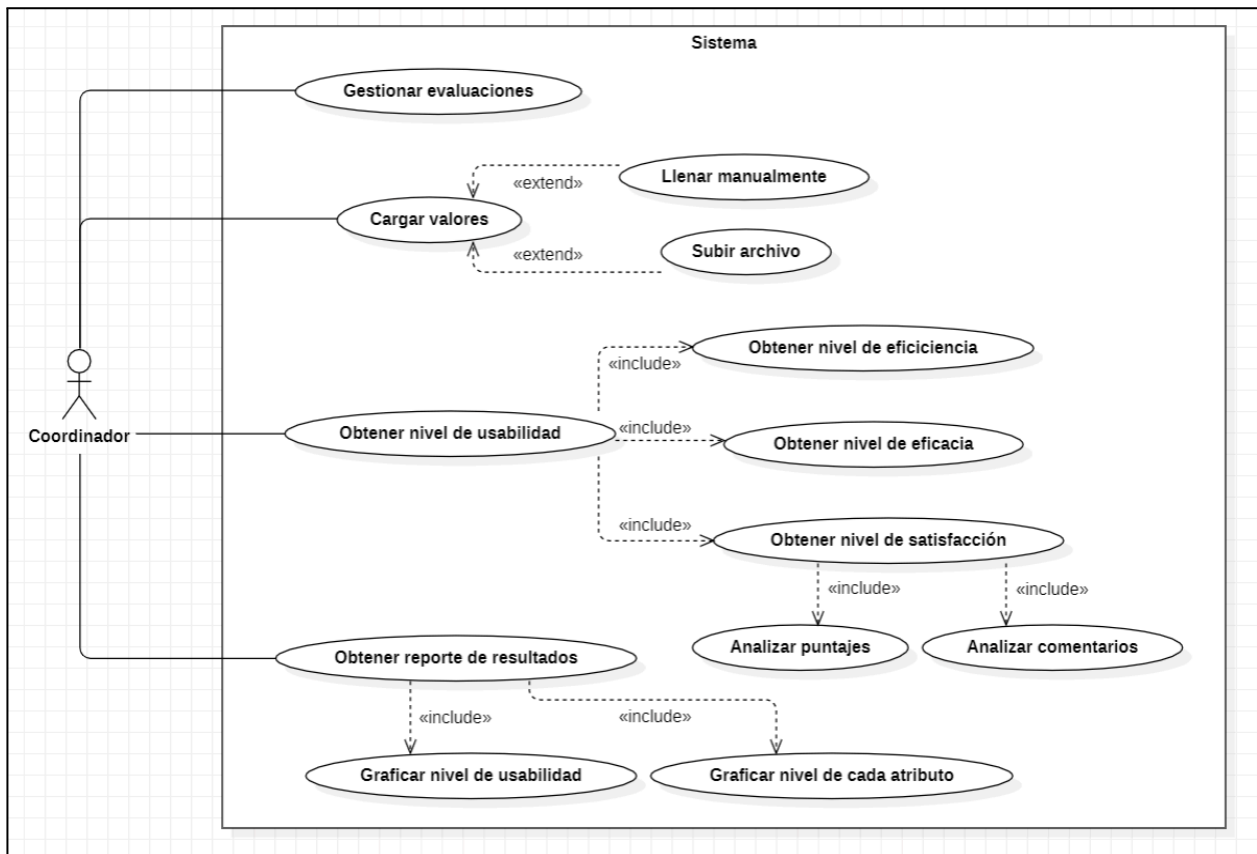
4.2.3. Vista de escenarios del sistema

La vista de escenarios forma parte del modelo 4+1 propuesto por Kruchten (2002), y tiene como propósito describir cómo el sistema responde a situaciones específicas desde la

perspectiva del usuario. Esta vista se utiliza para validar la arquitectura mediante escenarios concretos de uso, lo que facilita la comprensión de los requisitos funcionales. En este contexto, se emplean los casos de uso como técnica principal para modelar dichos escenarios.

Los casos de uso describen las funcionalidades o secuencia de acciones del sistema y la interacción con los actores involucrados (Li, Liu y Jifeng, 2004). Siguiendo ésto, se presenta el diagrama de casos de uso (en la *figura 5*) y se describen cada uno de ellos mediante tablas. En resumen, el siguiente diagrama describe la interacción del coordinador de una prueba de usabilidad quien, haciendo uso del sistema propuesto, puede gestionar las evaluaciones, cargar los valores, obtener los niveles de usabilidad y de los atributos de la usabilidad, y analizar el reporte que contiene gráficas.

Figura 5. Diagrama de casos de uso.



Fuente: elaboración propia.

Basado en el diagrama de casos de uso mostrado arriba, se presentan ahora las descripciones para cada caso de uso en las siguientes tablas. La *tabla 3* describe el caso de uso *Gestionar evaluaciones*:

Tabla 3. Descripción del caso de uso Gestionar evaluaciones.

Nombre del caso de uso	CU1 - GESTIONAR EVALUACIONES
Breve descripción	Crear, listar, editar y eliminar una o varias evaluaciones de usabilidad de software.
Actores involucrados	Coordinador de la prueba.
Requisitos involucrados	RF1.
Precondiciones	Se debe iniciar el aplicativo y esperar que cargue la página principal. Para editar una evaluación se deben proporcionar los valores recolectados en la prueba de usabilidad.
Flujo básico	1. Se inicia el aplicativo o se dirige a la página principal para listar las evaluaciones. 2. Se crea una nueva evaluación y se entregan nombre, versión y los demás datos solicitados del software a evaluar. 3. Se guarda la nueva evaluación. 4. Para editar se abre una evaluación y se cargan valores para reemplazar los existentes. 5. Para eliminar, en la página de inicio, se elige la opción eliminar presente en cada evaluación mediante un ícono de eliminar.

Flujos alternos	Si el coordinador no concluye la creación o la edición mediante la opción de confirmación o guardar, entonces queda cancelado cualquiera de esos procesos.
Postcondiciones	El software muestra el listado de las evaluaciones. En caso de no existir ninguna queda solamente la opción crear.
Frecuencia	Cada vez que el coordinador lo requiera, o cada vez que se vaya a hacer la evaluación de un software específico.

Fuente: elaboración propia.

Ahora, la *tabla 4* describe el caso de uso *Cargar valores*.

Tabla 4. Descripción del caso de uso Cargar valores.

Nombre del caso de uso	CU2 - CARGAR VALORES
Breve descripción	Cargar los valores de los puntajes, tiempos medidos y los comentarios de la prueba de usabilidad manualmente o mediante un archivo de formato SVC.
Actores involucrados	Coordinador de la prueba.
Requisitos involucrados	RF2, RF3.
Precondiciones	Debe haberse creado previamente una evaluación. El coordinador debe contar con los resultados de las pruebas de usabilidad de todos los usuarios. Además, el coordinador debe tener los valores de referencia para cada atributo.
Flujo básico	1. Si el coordinador elige entre llenar los valores manualmente: a. Se piden la cantidad de usuarios, luego la cantidad de tareas y cantidad de preguntas

	dependiendo el caso de cada atributo de la usabilidad. b. El coordinador rellena cada celda con los valores recogidos de la prueba de usabilidad. c. El coordinador confirma guardar los valores. 2. Si el coordinador elige llenar los valores manualmente: a. Elige un archivo local mediante búsqueda. b. El archivo trae la información para crear las tablas y rellenarlas automáticamente. c. El coordinador confirma guarda los valores.
Flujos alternos	Si el coordinador ingresa valores diferentes a enteros positivos se muestra una alerta para no permitir guardarse. Por otro lado, si el archivo presenta inconsistencias o los valores están incompletos se muestra una alerta para no permitir guardarse.
Postcondiciones	El software carga los valores de cada tabla y muestra una confirmación del almacenamiento. los cuales pueden editarse.
Frecuencia	Cada vez que el coordinador lo requiera, cada vez que se vaya a hacer la evaluación de un software específico, o la edición de una evaluación.

Fuente: elaboración propia.

Por otro lado, la *tabla 5* describe el caso de uso *Obtener nivel de usabilidad*.

Tabla 5. Descripción caso de uso Obtener nivel de usabilidad.

Nombre del caso de uso	CU3 - OBTENER NIVEL DE USABILIDAD
Breve descripción	Calcular el nivel de cada atributo y finalmente el nivel de usabilidad usando las ecuaciones de las métricas de usabilidad.
Actores involucrados	Coordinador de la prueba.
Requisitos involucrados	RF4, RF5, RF7.
Precondiciones	Los valores deben estar cargados y almacenados.
Flujo básico	1. Una vez estén cargados los valores, se hace el cálculo de cada atributo, es decir, se van calculando por atributos separados. 2. La usabilidad se calcula cuando estén todos los atributos previamente calculados. 3. El coordinador navega entre las pestañas de cada atributo de usabilidad y la pestaña de usabilidad para ver los reportes calculados automáticamente.
Flujos alternos	Si un atributo en específico no ha sido calculado aún entonces se presenta un mensaje de alerta dentro de la pestaña usabilidad.
Postcondiciones	El estado de la evaluación en base de datos cambia a analizado y en la parte principal se muestra dicha evaluación con el porcentaje de usabilidad obtenido.
Frecuencia	Cuando el coordinador finalice el cargue de los valores.

Fuente: elaboración propia.

Finalmente, la *tabla 6* describe el caso de uso *Obtener reporte de resultados*.

Tabla 6. Descripción caso de uso Obtener reporte resultados.

Nombre del caso de uso	CU4 - OBTENER REPORTE DE RESULTADOS
Breve descripción	Es presentado al coordinador el resultado de los porcentajes de eficacia, eficiencia, satisfacción y, por consiguiente, usabilidad total del software evaluado.
Actores involucrados	Coordinador de la prueba.
Requisitos involucrados	RF6, RF8, RF9.
Precondiciones	El sistema debe tener en almacenamiento o en repositorio de base de datos los porcentajes ya calculados.
Flujo básico	1. El sistema muestra en una frase los resultados de cada atributo en su pestaña específica. 2. Además se usan gráficas que describen el porcentaje de usabilidad por usuario. 3. El estado se muestra en la página principal.
Flujos alternos	Si el porcentaje de usabilidad es bajo entonces se pueden usar colores o alertas para describir el nivel crítico. Por otro lado, si existe gran cantidad de usuarios evaluados la gráfica es mostrada decrece.
Postcondiciones	El proceso de medición de la usabilidad para un software o una versión determinada ha finalizado en este punto, por lo tanto, el coordinador recibe la retroalimentación final.
Frecuencia	Cada vez que visualice una evaluación ya analizada.

Fuente: elaboración propia.

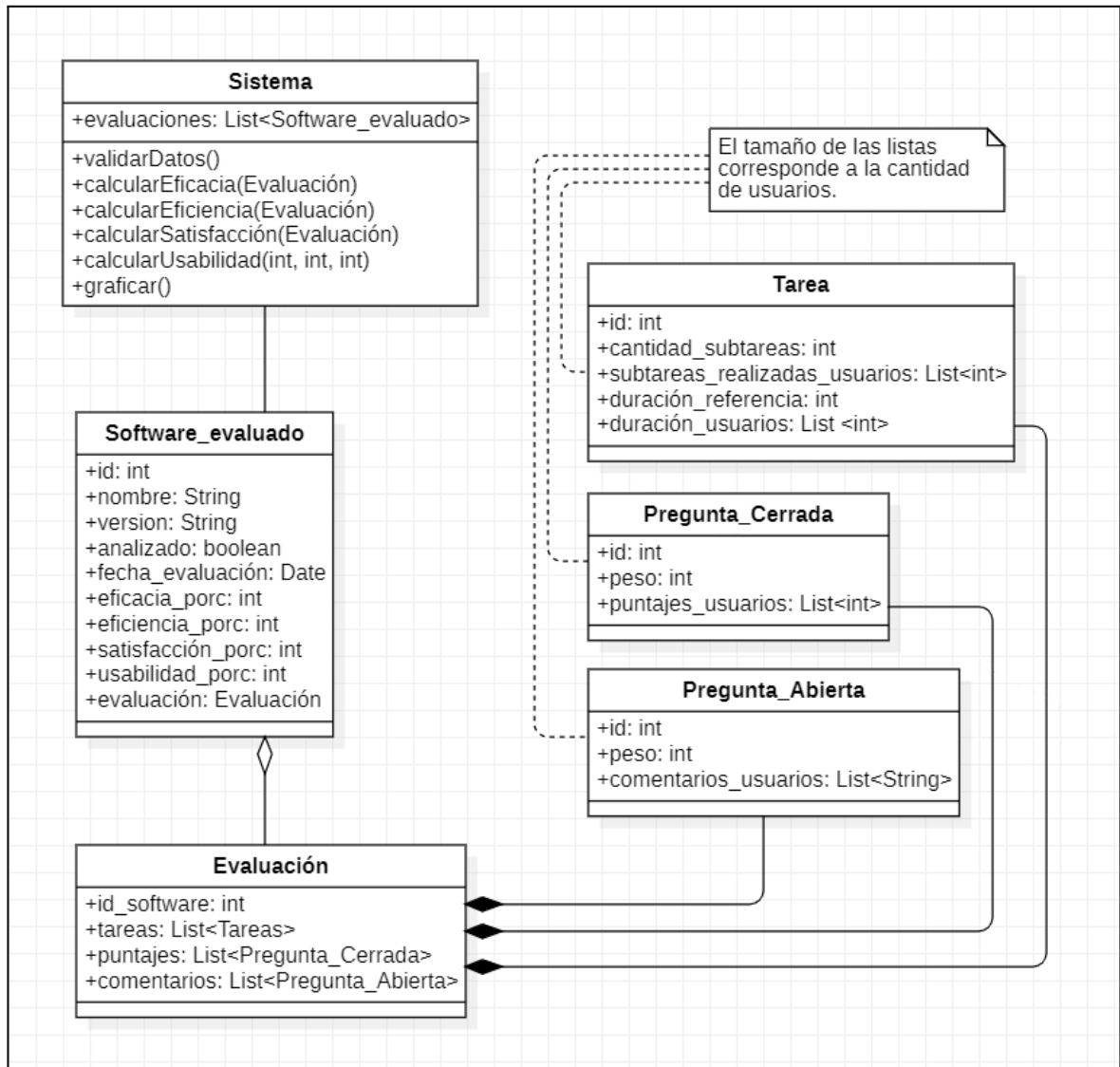
4.2.4. Vista lógica del sistema

La vista lógica del sistema, según el modelo 4+1 de Kruchten (2002), representa la estructura estática del software desde una perspectiva orientada a objetos. Esta vista organiza los principales componentes del sistema y sus relaciones, permitiendo comprender cómo se distribuyen las responsabilidades dentro de la arquitectura. A partir de esta vista, se construye el modelo de diseño, que define la estructura estática del sistema y las relaciones entre las clases. Para su representación se emplea el diagrama de clases, el cual muestra las clases del sistema, sus atributos y métodos, así como las asociaciones, herencias, interfaces y dependencias entre ellas (Ab Hamid et al., 2007).

En la *figura 6* se muestran las clases involucradas en el diseño de la solución conceptual. Específicamente, un objeto de la clase *Evaluación* que está compuesto por una lista de objetos de la clase *Tarea*, una lista de objetos de la clase *Pregunta_Cerrada* y una lista de objetos de la clase *Pregunta_Abierta*.

Dicho objeto está agregado en un objeto de la clase *Software_evaluado*, el cual tiene como atributos un identificador, un nombre, una versión, un estado, una fecha de evaluación, y los porcentajes de eficacia, eficiencia, satisfacción y usabilidad. Este objeto, finalmente, está asociado a la clase *Sistema* la cual ejecuta varios métodos para validar los datos, realizar los cálculos y graficar.

Figura 6. Diagrama de clases del sistema.

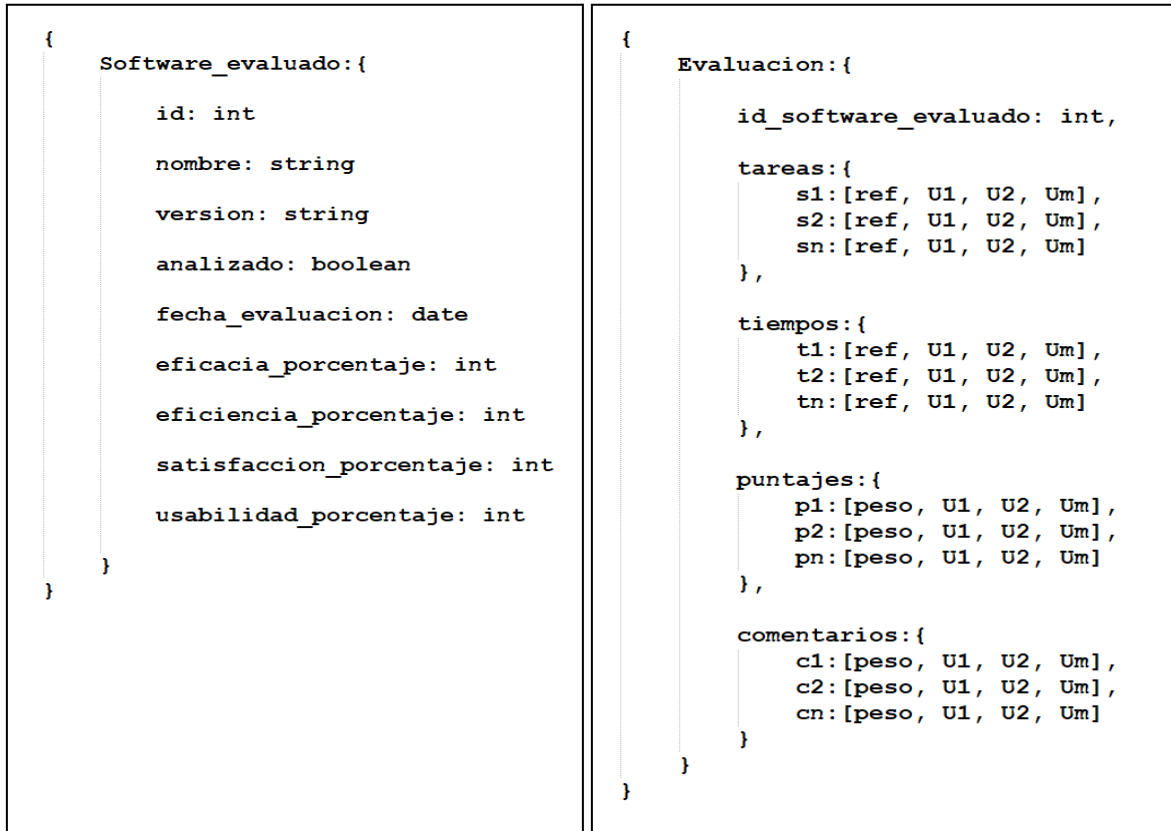


Fuente: elaboración propia.

4.2.5. Modelo de base de datos

En lo que respecta al almacenamiento y recuperación de los datos para el cálculo de la usabilidad se propuso un enfoque no relacional (NoSQL). Las estructuras de las colecciones de datos se presentan en la *figura 7*:

Figura 7. Estructuras de las colecciones de datos.



Fuente: elaboración propia.

Para una detallada descripción de la colección *Software_evaluado* usamos la siguiente *tablas 7* en donde se especifican los nombres de las claves, los tipos de valor y una descripción que muestra las reglas de negocio a nivel de base de datos para cada clave y un ejemplo que satisface al valor.

Tabla 7. Descripción de las claves y los valores de la colección *Software_evaluado*.

Clave	Tipo	Descripción
id	int	Es un identificador único para cada software evaluado. Es asignado automáticamente por el gestor de base de datos. Ejemplo: 1023.

nombre	String	Es el nombre del software evaluado. Ejemplo: “Calculator Pro”
versión	String	Es la versión del software evaluado. Ejemplo: “2.1.3”.
analizado	boolean	Indica si está analizado o no. Por defecto es falso. Ejemplo: False.
fecha_evaluación	Date	Almacena la fecha en que se creó la evaluación. Ejemplo: 2023-06-07.
eficacia_porcentaje	int	Es un número entre 0 y 100 que representa el porcentaje de eficacia obtenido. Ejemplo: 78.
eficiencia_porcentaje	int	Es un número entre 0 y 100 que representa el porcentaje de eficiencia obtenido. Ejemplo: 93.
satisfaccion_porcentaje	int	Es un número entre 0 y 100 que representa el porcentaje de satisfacción obtenido. Ejemplo: 59.
usabilidad_porcentaje	int	Es un número entre 0 y 100 que representa el porcentaje de satisfacción obtenido. Ejemplo: 77.

Fuente: elaboración propia.

También usamos la *tabla 8* para describir la colección *Evaluación*, en donde se especifican los nombres de las claves, los tipos de valor y una descripción que muestra las reglas de negocio a nivel de base de datos para cada clave y un ejemplo que satisface al valor.

Tabla 8. Descripción de las claves y los valores de la colección Evaluacion.

Clave	Tipo	Descripción
id_software_evaluado	int	Es un identificador tomado de un software creado previamente. Ejemplo: 1023.
tareas	object	• Es una colección de n tareas. • Cada tarea de la colección es simbolizada con las claves $s1, s2, \dots, sn$. • Toda tarea s es un vector de valores enteros de tamaño $m+1$, donde m es la cantidad de usuarios. • Para cada tarea s existe un valor de referencia ref en la primera posición que representa la cantidad de subtareas para dicha tarea. • En las demás posiciones de un vector s se encuentran las cantidades de subtareas realizadas por usuarios, representados por $U1, U2, \dots, Um$.
tiempos	object	• Es una colección de n tareas. • Cada tarea de la colección es simbolizada con las claves $t1, t2, \dots, tn$. • Toda tarea t es un vector de valores enteros de tamaño $m+1$, donde m es la cantidad de usuarios. • Para cada tarea t existe un valor de referencia ref en la primera posición que representa la duración normal o de referencia para dicha tarea.

		En las demás posiciones de un vector t se encuentran el tiempo en minutos que cada usuario toma para realizar dicha tarea, representados por $U1, U2, \dots, Um$.
puntajes	object	- Es una colección de n preguntas cerradas. - Cada pregunta cerrada de la colección es simbolizada con la clave $p1, p2, \dots, pn$. - Toda pregunta p es un vector de valores enteros de tamaño $m+1$, donde m es la cantidad de usuarios. - Para cada pregunta p existe un valor de ponderación $peso$ en la primera posición que representa el peso porcentual para dicha pregunta con respecto a las demás. - En las demás posiciones de un vector p se encuentran los puntajes asignados por cada usuario para dicha pregunta en valores de 1 a 5, representados por $U1, U2, \dots, Um$. - La suma de todos los pesos en el objeto <i>puntajes</i> debe ser 100.
comentarios	object	- Es una colección de n preguntas abiertas. - Cada pregunta de la colección es simbolizada con la clave $p1, p2, \dots, pn$. - Toda pregunta p es un vector de valores enteros de tamaño $m+1$, donde m es la cantidad de usuarios. - Para cada pregunta p existe un valor de ponderación $peso$ en la primera posición que

		representa el peso porcentual para dicha pregunta con respecto a las demás. • En las demás posiciones de un vector p se encuentran los comentarios de cada usuario como respuesta a dicha pregunta, representados por $U1, U2, \dots, Um$. • La suma de todos los pesos en el objeto <i>comentarios</i> debe ser 100.
--	--	---

Fuente: elaboración propia.

Adicionalmente, se usó una estructura de datos similar a la colección *Evaluación* para almacenar los resultados calculados, denominada colección *Resultados* y representada por el identificador "id_soft". Esta incluye porcentajes de resultados por subtarea en cada tarea, una lista de tiempos correspondientes a la duración de las tareas realizadas, y una lista de puntajes asignados a cada pregunta, todos organizados en el mismo orden secuencial.

Asimismo, se incorporó una lista de comentarios asociados a las tareas. Cada comentario contiene los valores "neg", "neu", "pos" y "comp", que representan la polaridad negativa, neutral, positiva y compuesta del texto, respectivamente. Cada conjunto de valores está vinculado a una pregunta específica, manteniendo el mismo orden de las tareas evaluadas.

4.2.6. Mockups de las vistas del sistema

En este apartado se muestran a continuación las mockups de las vistas de usuario del sistema. La *figura 8* muestra la vista principal en la cual se listan las evaluaciones ya creadas. En ella se ven las evaluaciones que están sin completar, es decir las que no tienen completos los datos requeridos para el cálculo de la usabilidad de un software en específico, y también las que ya tienen calculado el nivel de usabilidad, expresado mediante una barra porcentual. Adicionalmente, tiene una barra de búsqueda por nombre de software evaluado, y un botón para crear nueva evaluación. Este mockup está relacionado con el caso de uso *Gestionar evaluaciones*.

Figura 8. Mockup de la vista principal.



Fuente: elaboración propia.

Luego, en la *figura 9* se presenta la vista para el usuario con dos botones: uno para llenar los datos manualmente y otro para cargar los datos desde un archivo de formato separado por comas (CSV). Además, en la parte izquierda están varias pestañas que corresponden a los espacios para subir los valores de eficacia, eficiencia y preguntas cerradas, abiertas, y al final el espacio para mostrar la usabilidad total del sistema software evaluado. Este mockup está relacionado con los casos de uso *Gestionar evaluaciones* y *Cargar valores*.

Figura 9. Mockup de la vista al crear una evaluación.

Eficacia
Eficiencia
Preguntas cerradas
Preguntas abiertas
Usabilidad

+

Llenar datos
manualmente

Subir archivo
CSV

Fuente: elaboración propia.

Cuando los datos son cargados, sea manualmente o mediante archivo CSV aparece una tabla como se ve en la *figura 10*, donde aparecen, además, una pestaña de análisis y botones de guardar y analizar. Este mockup está relacionado con el caso de uso *Obtener nivel de usabilidad*.

Figura 10. Mockup de la vista al cargar los datos.

Datos

Análisis

	Tarea 1	Tarea 2	Tarea 3	Tarea 1
Valores de referencia	5	5	4	4
Usuario 1	3	4	1	1
Usuario 2	5	3	2	2
Usuario 3	4	4	3	4
Usuario 4	4	5	4	4
Usuario 5	2	4	1	2
Usuario 6	3	4	1	1

Cancelar

Analizar

Eficacia

Eficiencia

Preguntas cerradas

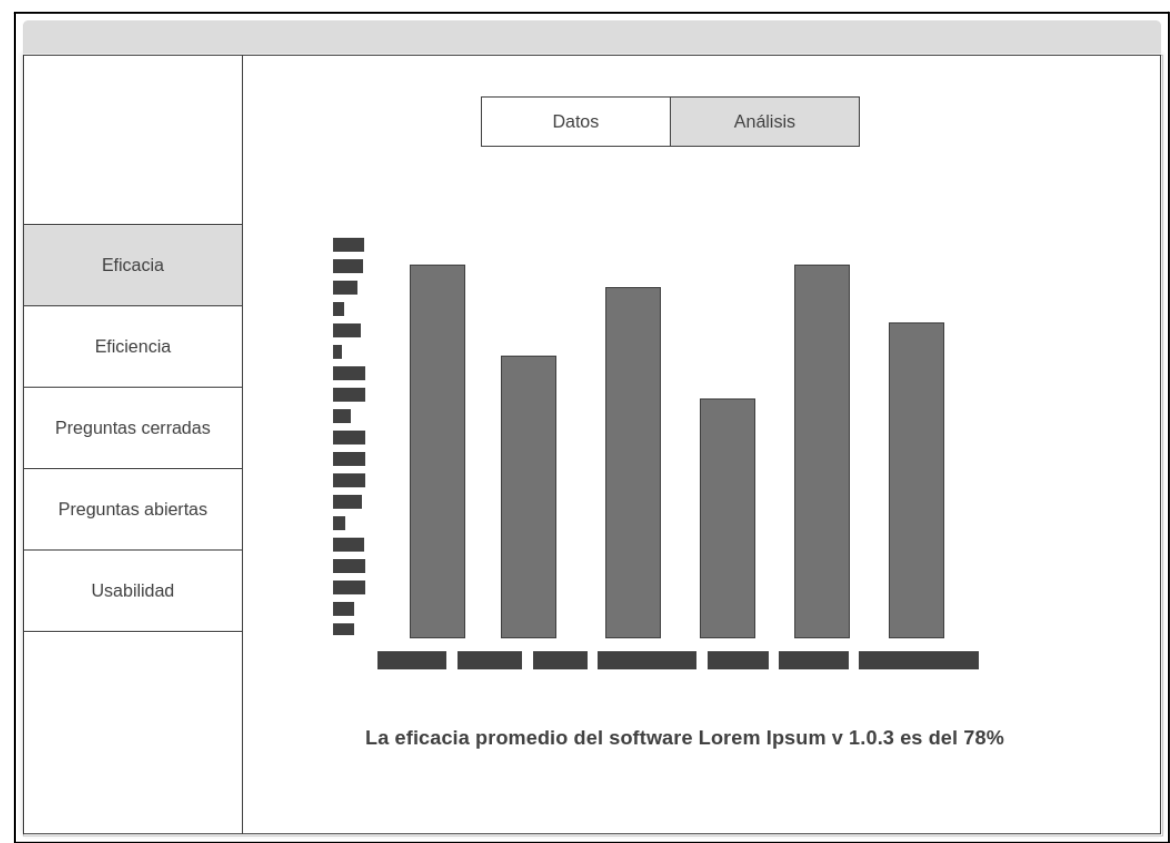
Preguntas abiertas

Usabilidad

Fuente: elaboración propia.

Por su parte, la opción “Analizar” muestra una gráfica con los valores por usuario, los cuales varían por atributo de acuerdo a la pestaña elegida en el menú. En la parte inferior, dentro de la pestaña análisis aparece una frase que describe el porcentaje del atributo, tal como se ve en la *figura 11*. Este mockup va con el caso de uso *Obtener reporte de resultados*.

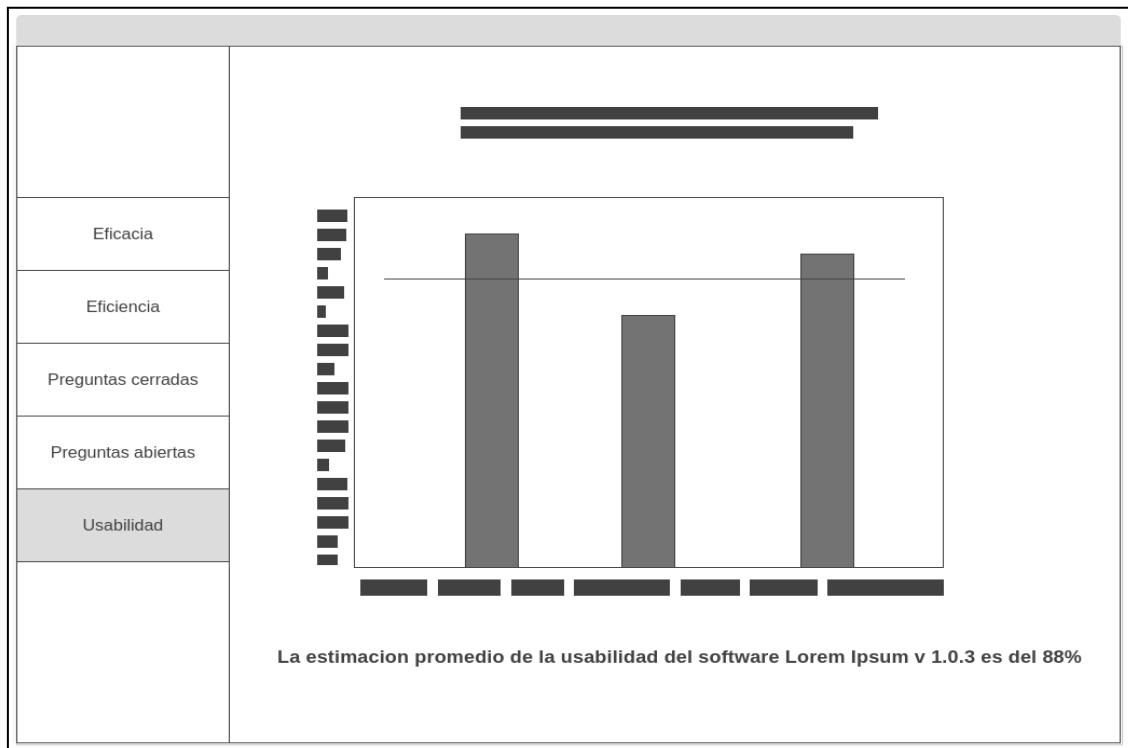
Figura 11. Mockup de la vista al analizar los datos.



Fuente: elaboración propia.

Finalmente, en la *figura 12*, se ve la gráfica que muestra la usabilidad del software con los valores de cada atributo con una frase que describe el porcentaje de usabilidad. Este mockup está relacionado con el caso de uso *Obtener reporte de resultados*.

Figura 12. Mockup de la vista al graficar la usabilidad.



Fuente: elaboración propia.

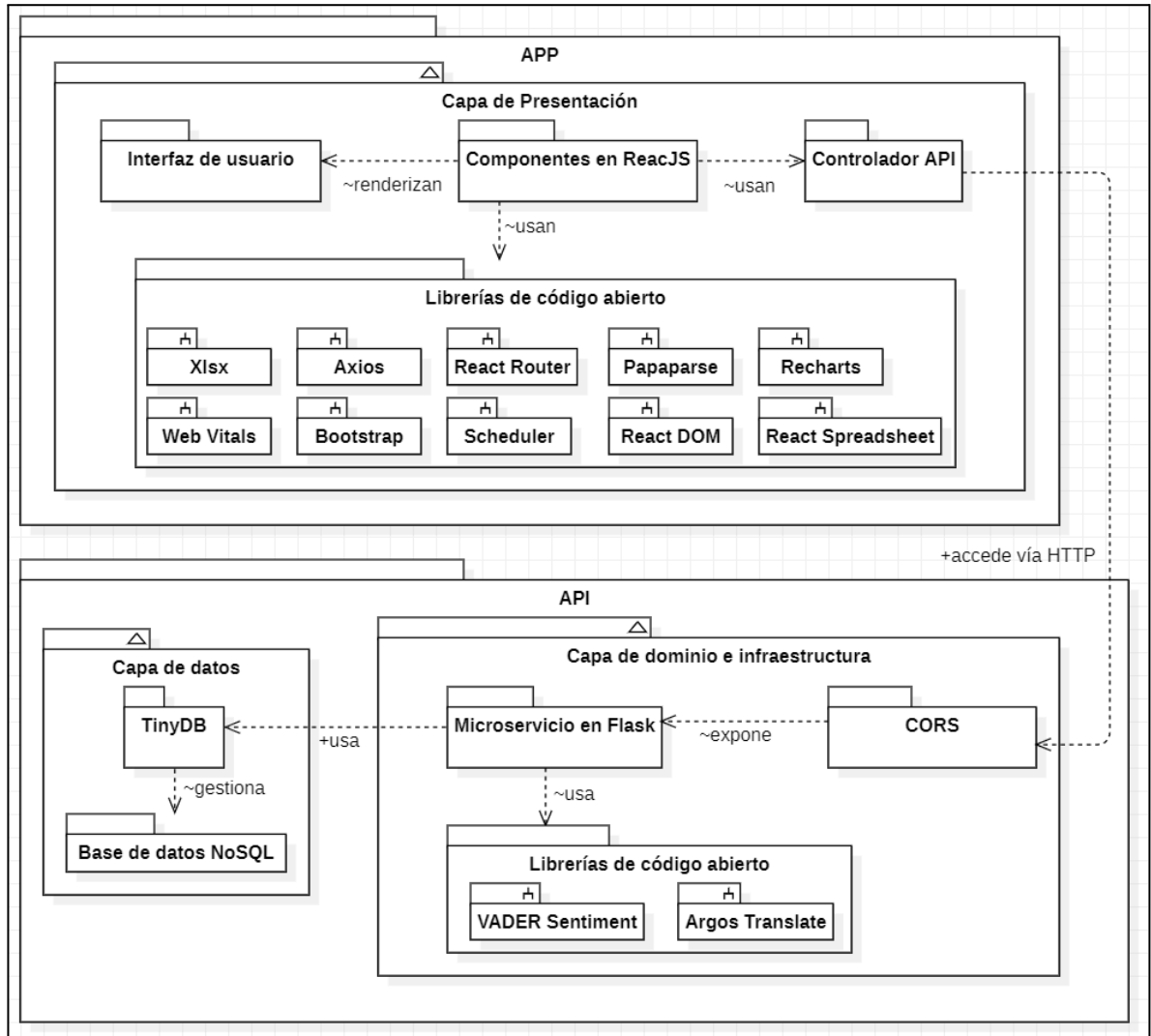
4.2.7. Vista de desarrollo del sistema

Desde la perspectiva arquitectónica propuesta por Kruchten en su modelo 4+1, la vista de desarrollo representa la organización estática del software en términos de sus componentes de implementación, tales como paquetes, módulos y subsistemas, y es fundamental para comprender la estructura del código y la asignación de responsabilidades entre desarrolladores (Kruchten, 2002). Para representar esta vista, se recurre al diagrama de paquetes, el cual permite visualizar la estructura del sistema a través de los paquetes que lo componen, las relaciones entre ellos y sus dependencias internas (Tonella y Potrich, 2005).

En la *figura 13* se presenta el diagrama de paquetes del sistema desarrollado, el cual se encuentra organizado en dos paquetes principales: APP y API. El paquete **APP** representa la capa de presentación, construida utilizando el framework ReactJS sobre el lenguaje de programación JavaScript. Esta capa contiene componentes que renderizan la interfaz de

usuario, y que a su vez utilizan diversas librerías de código abierto como Axios, Recharts, React Router, Bootstrap, entre otras. Estos componentes se comunican con el backend a través de un controlador API, estableciendo una interacción mediante peticiones HTTP hacia el microservicio.

Figura 13. Diagrama de paquetes del sistema.



Fuente: elaboración propia.

El paquete **API** agrupa la capa de dominio e infraestructura junto con la capa de datos. En la capa de dominio, se implementó un microservicio utilizando el framework Flask sobre Python, encargado de ejecutar el análisis de sentimientos y gestionar las solicitudes

provenientes del cliente. Para el análisis se utilizó la librería de código abierto VADER Sentiment, mientras que la traducción automática de comentarios, cuando fue requerida, se realizó con Argos Translate. El microservicio fue expuesto mediante el uso del paquete CORS (Cross-Origin Resource Sharing), el cual permitió el intercambio de recursos entre el cliente y el servidor dentro del mismo entorno local, respetando las políticas de seguridad del navegador.

Finalmente, en la capa de datos, el microservicio gestionó una base de datos NoSQL mediante el uso de la biblioteca TinyDB. Esta configuración permite almacenar de forma eficiente los comentarios procesados y sus resultados, asegurando una separación clara entre la lógica de negocio, la interfaz y el almacenamiento. La organización modular reflejada en este diagrama facilita la capacidad de mantenimiento, escalabilidad y comprensión del sistema durante su desarrollo.

En cuanto a la notación del diagrama, el símbolo (+) denota relaciones con visibilidad pública, es decir, accesibles entre paquetes; mientras que la virgulilla (~) indica relaciones internas dentro de un mismo paquete, como dependencias entre componentes o uso de librerías.

4.2.8. Elección de tecnologías de desarrollo

A continuación, en la *tabla 9*, se listan las tecnologías, librerías, paquetes o dependencias implementadas en el desarrollo del proyecto en la parte backend.

Tabla 9. Tecnologías implementadas en el backend.

Dependencia	Versión	Descripción
Flask	2.3.2	Flask es un framework web ligero y flexible para el lenguaje de programación Python. Se utiliza para crear aplicaciones web y servicios RESTful. En este proyecto, Flask se utiliza para implementar el backend del sistema de pruebas de usabilidad.

Itsdangerous	2.1.2	Itsdangerous es una biblioteca para manejar aspectos de seguridad en Flask. Es utilizado para garantizar la autenticidad y la integridad de los datos en el proyecto de pruebas de usabilidad.
TinyDB	4.7.1	TinyDB es una base de datos de documentos ligera y fácil de usar en Python. Puede estar siendo utilizado para almacenar y recuperar información relacionada con las tareas, usuarios y comentarios en el proyecto de pruebas de usabilidad.
Werkzeug	2.3.4	Werkzeug es una biblioteca utilizada internamente por Flask para manejar la manipulación de URLs, el enrutamiento y otros aspectos del desarrollo web. En este proyecto, se utiliza en conjunto con Flask para implementar el servidor web.
VaderSentiment	3.3.2	VADER (Valence Aware Dictionary and sEntiment Reasoner) es una herramienta de análisis de sentimientos basada en reglas y léxico que está específicamente en sintonía con los sentimientos expresados en las redes sociales y funciona bien en textos de otros dominios. Es usada en este proyecto para determinar la polaridad de los comentarios de los usuarios.
argostranslate	1.9.6	Herramienta para traducción automática de texto utilizando modelos de traducción neuronales. Es usada para traducir los comentarios al idioma inglés antes de ser analizados por la librería VaderSentiment.
Flask_cors	4.0.0	Flask-Cors es una biblioteca popular que facilita la configuración de CORS en una API de Flask. Es usada para

		permitir el acceso de recursos desde un puerto distinto al del microservicio.
--	--	---

Fuente: elaboración propia.

Por su parte, en la *tabla 10*, se listan las tecnologías, librerías, paquetes o dependencias implementadas en el desarrollo del proyecto en la parte frontend.

Tabla 10. Tecnologías implementadas en el frontend.

Dependencia	Versión	Descripción
@babel/plugin-proposal-private-property-in-object	7.16.7	Este plugin de Babel permite la sintaxis de propiedades privadas en objetos en JavaScript. Es utilizado para mejorar la encapsulación y privacidad en el código fuente del frontend.
@fortawesome/free-solid-svg-icons	6.4.0	Font Awesome es una biblioteca de iconos populares. Esta dependencia proporciona una colección de iconos de estilo sólido para usar en la interfaz de usuario.
@fortawesome/react-fontawesome	0.2.0	React Font Awesome es una biblioteca que permite utilizar los iconos de Font Awesome en aplicaciones React. Es utilizado para renderizar iconos en la interfaz de usuario del frontend.
@testing-library/jest-dom	5.16.5	Testing Library es una suite de utilidades para probar componentes de React. Jest DOM es una de las dependencias de Testing Library que proporciona funciones de aserción específicas para Jest. Está relacionado con las pruebas unitarias.
@testing-library/react	13.4.0	Testing Library es una suite de utilidades para probar componentes de React. Esta dependencia proporciona las

		herramientas necesarias para realizar pruebas unitarias en componentes React.
@testing-library/ user-event	13.5.0	Testing Library es una suite de utilidades para probar componentes de React. User Event es una de las dependencias de Testing Library que permite simular eventos del usuario en las pruebas.
Axios	1.4.0	Axios es una biblioteca para realizar solicitudes HTTP desde el navegador o desde Node.js. Es utilizado para realizar solicitudes al backend desde el frontend y manejar la comunicación con la API del proyecto.
Bootstrap	5.3.0	Bootstrap es un framework CSS popular para crear interfaces de usuario responsivas. Es usado para estilizar y diseñar la apariencia de los componentes en el frontend del proyecto.
Papaparse	5.4.1	Papa Parse es una biblioteca para analizar archivos CSV y texto delimitado. Es usado para procesar y analizar datos de forma estructurada en el frontend del proyecto.
React	18.2.0	React es una biblioteca de JavaScript para construir interfaces de usuario. Es la base del desarrollo de aplicaciones en React y se utiliza para crear componentes reutilizables y gestionar el estado en el frontend.
React-bootstrap	2.8.0	React Bootstrap es una biblioteca que proporciona componentes de Bootstrap para su uso en aplicaciones React. Es utilizado para crear componentes con estilos de Bootstrap en el frontend.

React-dom	18.2.0	React DOM es una biblioteca que proporciona la integración de React con el modelo de objetos del documento (DOM) en el navegador. Se utiliza para renderizar componentes React en el frontend.
React-router-dom	6.13.0	React Router DOM es una biblioteca que facilita la navegación y el enrutamiento en aplicaciones React. Es utilizado para manejar las rutas y la navegación entre diferentes componentes en el frontend del proyecto.
React-scripts	5.0.1	React Scripts es una colección de scripts y configuraciones predefinidas para el entorno de desarrollo de aplicaciones React. Se utiliza para compilar y ejecutar la aplicación en modo de desarrollo en el frontend.
React-spreadsheet	0.8.3	React Spreadsheet es una biblioteca que proporciona una hoja de cálculo interactiva en React. Es utilizada para mostrar y manipular datos tabulares en forma de hoja de cálculo en el frontend del proyecto.
Recharts	2.7.2	Recharts es una biblioteca de gráficos para React. Proporciona componentes para crear visualizaciones de datos, como gráficos de barras, gráficos de líneas y gráficos de pastel. Es utilizado para mostrar datos en forma gráfica en el frontend.
Scheduler	0.23.0	Scheduler es una biblioteca que proporciona programación reactiva en JavaScript. Es utilizada para manejar tareas programadas o repetitivas en el frontend del proyecto.
Web-Vitals	2.1.4	Web Vitals es una biblioteca que ayuda a medir y rastrear las métricas de rendimiento web clave. Es utilizado para

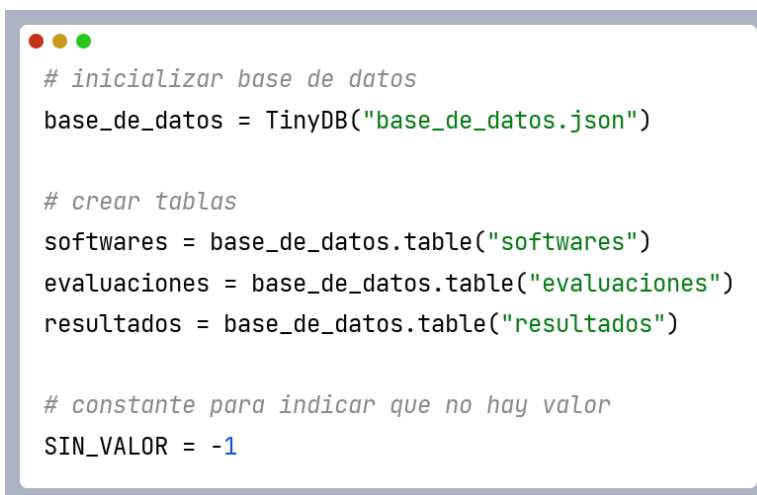
		monitorear el rendimiento y la experiencia del usuario en el frontend del proyecto.
XLSX	0.18.5	XLSX es una biblioteca para leer y escribir archivos de hojas de cálculo Excel en formato XLSX. Es utilizada para importar y exportar datos en formato Excel en el frontend del proyecto.

Fuente: elaboración propia.

4.2.9. Desarrollo del lado del servidor

En esta sección se muestran algunas partes del código fuente del microservicio desarrollado en Flask en Python, que también se puede encontrar en el repositorio en el [Anexo A](#). En la *figura 14* se muestra la instancia de la base de datos con TinyDB y la creación de las tablas usadas para almacenar los datos suministrados. Se define una constante llamada “SIN_VALOR” y se le asigna el valor -1 para indicar que no hay un valor en una variable en particular.

Figura 14. Fragmento del código fuente en la parte del servidor.



```

# inicializar base de datos
base_de_datos = TinyDB("base_de_datos.json")

# crear tablas
softwares = base_de_datos.table("softwares")
evaluaciones = base_de_datos.table("evaluaciones")
resultados = base_de_datos.table("resultados")

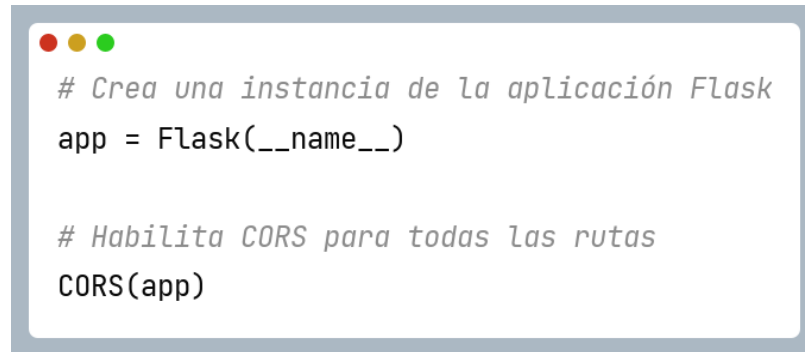
# constante para indicar que no hay valor
SIN_VALOR = -1

```

Fuente: elaboración propia.

En la *figura 15* se puede leer que se crea una instancia de la aplicación Flask y se habilita CORS para todas las rutas.

Figura 15. Fragmento del código fuente del lado del servidor.



```
# Crea una instancia de la aplicación Flask  
app = Flask(__name__)  
  
# Habilita CORS para todas las rutas  
CORS(app)
```

Fuente: elaboración propia.

En la *figura 16* se muestra como se El código define una ruta POST en Flask para crear un nuevo software. Verifica que los campos “nombre” y “versión” estén presentes en la solicitud JSON. Luego, inserta un documento en la colección de softwares con datos predeterminados, como un ID vacío y fechas. También crea y almacena documentos vacíos en las colecciones evaluaciones y resultados relacionados con el software. Si todo es exitoso, responde con un mensaje de éxito y el ID del software; si ocurre algún error, responde con un mensaje de error detallado.

Figura 16. Código fuente para crear nuevo software.

```
@app.route('/nuevo_soft', methods=['POST'])
def nuevo_software():
    """Crea un nuevo software..."""

    # Verificar si el campo "nombre" y "version" están presentes en la solicitud JSON
    if "nombre" not in request.json or "version" not in request.json:
        response = {"error": "Campos 'nombre' y 'version' faltantes en la solicitud"}
        return jsonify(response), 400

    soft = request.json

    # Crear el documento de software
    software = {...}

    # Insertar el documento de software en la colección "softwares"
    id_gen = softwares.insert(software)

    # Actualizar el campo "id" con el ID generado
    softwares.update({"id_soft": id_gen}, doc_ids=[id_gen])

    # Crear el documento de evaluación asociado al software
    evaluacion = {...}

    # Insertar el documento de evaluación en la colección "evaluaciones"
    evaluaciones.insert(evaluacion)

    # Crear el documento de resultado asociado al software
    resultado = {...}

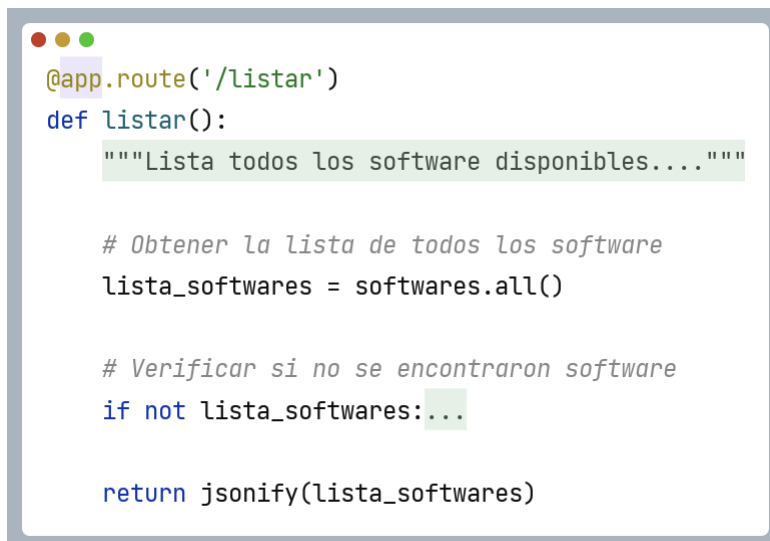
    # Insertar el documento de resultado en la colección "resultados"
    resultados.insert(resultado)

    response = {"message": "Software creado exitosamente"}
    return jsonify(response)
```

Fuente: elaboración propia.

Por otro lado, en la *figura 17* el código define una ruta en Flask que lista todos los softwares en la base de datos. Si no se encuentran, devuelve un mensaje de error con un código 404; si hay resultados, los retorna en formato JSON.

Figura 17. Código fuente para listar softwares.

A screenshot of a code editor window with a light blue border and three colored window control buttons (red, yellow, green) in the top-left corner. The code is written in Python and defines a Flask route. The code is as follows:

```
@app.route('/listar')
def listar():
    """Lista todos los software disponibles...."""

    # Obtener la lista de todos los software
    lista_softwares = softwares.all()

    # Verificar si no se encontraron software
    if not lista_softwares:...

    return jsonify(lista_softwares)
```

Fuente: elaboración propia.

En la *figura 18* se puede observar el flujo del proceso que define una ruta DELETE en Flask para eliminar un software y sus datos asociados. Primero, se verifica que el campo “id_soft” esté presente en la solicitud. Luego, se eliminan el software y sus registros en las colecciones “softwares”, “evaluaciones” y “resultados”. Si la operación es exitosa, se devuelve un mensaje de confirmación; si falta el campo, se responde con un error 400.

Figura 18. Código fuente para la eliminación de softwares.

```
@app.route('/eliminar_soft', methods=['DELETE'])
def eliminar_soft():
    """Elimina un software y sus datos asociados..."""

    # Verificar si el campo "id_soft" está presente en la solicitud JSON
    if "id_soft" not in request.json: ...

    # Obtener el ID del software especificado y convertirlo a entero
    id_soft = int(request.json["id_soft"])

    # Eliminar el software y sus datos asociados
    softwares.remove(Query().id_soft == id_soft)
    evaluaciones.remove(Query().id_soft == id_soft)
    resultados.remove(Query().id_soft == id_soft)

    response = {"message": "Borrado exitosamente"}
    return jsonify(response)
```

Fuente: elaboración propia.

En la *figura 19* se puede observar el proceso de la ruta “/guardar_tareas”, que maneja una solicitud POST para guardar tareas de un usuario en un software. El código valida que la solicitud sea un JSON y contenga los campos necesarios. Si el “id_soft” no existe, responde con un error 404. Si el software existe, calcula la eficacia y actualiza la base de datos, devolviendo un mensaje de éxito o error según corresponda.

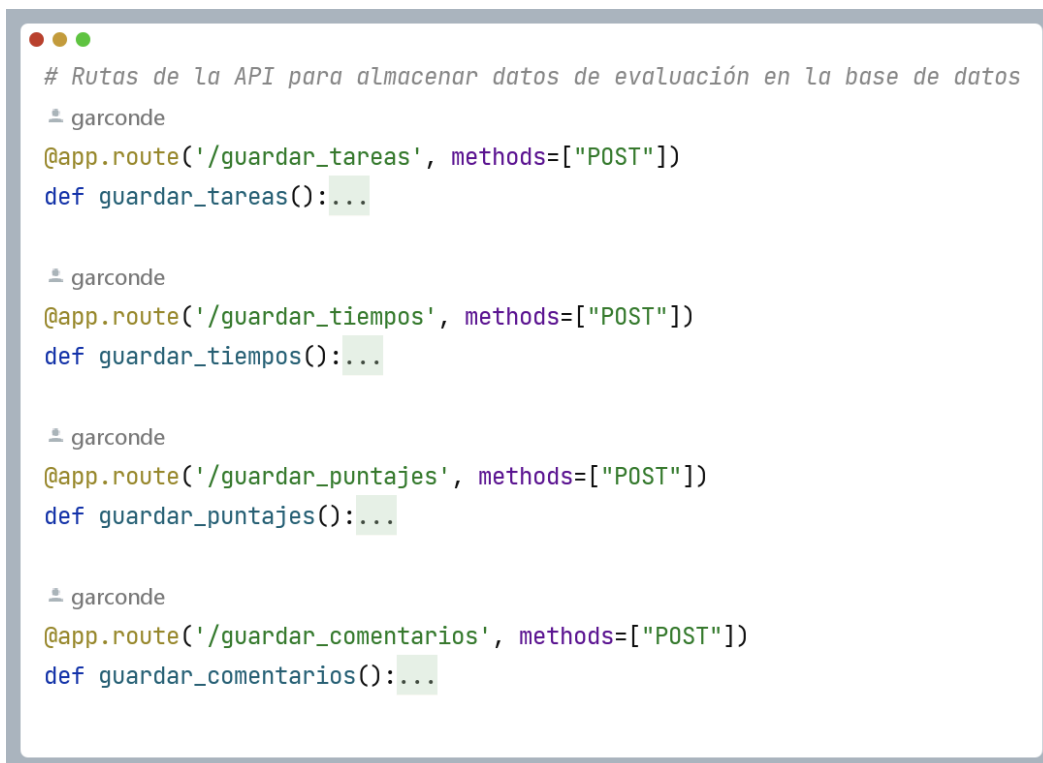
Figura 19. Código fuente para guardar tareas de un usuario.

```
@app.route('/guardar_tareas', methods=["POST"])
def guardar_tareas():
    try:
        if not request.is_json:
            return jsonify({"error": "El cuerpo de la solicitud debe ser un JSON"}), 400
        r = request.json
        if "id_soft" not in r or "tareas" not in r:
            return jsonify({"error": "Faltan campos obligatorios en el JSON"}), 400
        id_soft = int(r["id_soft"])
        if not softwares.contains(doc_id=id_soft):
            return jsonify({"error": f"No se encontró el id_soft {id_soft} en la base de datos"}), 404
        tareas = r["tareas"]
        try:
            calcular_eficacia(id_soft, tareas)
        except ValueError as e:
            return jsonify({"error": ": " + str(e)}), 400
        except ZeroDivisionError as e:
            return jsonify({"error": "error de división por cero: " + str(e)}), 400
        except Exception as e:
            return jsonify({"error": "error inesperado: " + str(e)}), 500
        evaluaciones.update({"tareas": tareas, Query().id_soft == id_soft})
        return jsonify({"mensaje": "Tareas asignadas exitosamente"}), 200
    except Exception as e:
        return jsonify({"error": ": " + str(e)}), 500
```

Fuente: elaboración propia.

Parecido al procedimiento anterior, en la *figura 20* se puede observar el flujo de varias rutas de la API para almacenar datos de evaluación en la base de datos. Cada ruta recibe una solicitud POST con datos JSON que incluyen el ID del software y los datos correspondientes (tareas, tiempos, puntajes o comentarios). Primero, se valida la solicitud para asegurarse de que contiene los campos necesarios y que el `id_soft` existe en la base de datos. Luego, dependiendo de la ruta, se calculan métricas (eficacia, eficacia, satisfacción por puntajes y satisfacción por comentarios) y se actualizan los datos en la base de datos. Si todo es correcto, se responde con un mensaje de éxito (código 200); si ocurre un error, se maneja con respuestas adecuadas y códigos de error.

Figura 20. Código fuente para almacenar datos de evaluación.



```
# Rutas de la API para almacenar datos de evaluación en la base de datos
# garconde
@app.route('/guardar_tareas', methods=["POST"])
def guardar_tareas():...

# garconde
@app.route('/guardar_tiempos', methods=["POST"])
def guardar_tiempos():...

# garconde
@app.route('/guardar_puntajes', methods=["POST"])
def guardar_puntajes():...

# garconde
@app.route('/guardar_comentarios', methods=["POST"])
def guardar_comentarios():...
```

Fuente: elaboración propia.

Como se ve en la *figura 21*, el método “calcular_eficacia” hace el cálculo de la eficacia de las tareas asignadas a un software, comparando los resultados de los usuarios con valores de referencia. Primero, valida las entradas para asegurar que los datos sean correctos, luego calcula la eficacia de cada usuario dividiendo sus resultados entre las referencias correspondientes. Posteriormente, obtiene el porcentaje promedio de eficacia de todos los usuarios y lo guarda en la base de datos. Finalmente, actualiza el estado del software indicando que ha sido analizado.

Figura 21. Código fuente para calcular la eficacia.

```
def calcular_eficacia(id_soft, tareas):
    if not isinstance(tareas, list) or len(tareas) < 2:
        raise ValueError("La lista de tareas debe contener al menos dos elementos.")
    eficacia_usuarios = []
    referencias = tareas[0]
    if not all(isinstance(e, int) for e in referencias):
        raise ValueError("Los valores de la lista de referencias deben ser numéricos.")
    if any(e == 0 for e in referencias):
        raise ZeroDivisionError("No se puede dividir por cero. Algún elemento de la lista de referencias es cero.")
    if any(e < 0 for e in referencias):
        raise ValueError("No se pueden proporcionar valores negativos en la lista de referencias.")
    for tarea in tareas[1:]:
        cont = 0
        valores = []
        for e in tarea:
            if not isinstance(e, int):
                raise ValueError("Los valores de las tareas deben ser numéricos.")
            if e < 0:
                raise ValueError("No se pueden proporcionar valores negativos en los valores.")
            if e > referencias[cont]:
                raise ValueError("Los valores de las tareas no pueden ser mayores que los de la lista de referencias.")
            operacion = e / referencias[cont]
            valores.append(operacion)
            cont = cont + 1
        prom_usuario = (sum(valores) / len(valores))
        porcentaje_usuario = round(prom_usuario * 100)
        eficacia_usuarios.append(porcentaje_usuario)
    eficacia_porcentaje = round(sum(eficacia_usuarios) / len(eficacia_usuarios))
    resultados.update({"tareas": eficacia_usuarios}, Query().id_soft == id_soft)
    softwares.update({"eficacia": eficacia_porcentaje}, Query().id_soft == id_soft)
    es_analizado(id_soft)
```

Fuente: elaboración propia.

Como se ve en la *figura 22*, el método “calcular_eficiencia” calcula la eficiencia de las tareas asignadas a un usuario, comparando los tiempos tomados con los tiempos de referencia. Valida las entradas para asegurarse de que los datos sean correctos, luego calcula la eficiencia de cada usuario dividiendo los tiempos de referencia entre los tiempos registrados. Después, obtiene el porcentaje promedio de eficiencia de todos los usuarios y lo guarda en la base de datos. Al final, actualiza el estado del software indicando que ha sido analizado.

Figura 22. Código fuente para calcular la eficiencia.

```
def calcular_eficiencia(id_soft, tiempos):
    if not isinstance(tiempos, list) or len(tiempos) < 2:
        raise ValueError("La lista de tiempos debe contener al menos dos elementos.")
    eficiencia_usuarios = []
    referencias = tiempos[0]
    if not all(isinstance(e, int) for e in referencias):
        raise ValueError("Los valores de la lista de referencias deben ser numéricos.")
    if any(e < 0 for e in referencias):
        raise ValueError("No se pueden proporcionar valores negativos en la lista de referencias.")
    for tiempo in tiempos[1:]:
        cont = 0
        valores = []
        for e in tiempo:
            if not isinstance(e, int):
                raise ValueError("Los valores de las tiempos deben ser numéricos.")
            if e == 0:
                raise ZeroDivisionError("No se puede dividir por cero. Algún valor de la lista de tiempos es cero.")
            if e < 0:
                raise ValueError("No se pueden proporcionar valores negativos en los valores.")
            operacion = referencias[cont] / e
            valores.append(operacion)
            cont = cont + 1
        prom_usuario = (sum(valores) / len(valores))
        porcentaje_usuario = round(prom_usuario * 100)
        eficiencia_usuarios.append(porcentaje_usuario)
    eficacia_porcentaje = round(sum(eficiencia_usuarios) / len(eficiencia_usuarios))
    resultados.update({"tiempos": eficiencia_usuarios}, Query().id_soft == id_soft)
    softwares.update({"eficiencia": eficacia_porcentaje}, Query().id_soft == id_soft)
    es_analizado(id_soft)
```

Fuente: elaboración propia.

En la *figura 23* se puede ver el tercer método, “calcular_sat_puntajes”, el cual tiene como objetivo calcular la satisfacción de los usuarios de un software basada en los puntajes obtenidos en preguntas cerradas. Recibe un identificador del software (id_soft) y una lista de puntajes, donde la primera lista contiene los pesos de cada pregunta y las siguientes listas contienen los puntajes asignados por los usuarios para esas preguntas. El método valida que los puntajes sean numéricos y se encuentren en el rango de 1 a 5. Luego, para cada usuario, calcula la satisfacción multiplicando cada puntaje por su peso correspondiente y convirtiendo el resultado a un porcentaje. Después de procesar los puntajes de todos los usuarios, se calcula un puntaje promedio de satisfacción para cada uno, que se almacena en la base de datos. Finalmente, el método calcula la satisfacción promedio general,

actualizando esta información en la base de datos y también llama a otro método para calcular la satisfacción general.

Figura 23. Código fuente de calcular la satisfacción por puntajes.

```
def calcular_sat_puntajes(id_soft, puntajes):
    if not isinstance(puntajes, list) or len(puntajes) < 2:
        raise ValueError("La lista de puntajes debe contener al menos dos elementos.")
    puntajes_usuarios = []
    pesos = puntajes[0]
    if not all(isinstance(e, int) for e in pesos):
        raise ValueError("Los valores de la lista de pesos deben ser numéricos.")
    if any(e < 0 for e in pesos):
        raise ValueError("No se pueden proporcionar valores negativos en la lista de pesos.")
    for puntaje in puntajes[1:]:
        cont = 0
        valores = []
        for e in puntaje:
            if not isinstance(e, int):
                raise ValueError("Los valores de las puntajes deben ser numéricos.")
            if e < 1 or e > 5:
                raise ValueError("Los valores de las puntajes deben estar entre 1 y 5.")
            if e < 0:
                raise ValueError("No se pueden proporcionar valores negativos en los valores.")
            operacion = e * pesos[cont] * 20
            valores.append(operacion)
            cont = cont + 1
        porcentaje_usuario = round(sum(valores) / sum(pesos))
        puntajes_usuarios.append(porcentaje_usuario)
    puntajes_porcentaje = round(sum(puntajes_usuarios) / len(puntajes_usuarios))
    resultados.update({"puntajes": puntajes_usuarios}, Query().id_soft == id_soft)
    softwares.update({"satisfaccion_pun": puntajes_porcentaje}, Query().id_soft == id_soft)
    calcular_satisfaccion(id_soft)
```

Fuente: elaboración propia.

En la *figura 24* se ve el método “calcular_sat_comentarios” para calcular la satisfacción de los usuarios de un software basándose en los comentarios de las preguntas abiertas. Recibe un identificador del software (id_soft) y una lista de comentarios, donde la primera lista contiene los pesos de cada comentario, y las siguientes listas contienen los comentarios realizados por los usuarios. Utiliza el analizador de sentimientos VADER para evaluar el tono de cada comentario, extrayendo cuatro métricas: negativa, neutral, positiva y

compuesta. Cada una de estas métricas se convierte en un porcentaje y se ajusta según el peso correspondiente.

Luego, se calcula un promedio ponderado de cada tipo de sentimiento (negativo, neutral, positivo y compuesto) para cada usuario, y los resultados se almacenan en la base de datos. Al final, el método calcula la satisfacción promedio general a partir del sentimiento compuesto de los comentarios y actualiza la base de datos, también invocando otro método para calcular la satisfacción general del software.

Figura 24. Cálculo de la satisfacción por comentarios.

```
def calcular_sat_comentarios(id_soft, comentarios):
    if not isinstance(comentarios, list) or len(comentarios) < 2:
        raise ValueError("La lista de puntajes debe contener al menos dos elementos.")
    comentarios_usuarios = []
    pesos = comentarios[0]
    analizador = SentimentIntensityAnalyzer()
    for comentario in comentarios[1:]:
        cont = 0
        valores_neg = []
        valores_neu = []
        valores_pos = []
        valores_comp = []
        porcentaje_usuario = {}
        for e in comentario:
            polaridad = {}
            if not isinstance(e, str):
                raise ValueError("Los valores de las puntajes deben ser cadenas de texto.")
            puntaje = analizador.polarity_scores(e)
            valor_neg = puntaje['neg']
            valor_neu = puntaje['neu']
            valor_pos = puntaje['pos']
            valor_compuesto = puntaje['compound']
            porc_neg = round(((valor_neg + 1) / 2) * 100)
            porc_neu = round(((valor_neu + 1) / 2) * 100)
            porc_pos = round(((valor_pos + 1) / 2) * 100)
            porc_comp = round(((valor_compuesto + 1) / 2) * 100)
            polaridad['neg'] = round((porc_neg * pesos[cont]))
            polaridad['neu'] = round((porc_neu * pesos[cont]))
            polaridad['pos'] = round((porc_pos * pesos[cont]))
            polaridad['comp'] = round((porc_comp * pesos[cont]))
            valores_neg.append(polaridad['neg'])
            valores_neu.append(polaridad['neu'])
            valores_pos.append(polaridad['pos'])
            valores_comp.append(polaridad['comp'])
            cont = cont + 1
        porcentaje_usuario['neg'] = round((sum(valores_neg) / sum(pesos)))
        porcentaje_usuario['neu'] = round((sum(valores_neu) / sum(pesos)))
        porcentaje_usuario['pos'] = round((sum(valores_pos) / sum(pesos)))
        porcentaje_usuario['comp'] = round((sum(valores_comp) / sum(pesos)))
        comentarios_usuarios.append(porcentaje_usuario)
    suma = sum(c['comp'] for c in comentarios_usuarios)
    cant = len(comentarios_usuarios)
    comentarios_porcentaje = round(suma / cant)
    resultados.update({"comentarios": comentarios_usuarios}, Query().id_soft == id_soft)
    softwares.update({"satisfaccion_com": comentarios_porcentaje}, Query().id_soft == id_soft)
    calcular_satisfaccion(id_soft)
```

Fuente: elaboración propia.

Finalmente, Se ve en la *figura 25* que el método “calcular_satisfaccion” calcula la satisfacción general de un software a partir de los valores `satisfaccion_pun` y `satisfaccion_com`, que reflejan la satisfacción con los puntajes de preguntas cerradas y los comentarios, respectivamente. Si alguno de estos valores es `SIN_VALOR`, el método no calculará la satisfacción general y actualizará el campo con este valor. Si ambos valores son válidos, calcula su promedio y actualiza la base de datos con la satisfacción general del software. Finalmente, se actualiza el estado del análisis del software.

Figura 25. Código fuente para el cálculo de la satisfacción.

```
def calcular_satisfaccion(id_soft):
    puntuacion_satisfaccion = softwares.get(Query().id_soft == id_soft)["satisfaccion_pun"]
    comentario_satisfaccion = softwares.get(Query().id_soft == id_soft)["satisfaccion_com"]
    if puntuacion_satisfaccion == SIN_VALOR or comentario_satisfaccion == SIN_VALOR:
        softwares.update({"satisfaccion": SIN_VALOR}, Query().id_soft == id_soft)
    else:
        satisfaccion_promedio = round((puntuacion_satisfaccion + comentario_satisfaccion) / 2)
        softwares.update({"satisfaccion": satisfaccion_promedio}, Query().id_soft == id_soft)
    es_analizado(id_soft)
```

Fuente: elaboración propia.

En la *figura 26* se ve que el método “es_analizado” actualiza el campo “usabilidad” de un software utilizando los valores de “eficacia”, “eficiencia” y “satisfacción”. Primero, obtiene estos valores del software según su ID. Luego, si todos los valores son válidos, calcula la usabilidad promedio redondeada y la actualiza en la base de datos. Finalmente, marca el software como analizado.

Figura 26. Método para validar software analizado.

```
def es_analizado(id_soft):
    eficacia = softwares.get(Query().id_soft == id_soft)["eficacia"]
    eficiencia = softwares.get(Query().id_soft == id_soft)["eficiencia"]
    satisfaccion = softwares.get(Query().id_soft == id_soft)["satisfaccion"]
    if eficacia > SIN_VALOR and eficiencia > SIN_VALOR and satisfaccion > SIN_VALOR:
        usabilidad = round((eficacia + eficiencia + satisfaccion) / 3)
        softwares.update({"usabilidad": usabilidad}, Query().id_soft == id_soft)
        softwares.update({"analizado": True}, Query().id_soft == id_soft)
```

Fuente: elaboración propia.

Como se ve en la *figura 27*, las rutas definidas en esa parte del código proporcionan diversos endpoints para obtener información sobre un software específico a través de solicitudes POST. Cada ruta recibe un JSON con el campo “id_soft”, que es utilizado para buscar el software o los datos relacionados en la base de datos. Dependiendo del tipo de información solicitada, ya sea tareas, tiempos, puntajes o comentarios, las rutas consultan la base de datos para devolver los resultados correspondientes. Si no se encuentra el software o la información solicitada, se devuelve un mensaje de error con el código de estado adecuado.

Figura 27. Rutas y métodos para obtener datos de la BD.

```
@app.route('/obtener_soft', methods=['POST'])
def obtener_soft():...

@app.route('/obtener_val_tareas', methods=['POST'])
def obtener_val_tareas():...

@app.route('/obtener_val_tiempos', methods=['POST'])
def obtener_val_tiempos():...

@app.route('/obtener_val_puntajes', methods=['POST'])
def obtener_val_puntajes():...

@app.route('/obtener_val_comentarios', methods=['POST'])
def obtener_val_comentarios():...

@app.route('/obtener_res_tareas', methods=['POST'])
def obtener_res_tareas():...

@app.route('/obtener_res_tiempos', methods=['POST'])
def obtener_res_tiempos():...

@app.route('/obtener_res_puntajes', methods=['POST'])
def obtener_res_puntajes():...

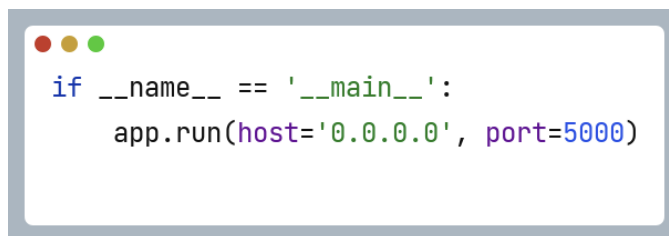
@app.route('/obtener_res_comentarios', methods=['POST'])
def obtener_res_comentarios():...
```

Fuente: elaboración propia.

En último lugar, como se puede ver en la *figura 28*, el bloque de código al final del script es responsable de ejecutar la aplicación Flask. Cuando el script se ejecuta directamente, la aplicación se inicia en el host '0.0.0.0' y en el puerto 5000, permitiendo que sea accesible desde cualquier dispositivo en la misma red local. Este bloque asegura que la aplicación

esté activa y escuchando solicitudes, proporcionando los servicios definidos en las rutas configuradas a lo largo del script.

Figura 28. Código para ejecutar la aplicación Flask.

A code editor window with a light blue border and three colored window control buttons (red, yellow, green) in the top-left corner. The editor contains two lines of Python code: `if __name__ == '__main__':` and `app.run(host='0.0.0.0', port=5000)`. The code is color-coded: `if` is blue, `__name__` is black, `==` is black, `'__main__'` is in single quotes, `:` is black, `app` is black, `.` is black, `run` is black, `(` is black, `host` is black, `=` is black, `'0.0.0.0'` is in single quotes, `,` is black, `port` is black, `=` is black, `5000` is black, and `)` is black.

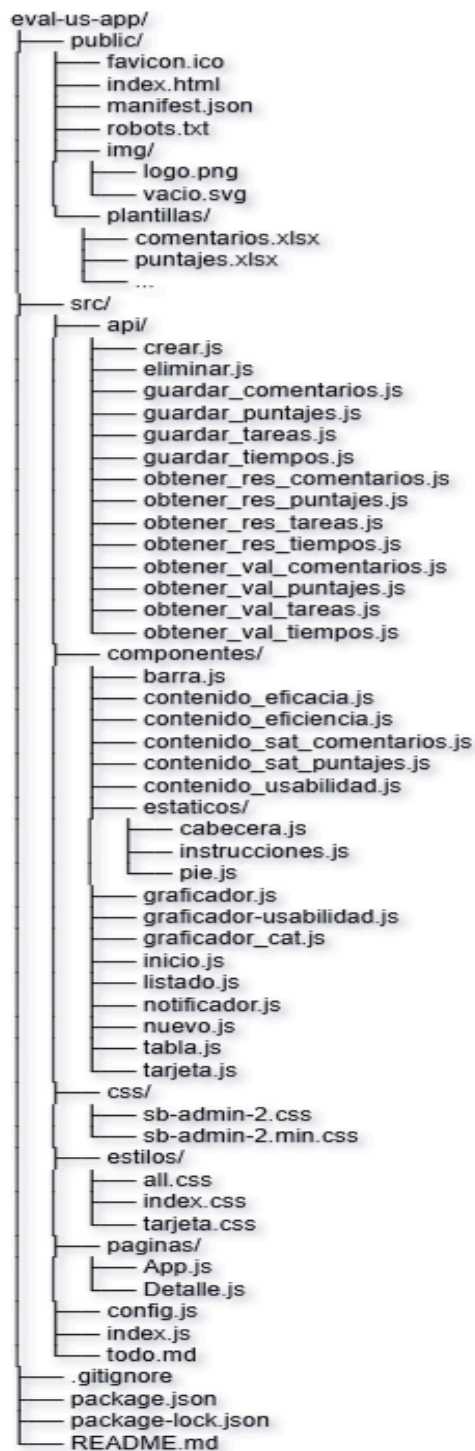
```
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Fuente: elaboración propia.

4.2.10. Desarrollo del lado del cliente

El desarrollo del lado del cliente se centra en ofrecer una interfaz intuitiva para que los usuarios interactúen con el sistema. Aquí se detallan las principales funcionalidades de la vista del usuario y el código utilizado para su implementación. En primer lugar, en la *figura 29*, se muestra una estructura de carpetas del proyecto del lado del cliente.

Figura 29. Estructura del proyecto del lado del cliente.



Fuente: elaboración propia.

Como se ve en la figura anterior, la raíz principal de la estructura del proyecto se llama “*eval-us-app*” el cual se compone de varios submódulos detallados a continuación:

public/

Contiene archivos accesibles públicamente, como imágenes, archivos estáticos y configuraciones iniciales.

- **favicon.ico**: El ícono que se muestra en la pestaña del navegador.
- **index.html**: El archivo HTML principal que se carga al iniciar la aplicación.
Contiene la estructura básica de la interfaz y la carga de recursos.
- **manifest.json**: El archivo de manifiesto para configurar cómo la aplicación web se comporta cuando se instala en un dispositivo, como el nombre, el ícono, y la pantalla de inicio.
- **robots.txt**: Archivo que instruye a los motores de búsqueda sobre qué contenido debe ser indexado o ignorado.
- **img/**: Carpeta que contiene imágenes usadas en la aplicación.
 - ◆ **logo.png**: El logo de la aplicación.
 - ◆ **vacio.svg**: Una imagen SVG que probablemente se utiliza como placeholder o para representar un estado vacío.
- **plantillas/**: Carpeta que contiene archivos plantillas para importar datos.
 - ◆ **comentarios.xlsx**: Plantilla para cargar comentarios en formato Excel.
 - ◆ **puntajes.xlsx**: Plantilla para cargar puntajes en formato Excel.

src/

Contiene todos los archivos fuente de la aplicación, incluidas las funciones, los componentes y las páginas.

- **api/**: Carpeta que contiene archivos JavaScript con funciones para interactuar con la API backend. Cada archivo corresponde a una acción específica.
 - ◆ **crear.js**: Función para crear nuevos registros en la base de datos.

- ◆ **eliminar.js**: Función para eliminar registros de la base de datos.
 - ◆ **guardar_comentarios.js**: Función para guardar comentarios en la base de datos.
 - ◆ **guardar_puntajes.js**: Función para guardar puntajes en la base de datos.
 - ◆ **guardar_tareas.js**: Función para guardar tareas en la base de datos.
 - ◆ **guardar_tiempos.js**: Función para guardar tiempos en la base de datos.
 - ◆ **obtener_res_comentarios.js**: Función para obtener los resultados de los comentarios.
 - ◆ **obtener_res_puntajes.js**: Función para obtener los resultados de los puntajes.
 - ◆ **obtener_res_tareas.js**: Función para obtener los resultados de las tareas.
 - ◆ **obtener_res_tiempos.js**: Función para obtener los resultados de los tiempos.
 - ◆ **obtener_val_comentarios.js**: Función para obtener los valores de los comentarios.
 - ◆ **obtener_val_puntajes.js**: Función para obtener los valores de los puntajes.
 - ◆ **obtener_val_tareas.js**: Función para obtener los valores de las tareas.
 - ◆ **obtener_val_tiempos.js**: Función para obtener los valores de los tiempos.
- **componentes/**: Carpeta que contiene los componentes reutilizables de la interfaz de usuario.
- ◆ **barra.js**: Componente que probablemente representa la barra de navegación de la aplicación.
 - ◆ **contenido_eficacia.js**: Componente que muestra los datos relacionados con la eficacia del software.
 - ◆ **contenido_eficiencia.js**: Componente que muestra los datos relacionados con la eficiencia del software.
 - ◆ **contenido_sat_comentarios.js**: Componente que muestra los comentarios de satisfacción.

- ◆ **contenido_sat_puntajes.js**: Componente que muestra los puntajes de satisfacción.
- ◆ **contenido_usabilidad.js**: Componente que muestra la usabilidad del software.
- ◆ **estaticos/**: Carpeta que contiene componentes estáticos.
 - **cabecera.js**: Componente que muestra la cabecera de la aplicación.
 - **instrucciones.js**: Componente que proporciona instrucciones o guías sobre el uso de la aplicación.
 - **pie.js**: Componente que representa el pie de página de la aplicación.
- ◆ **graficador.js**: Componente que maneja la generación de gráficos.
- ◆ **graficador-usabilidad.js**: Componente específico para graficar los datos de usabilidad.
- ◆ **graficador_cat.js**: Componente para graficar categorías.
- ◆ **inicio.js**: Componente para la página de inicio de la aplicación.
- ◆ **listado.js**: Componente que muestra un listado de elementos, como los softwares evaluados.
- ◆ **notificador.js**: Componente que muestra notificaciones o alertas al usuario.
- ◆ **nuevo.js**: Componente que maneja la creación de nuevos registros.
- ◆ **tabla.js**: Componente que muestra datos en formato tabla.
- ◆ **tarjeta.js**: Componente que muestra información en un formato tipo tarjeta.
- **css/**: Carpeta que contiene archivos CSS generales para el estilo de la aplicación.
 - ◆ **sb-admin-2.css**: Hoja de estilo principal del tema admin SB.
 - ◆ **sb-admin-2.min.css**: Versión comprimida del archivo anterior.
- **estilos/**: Carpeta que contiene estilos personalizados para la aplicación.
 - ◆ **all.css**: Hoja de estilo que agrupa todos los estilos generales.
 - ◆ **index.css**: Hoja de estilo específica para la página principal.
 - ◆ **tarjeta.css**: Estilos específicos para las tarjetas de la aplicación.
- **paginas/**: Carpeta que contiene las páginas principales de la aplicación.
 - ◆ **App.js**: Componente que maneja las rutas y la estructura global de la aplicación.

◆ **Detalle.js**: Componente que maneja la vista de detalles de un software específico.

→ **config.js**: Archivo de configuración que maneja la configuración del servidor y otras variables de entorno.

→ **index.js**: El punto de entrada de la aplicación. Generalmente se encarga de montar el componente principal de la aplicación.

→ **todo.md**: Lista de tareas pendientes o mejoras para implementar en el proyecto.

Archivos principales en el directorio raíz:

❖ **.gitignore**: Archivo que especifica qué archivos o directorios deben ser ignorados por Git. Generalmente contiene configuraciones para evitar subir archivos temporales, dependencias, o archivos locales.

❖ **package.json**: Archivo de configuración de Node.js que contiene información sobre el proyecto, dependencias y scripts de ejecución.

❖ **package-lock.json**: Archivo generado automáticamente que asegura la integridad de las dependencias del proyecto, con versiones específicas de cada paquete.

❖ **README.md**: Archivo de documentación que describe el proyecto, su propósito, cómo instalarlo, configurarlo y ejecutarlo.

Esta estructura de proyecto es característica de aplicaciones modernas desarrolladas en JavaScript, basadas en React, y optimiza la organización y escalabilidad, permitiendo una gestión eficiente de componentes, API y recursos estáticos para facilitar su mantenimiento y expansión.

En la *figura 30* se puede observar el código del archivo App.js el cual es el punto de partida del proyecto en el lado del cliente, implementado con React. Utiliza React Router para gestionar las rutas, donde la ruta principal ("/") carga el componente Inicio, y la ruta dinámica ("/detalles/:id") muestra el componente Detalle según el parámetro id en la URL. Además, incluye un componente Cabecera estático visible en todas las páginas.

De esta manera, se carga en pantalla una vista de inicio que incluye una cabecera fija y un listado de los softwares evaluados. Los softwares se muestran en formato de tarjetas, para lo cual se invoca el componente `tarjeta.js`, permitiendo una visualización ordenada y estructurada de la información.

Figura 30. Código del componente `App.js` del lado del cliente.



```
App.js

import React from 'react';
import Cabecera from '../componentes/estaticos/cabecera';
import Inicio from '../componentes/inicio';
import Detalle from '../paginas/Detalle';

import 'bootstrap/dist/css/bootstrap.min.css';

import { BrowserRouter, Route, Routes } from 'react-router-dom';

function App() {

  return (

    <div className="">
      <Cabecera />
      <BrowserRouter>
        <Routes>
          <Route exact path="/" element={<Inicio />} />
          <Route path="/detalles/:id" element={<Detalle />} />
        </Routes>
      </BrowserRouter>
    </div>

  );
}

export default App;
```

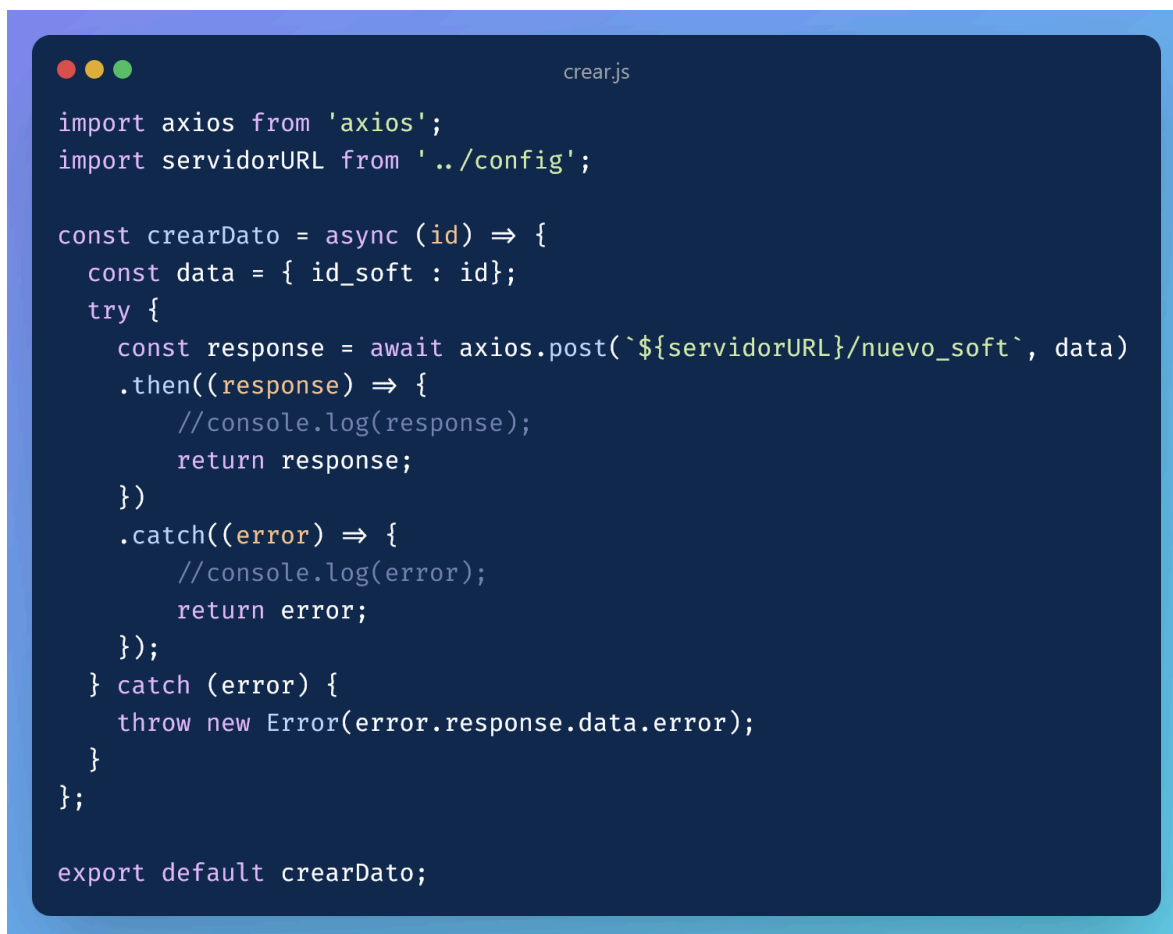
Fuente: elaboración propia.

Por otro lado, el fragmento de código de la *figura 31* define una función `crearDato`, que realiza una solicitud POST a la API para crear un nuevo software en el servidor. Utiliza `axios`, una librería comúnmente empleada para manejar peticiones HTTP.

La función pasa un objeto `data` que contiene el `id_soft` como parámetro en la solicitud. La URL de la API está definida en la variable `servidorURL`, que se importa desde un archivo

de configuración, lo que facilita la gestión de la URL base del servidor y su reutilización en todo el proyecto. Si la respuesta de la API es exitosa, se retorna la respuesta; si ocurre un error, este se captura y se maneja adecuadamente en el bloque catch.

Figura 31. Código para crear un nuevo software a evaluar.

The image shows a code editor window with a dark blue background and light blue text. The file name 'crear.js' is visible in the top right corner. The code is written in JavaScript and uses ES6 syntax, including 'import', 'export', 'const', 'async', 'await', and arrow functions. It uses 'axios' for HTTP requests and includes a try-catch block for error handling. The code is as follows:

```
import axios from 'axios';
import servidorURL from '../config';

const crearData = async (id) => {
  const data = { id_soft : id};
  try {
    const response = await axios.post(`${servidorURL}/nuevo_soft`, data)
    .then((response) => {
      //console.log(response);
      return response;
    })
    .catch((error) => {
      //console.log(error);
      return error;
    });
  } catch (error) {
    throw new Error(error.response.data.error);
  }
};

export default crearData;
```

Fuente: Elaboración propia.

De manera similar, se realizan todas las consultas mediante la API en los demás componentes del proyecto, garantizando una comunicación fluida con el servidor para obtener o enviar datos. Las funciones se exportan para ser utilizada en otras partes del proyecto.

Para no extender la sección de resultados, el código fuente completo del proyecto se incluye como anexo junto con el Manual Del Usuario en el [Anexo A](#).

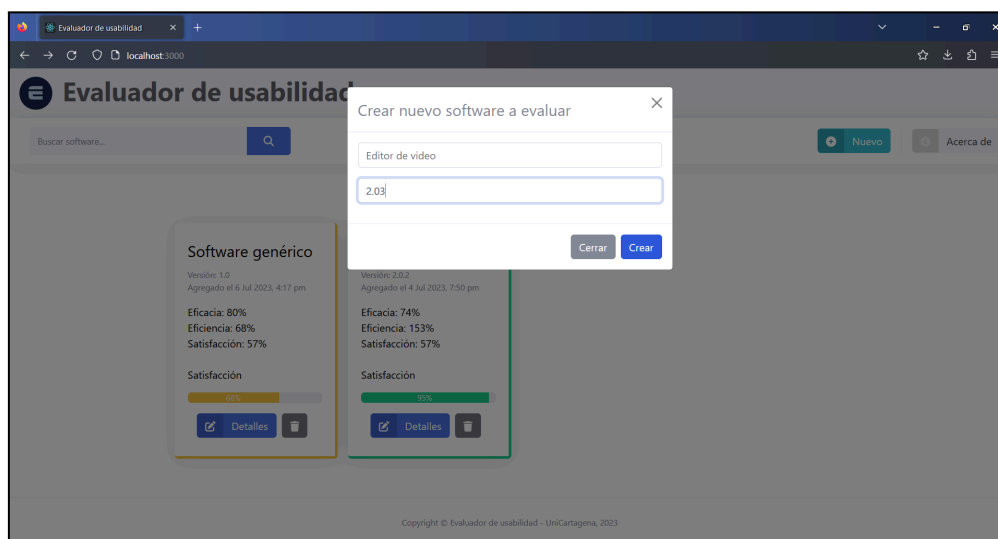
4.2.11. Vistas del aplicativo

En este apartado se muestran las funcionalidades del sistema por casos de uso a través de capturas de pantalla de como lo ve el coordinador de pruebas de usabilidad de software, quien es el usuario directo del sistema (ir al [Anexo A](#) para ver el Manual Del Usuario).

Estas capturas de pantalla demuestran el cumplimiento de los requisitos funcionales que están ampliamente ligados a los casos de usos. Dentro de las funcionalidades mostradas están, por ejemplo, agregar elementos, editar, validar, borrar, hacer cálculos y mostrar reportes, entre otros. Cada imagen contiene una descripción seguido del código del caso de uso al que hace referencia.

Como se muestra en la *figura 32*, al hacer clic en el botón "Nuevo", se abre un cuadro modal que contiene dos campos de texto: uno para ingresar un nombre y otro para especificar la versión. Además, se incluyen dos botones: "Cerrar" y "Crear". El botón "Crear" permite generar un nuevo software para evaluar. En caso de que no se introduzcan valores en los campos de nombre y versión, el software se creará con valores predeterminados.

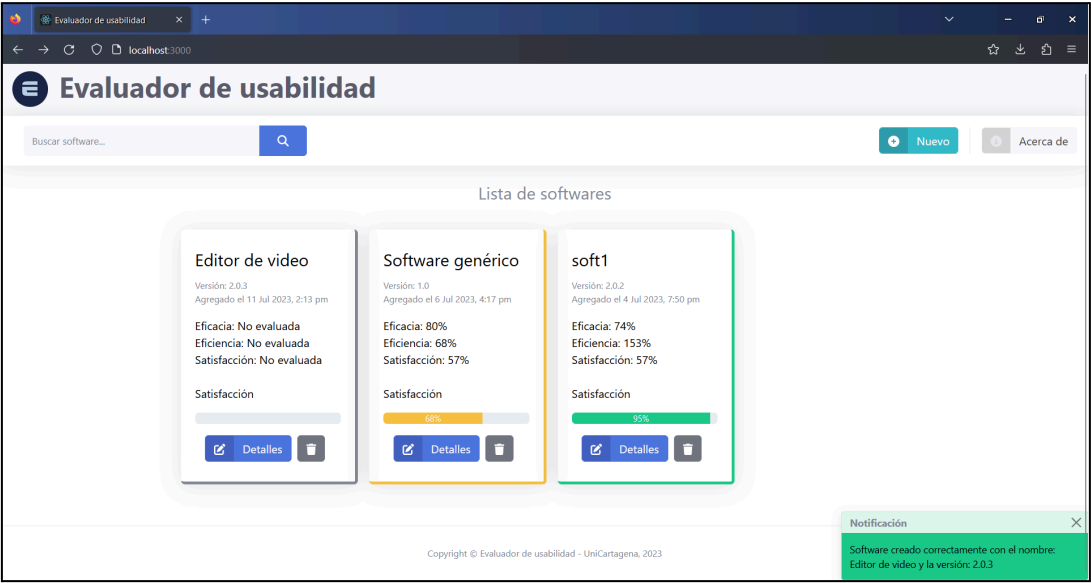
Figura 32. Agregar nuevo software a evaluar, CU1.



Fuente: elaboración propia.

La *figura 33* muestra la pantalla de inicio del aplicativo, donde se presenta la lista de softwares disponibles para evaluar y la notificación de la creación de uno nuevo.

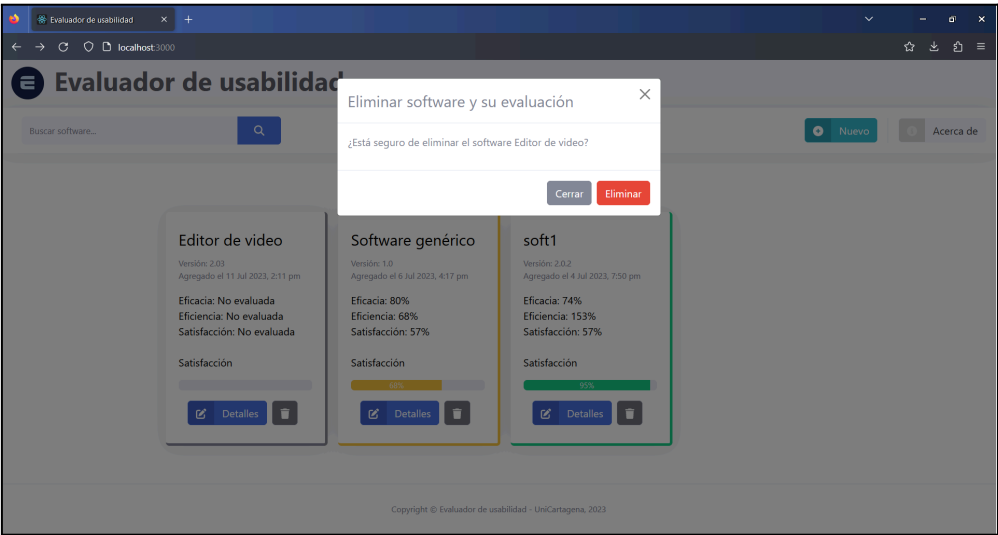
Figura 33. Notificación del software creado y listado, CU1.



Fuente: elaboración propia.

Por su parte, la *figura 34* muestra el proceso de eliminación.

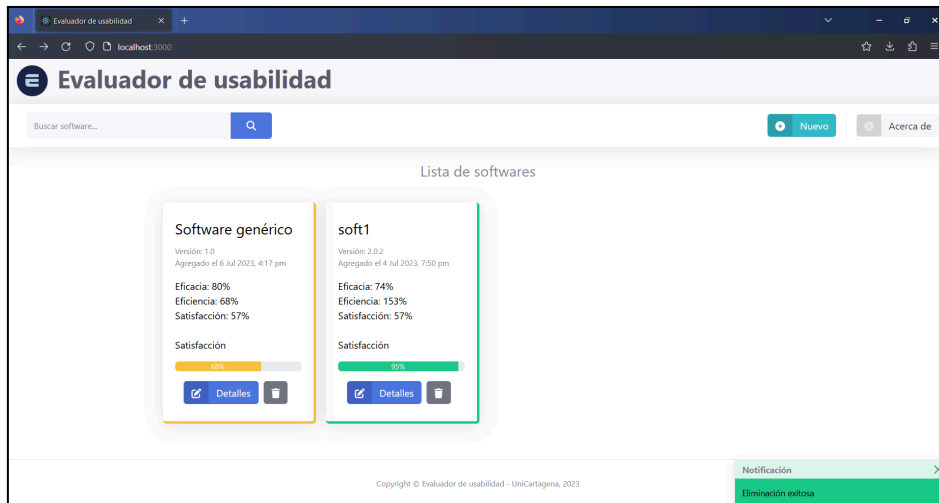
Figura 34. Eliminación del software y su evaluación, CU1.



Fuente: elaboración propia.

Por otro lado, la *figura 35* muestra la notificación y el nuevo listado.

Figura 35. Notificación eliminación, listado actualizado, CU1.



Fuente: elaboración propia.

La *figura 36* presenta un panel de bienvenida al ingresar a un software específico, con un menú lateral a la izquierda que ofrece opciones para trabajar en los diferentes atributos del software seleccionado.

Figura 36. Panel de bienvenida, opciones e instrucciones para cada software, CU1.



Fuente: elaboración propia.

La *figura 37* muestra cómo cargar datos manualmente en el atributo eficacia, procedimiento aplicable a otros atributos numéricos. Se usa un formulario para ingresar cantidad de subtareas y de usuarios.

Figura 37. Creación de tabla para cargar datos, CU1.



Fuente: elaboración propia.

En la *figura 38* se muestra como se crea la tabla con la que se pueden digitalizar los datos.

Figura 38. Carga y edición de valores en la tabla, CU2.



Fuente: elaboración propia.

La *figura 39* muestra que la tabla está protegida contra valores inválidos o nulos, enviando una notificación para informar a la hora de guardar.

Figura 39. Validación de los datos ingresados, CU2.

#T #U Nueva tabla

cargue los datos mediante un archivo CVS o Excel:

Browse... No file selected. Descargar ejemplo

La fila REF corresponde la cantidad de subtareas que tiene cada tarea.
Debe ser el valor más alto de la columna.

	T1	T2	T3
REF	4	3	5
U1	3		3
U2	4		4
U3	2		4
U4	3		5

Guardar datos

Notificación
Algún valor de la matriz no es válido, revise los datos ingresados

Fuente: elaboración propia.

En la *figura 40* se puede observar que la tabla de datos se puede editar mediante funciones de copiar, cortar y pegar.

Figura 40. Tabla editable con funciones de copiar, pegar, cortar, mover, etc, CU2.

#T #U Nueva tabla

O cargue los datos mediante un archivo CVS o Excel:

Browse... No file selected. Descargar ejemplo

La fila REF corresponde la cantidad de subtareas que tiene cada tarea.
Debe ser el valor más alto de la columna.

	T1	T2	T3
REF	4	3	5
U1	3	3	3
U2	4	3	4
U3	2	2	4
U4	3	2	5

Guardar datos

Fuente: elaboración propia.

La *figura 41* muestra una notificación que confirma el guardado de los datos ingresados manualmente en la tabla.

Figura 41. Datos guardados correctamente, CU2.

tos mediante un archivo CVS o Excel:

No file selected.

Descargar ejemplo

La fila **REF** corresponde la cantidad de subtareas que tiene cada tarea.
Debe ser el valor más alto de la columna.

	T1	T2	T3
REF	4	3	5
U1	3	3	3
U2	4	3	4
U3	2	2	4
U4	3	2	5

Guardar datos

Notificación

Datos guardados correctamente

Fuente: elaboración propia.

La *figura 42*, por su parte, muestra la descarga de una plantilla en formato Excel (xlsx o csv) para editar los valores.

Figura 42. Descarga de la plantilla de ejemplo, CU2.

realizadas por cada usuario.

Nueva tabla

Descargar ejemplo

subtareas que tiene cada tarea.
de la columna.

T2	T3
3	5
3	3
3	4

Reporte

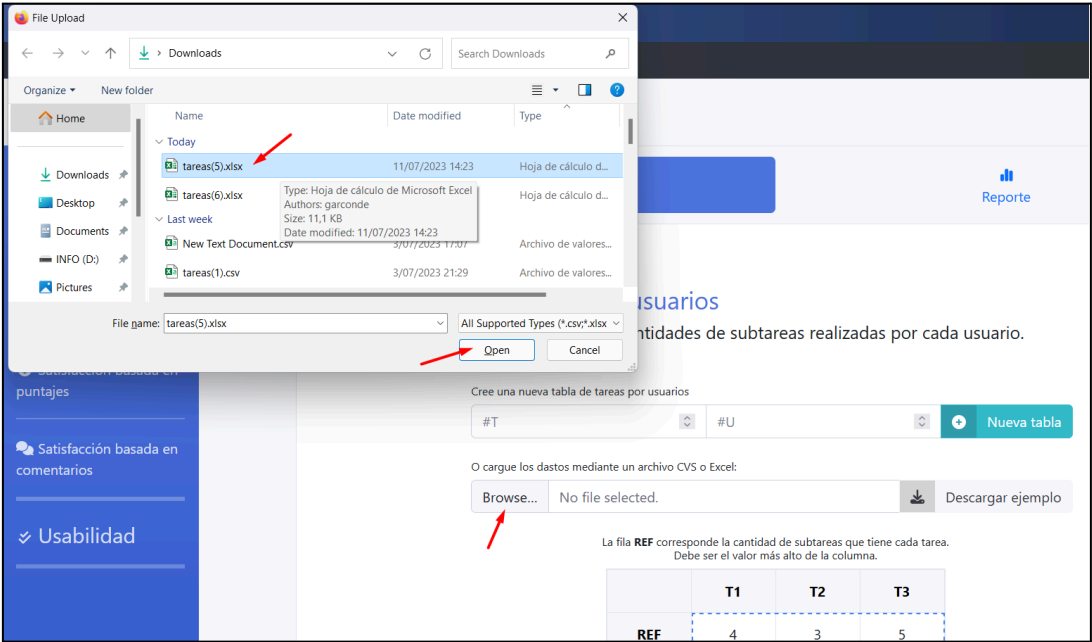
tas realizadas por cada usuario.

Nueva tabla

Fuente: elaboración propia.

La *figura 43* muestra cómo se adjunta el archivo para su lectura.

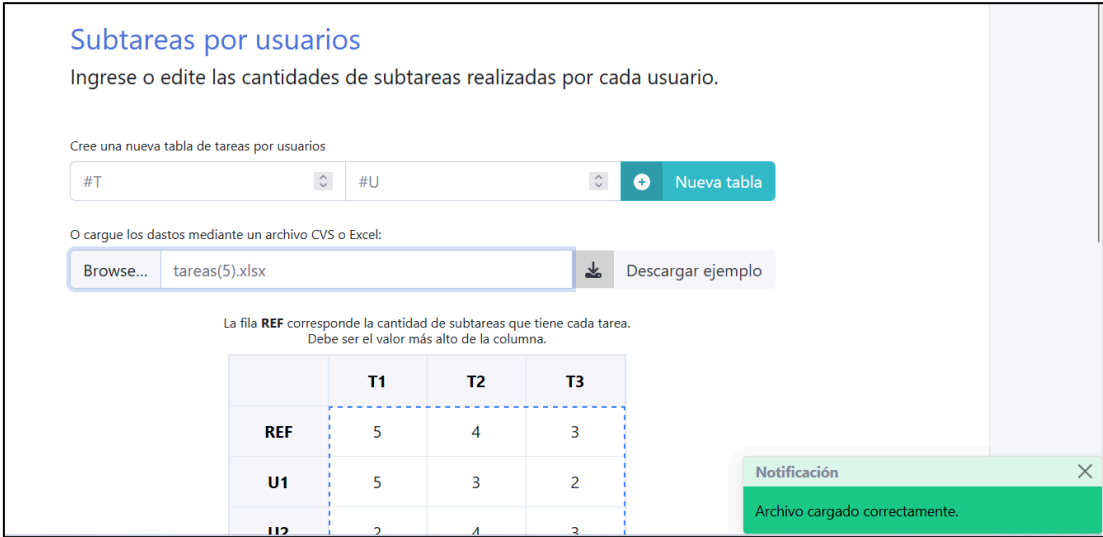
Figura 43. Cargue de datos en formato csv o tipo Excel, CU2.



Fuente: elaboración propia.

La *figura 44* ilustra la notificación que confirma el guardado de los datos cargados desde dicho archivo.

Figura 44. Datos guardados correctamente, CU2.



Fuente: elaboración propia.

La *figura 45* muestra la pestaña “Reporte” dentro de cada ítem de atributo, donde se grafican la eficacia o el atributo correspondiente, tanto por tareas como por usuarios. Las gráficas, con diferentes colores para facilitar la lectura, incluyen un valor global para el atributo.

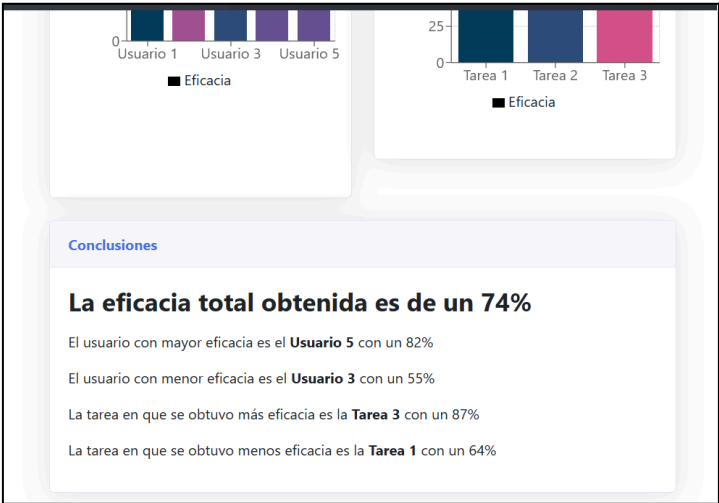
Figura 45. Reporte con gráficas, CU3.



Fuente: elaboración propia.

La *figura 46* muestra un resumen, que destaca las tareas y usuarios más y menos eficaces, con el porcentaje total obtenido en eficacia. Esto cumple con el caso de uso número 3.

Figura 46. Conclusiones en el reporte, CU3.



Fuente: elaboración propia.

La *figura 47* muestra el cargue de comentarios de los usuarios en preguntas abiertas, los cuales se visualizan en una tabla editable para su revisión y análisis, cumpliendo con el caso de uso número 2 (CU2).

Figura 47. Cargue de los comentarios de los usuarios en las preguntas abiertas, CU2.

Comentarios por usuarios

Ingrese o edite los comentarios que cada usuario le dio como respuesta a cada pregunta abierta.

Cree una nueva tabla de comentarios por usuarios

#P

#U

Nueva tabla

O cargue los datos mediante un archivo CVS o Excel:

Browse...

comentarios.xlsx

Descargar ejemplo

La fila PESO corresponde al peso porcentual que tiene cada pregunta

La suma de los pesos debe ser 100 y los comentarios deben ser cadenas de texto en cualquier idioma.

	P1	P2
PESO	40	60
U1	¡¡¡Me encanta la interfaz intuitiva y fácil de usar!!!	No me gusta la falta de opciones de personalización
U2	Las características son increíbles, cumplen con todas mis necesidades	A veces experimento problemas de rendimiento
U3	El soporte al cliente es excepcional, siempre resuelven mis dudas	La falta de actualizaciones frecuentes es decepcionante
U4	¡¡¡La integración con otras herramientas es perfecta!!!	A veces encuentro errores que interrumpen mi flujo de trab
U5	La velocidad y eficiencia del software son impresionantes	Algunas funciones no son tan intuitivas como me gustaría

Guardar datos

Notificación

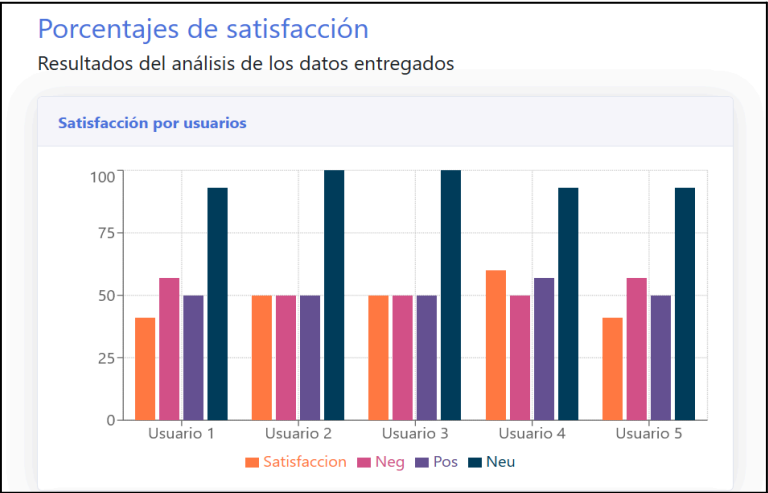
Datos guardados correctamente

Fuente: elaboración propia.

Por su parte, la *figura 48* presenta un reporte gráfico del nivel de sentimiento detectado en los comentarios, lo que permite interpretar de manera visual las opiniones expresadas por los usuarios. Esta funcionalidad corresponde al cumplimiento del caso de uso número 3 (CU3).

114

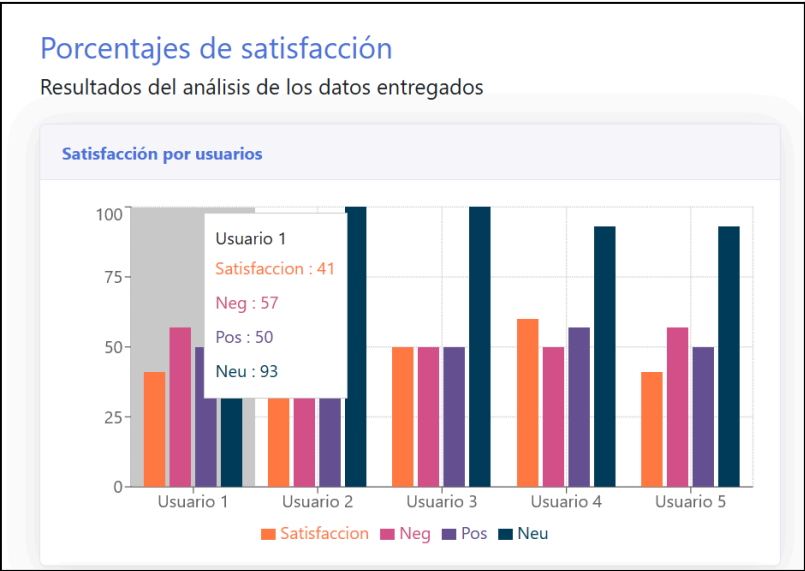
Figura 48. Reporte del nivel de sentimiento encontrado en los comentarios, CU3.



Fuente: elaboración propia.

La *figura 49* muestra la descripción detallada de los valores al pasar el cursor sobre la gráfica de sentimientos, permitiendo al usuario obtener información específica de cada categoría. Esta funcionalidad mejora la interpretación de los datos visualizados y facilita el análisis puntual de los comentarios. Corresponde al cumplimiento del caso de uso número 3 (CU3).

Figura 49. Descripción detallada de los valores al pasar el cursor sobre la gráfica, CU3.

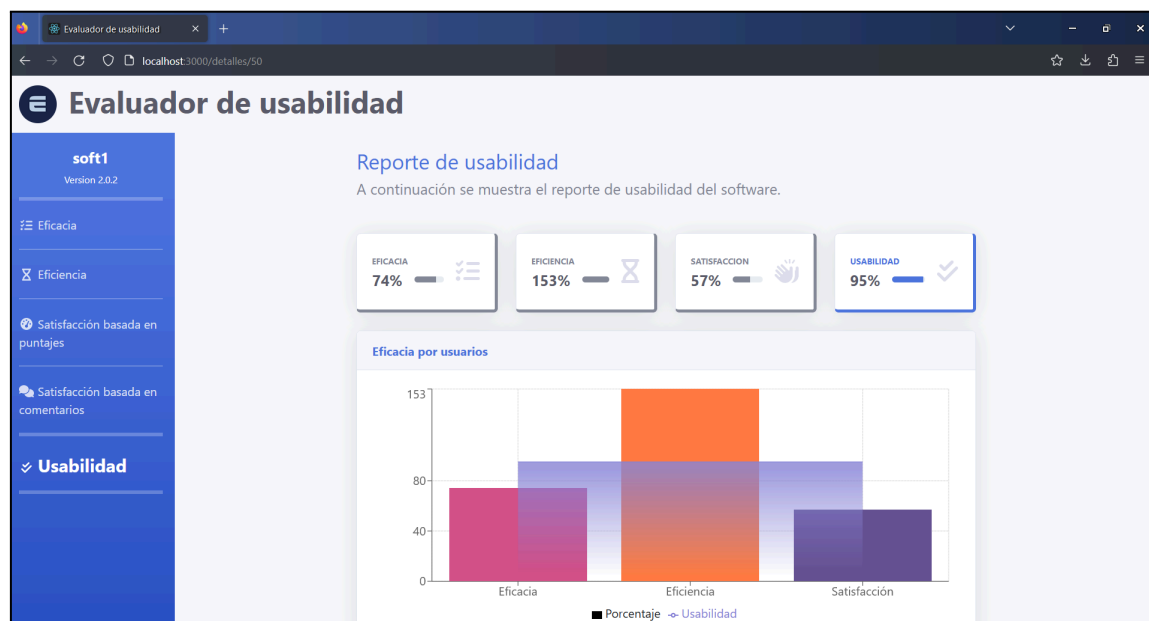


Fuente: elaboración propia.

La *figura 50* muestra el reporte final de la usabilidad del software evaluado, presentado mediante paneles o tarjetas informativas que indican el porcentaje alcanzado en cada uno de los atributos de usabilidad, junto con el cálculo del valor total obtenido. Esta representación clara y estructurada permite al usuario comprender fácilmente el desempeño del software en términos de eficacia, eficiencia y satisfacción.

Además, se incluye una gráfica que representa los porcentajes de los tres atributos evaluados. Esta funcionalidad cumple con el caso de uso número 4 (CU4) y responde directamente al requisito funcional de prioridad ALTA "Reporte de la usabilidad del software evaluado", así como al de prioridad MEDIA "Graficar el porcentaje de usabilidad".

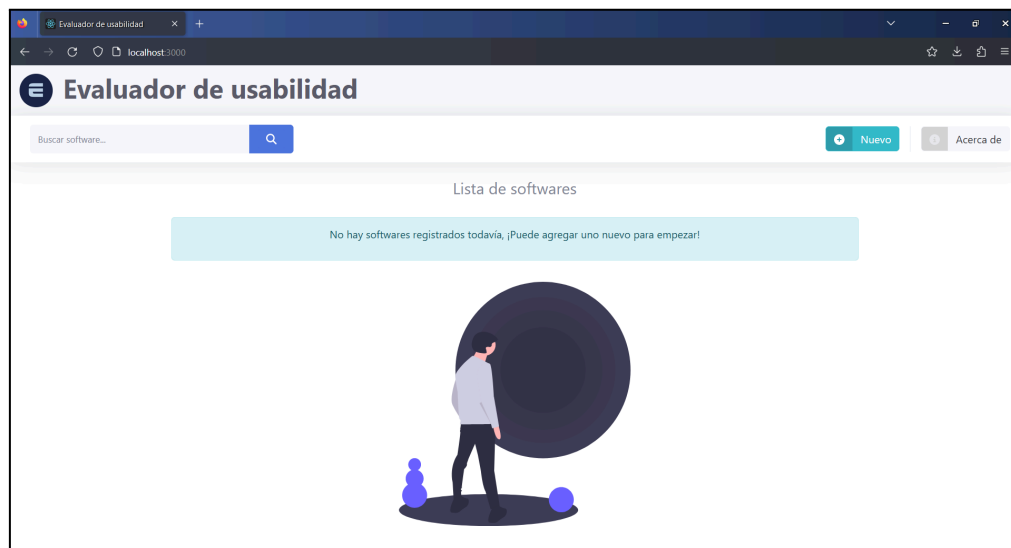
Figura 50. Reporte final de la usabilidad total obtenida del software evaluado, CU4.



Fuente: elaboración propia.

Finalmente, la *figura 51* muestra la vista inicial cuando no se ha creado ningún software para evaluar o todos han sido eliminados. Allí se indica la ausencia de registros y se ofrece la opción de crear un nuevo software, cumpliendo con el caso de uso número 1 (CU1).

Figura 51. Vista inicial cuando no hay ningún software a evaluar, CU1.



Fuente: elaboración propia.

4.3. Paquete de trabajo 3: Pruebas del sistema

En esta fase se aplicó una prueba de usabilidad haciendo uso del software desarrollado, cuyo propósito fue medir el nivel de satisfacción de los usuario, pero también verificar el funcionamiento del sistema en un entorno simulado, identificando posibles fallos, y oportunidades de mejora.

4.3.1. Etapas de la prueba de usabilidad realizada

De acuerdo con la metodología establecida, la prueba de usabilidad se dividió en tres etapas, como se muestra arriba en la [figura 2](#).

Primero, se presentó a los participantes un acuerdo de confidencialidad para resguardar la información obtenida. A continuación, se aplicó un cuestionario pre-test para conocer sus expectativas y conocimientos previos. Luego, los usuarios realizaron una serie de tareas prácticas diseñadas para evaluar la interacción con el sistema. Por último, se aplicó un cuestionario post-test con el fin de analizar su experiencia y nivel de satisfacción.

4.3.1.1. Acuerdo de confidencialidad

En primer lugar, el investigador comunicó al participante la confidencialidad de los datos proporcionados durante la prueba. Se dejó claro que cualquier información obtenida se utilizará únicamente con fines de investigación y no se compartirá con terceros sin el consentimiento previo del participante.

En el [Anexo B](#) se explana detalladamente el acuerdo de confidencialidad usado en la prueba.

4.3.1.2. Etapa 1: cuestionario pre-test

Después, se administró un cuestionario previo a la prueba. Su principal objetivo fue recopilar información sobre los usuarios, su experiencia previa y el contexto habitual en el uso de programas de edición de vídeo.

A continuación, se muestra en la *tabla 11* el cuestionario realizado en la primera etapa de la prueba.

Tabla 11. Etapa 1: preguntas del cuestionario pre-test.

Número	Pregunta	Tipo de respuesta
1	Nombre	Texto corto
2	Sexo	Opción múltiple
3	Edad	Texto corto
4	Nivel de educación completado más recientemente	Opción múltiple
5	Ocupación actualmente	Texto corto
6	¿Con qué frecuencia usa programas o apps de edición de video?	Opción múltiple

7	¿Alguna vez ha usado el software OpenShot Video Editor?	Opción múltiple
8	¿Cuáles de estos programas o apps de edición de video ha usado antes?	Opción múltiple

Fuente: elaboración propia.

Véase el [Anexo C](#) para mayor detalle del cuestionario pre-test usado en la prueba de usabilidad.

4.3.1.3. Etapa 2: listado de tareas

Finalizado el cuestionario pre-test, se asignaron a los participantes tareas basadas en las principales funciones de OpenShot. Estas simulaban situaciones reales y permitieron evaluar la usabilidad del sistema, abarcando desde acciones simples hasta procesos más complejos.

En la *tabla 12* se muestra el listado de tareas propuestas en la segunda etapa de la prueba.

Tabla 12. Etapa 2: listado de tareas para el participante.

Considere el siguiente escenario: Usted ha decidido crear video nuevo a partir de varios recursos gratuitos descargados de internet, alojados en la carpeta Descargas: Software de edición de video: https://www.openshot.org/es/download/ Vídeo 1: https://www.pexels.com/video/flock-of-goose-eating-on-the-lake-water-4872339/ Vídeo 2: https://www.pexels.com/video/brown-bird-in-the-nature-6247699/

Vídeo 3:

<https://www.pexels.com/video/pigeons-on-a-lot-of-grass-looking-for-food-3577687/>

Audio: <https://pixabay.com/music/acoustic-group-summer-walk-152722/>

Con base en lo anterior, se le pide por favor realizar las tareas descritas a continuación:

Tareas	Descripción
1	Importar archivos al software de edición de video. 1. Abrir el software OpenShot y ponerlo en pantalla maximizada. 2. Importar archivos, tanto videos como audios descargados. 3. Previsualizar cualquiera de ellos, sin añadirlo a la línea de tiempo.
2	Crear secuencia de video. 1. Agregue los videos a la línea de tiempo, usando una misma pista para todos los videos. 2. Silencie todos los videos agregados a la línea de tiempo. 3. Agregue el audio a la línea de tiempo usando una pista exclusiva. 4. Agregue un subtítulo en la parte inferior de toda la secuencia que diga “La naturaleza es bella”. 5. Agregue un emoticón o un objeto de la biblioteca de Emoticones, de tal manera que se muestre finalizando la secuencia. 6. Agregue una transición entre cada video, cualquier tipo de transición. 7. Previsualice el resultado de cómo va quedando la secuencia.
3	Exporte la secuencia. 1. Reduzca u organice los elementos que sobresalen de la duración total de los vídeos agregados. 2. Guarde en cualquier ruta todo el proyecto que ha estado trabajando. 3. Seleccione el perfil “FHD 1080p 59.94 fps”.

	4. Exporte el vídeo con el nombre “naturaleza”. 5. Cambie la ruta de guardado por defecto y establezca la carpeta Videos como destino.
--	--

Fuente: elaboración propia.

A criterios del coordinador de la prueba se establecieron las cantidades de subtareas de cada tarea y los tiempos estimados para completarlas. En la *tabla 13* se pueden ver los valores de referencia de cada tarea:

Tabla 13. Valores de referencia para cada tarea a criterio del evaluador.

Tarea	Cantidad de subtareas	Tiempo estimado para ser completada
1	3	2 minutos
2	7	10 minutos
3	5	5 minutos

Fuente: elaboración propia.

4.3.1.4. Etapa 3: cuestionario post-test

Finalmente, se aplicó el cuestionario post-test una vez que los participantes interactuaron con el software OpenShot, para recopilar información sobre la experiencia de uso, incluyendo percepciones, dificultades encontradas, sugerencias de mejora y otros aspectos relevantes para evaluar la usabilidad del sistema. En la *tabla 14* se encuentran las preguntas realizadas en la tercera etapa de la prueba.

Los porcentajes asignados a cada pregunta, mostrados en la última columna de la *tabla 14*, fueron definidos según el criterio del evaluador. Estos valores de ponderación, establecidos conforme a la caracterización de las pruebas de usabilidad (ver sección “[Medición de la usabilidad](#)” en el Paquete de Trabajo 1 del capítulo Resultados), permiten otorgar un peso específico a cada pregunta.

De este modo, se busca evaluar con mayor precisión el nivel de satisfacción del usuario. Es importante señalar que la suma de los porcentajes asignados a las preguntas cerradas debe ser del 100 %, al igual que en el caso de las preguntas abiertas.

Tabla 14. Etapa 3: preguntas del cuestionario post-test.

#	Pregunta	Tipo de respuesta	Porcentaje
1	¿Pudo completar las tareas planteadas?	Opción múltiple	10%
2	¿Considera que la información requerida en la prueba ha sido fácil de encontrar?	Opción múltiple	10%
3	¿Se ha sentido orientado dentro del software OpenShot?	Opción múltiple	10%
4	¿Considera que es fácil navegar a través de las partes del software OpenShot?	Opción múltiple	10%
5	¿Le parecen útiles opciones para edición de video que ofrece el software OpenShot?	Opción múltiple	10%
6	Su grado de satisfacción en cuanto a la facilidad de importar archivos al proyecto es:	Opción múltiple	10%
7	Su grado de satisfacción en cuanto a la facilidad de editar un video y agregarle transiciones es:	Opción múltiple	10%
8	Su grado de satisfacción en cuanto a la facilidad de exportar un video en un formato o perfil específico es:	Opción múltiple	10%
9	Su grado de satisfacción en cuanto a la facilidad de uso general del software OpenShot es:	Opción múltiple	10%

10	¿Volvería a usar el software OpenShot?	Opción múltiple	10%
11	Siendo lo más descriptivo posible, ¿Qué fue lo que más LE GUSTÓ del software OpenShot? Proporcione detalles en su respuesta.	Pregunta abierta	40%
12	Siendo lo más descriptivo posible, Si recomienda este software ¿cuáles serían las razones? Proporcione detalles en su respuesta.	Pregunta abierta	40%
13	Siendo lo más descriptivo posible, ¿Qué recomendaciones puede dar para el software OpenShot? Proporcione detalles en su respuesta.	Pregunta abierta	20%

Fuente: elaboración propia.

Véase el [Anexo D](#) para mayor detalle del cuestionario post-test usado en la prueba de usabilidad.

4.3.2. Participantes de la prueba

El software seleccionado fue evaluado en un entorno controlado de pruebas, con la participación de cinco (5) personas escogidas aleatoriamente por el coordinador de la prueba. La *tabla 15* muestra los nombres y edades de los participantes (Ver [Anexo E](#) para más detalles).

Tabla 15. Personal seleccionado para la prueba de usabilidad.

Usuario	Perfil	Edad
1	Aprendiz área logística	21
2	Desarrollador frontend	28
3	Pastor en iglesia cristiana	56

4	Asesora de ventas y reservas	24
5	Obrero de construcciones civiles	42

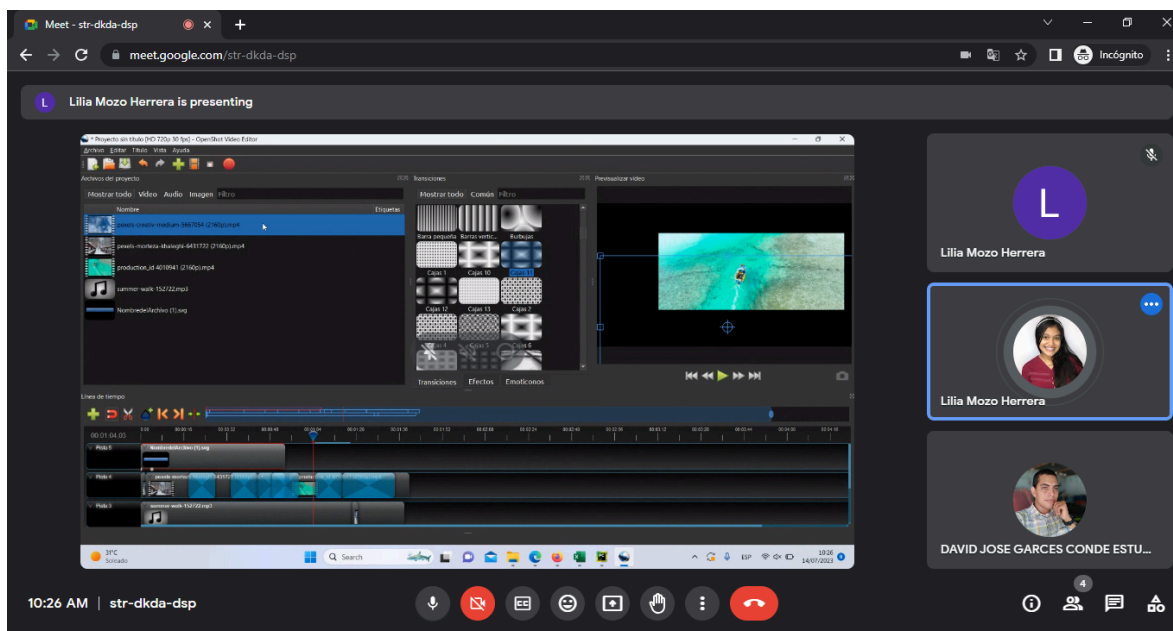
Fuente: elaboración propia.

4.3.3. Ejecución de la prueba

Las siguientes figuras muestran el entorno controlado en el que se desarrolló la prueba, la cual se llevó a cabo de forma remota a través de Internet, utilizando la plataforma Google Meet y su servicio de videotelefonía. En cada figura se observa la participación de dos personas: el investigador y el participante.

El investigador tuvo la responsabilidad de supervisar el desarrollo de la prueba, prestando atención a cualquier eventualidad durante la interacción con el software OpenShot. Por su parte, el participante se encargó de completar las tareas asignadas utilizando la aplicación. En la *figura 52* se puede observar a la participante 1 en el entorno de la prueba de usabilidad del software.

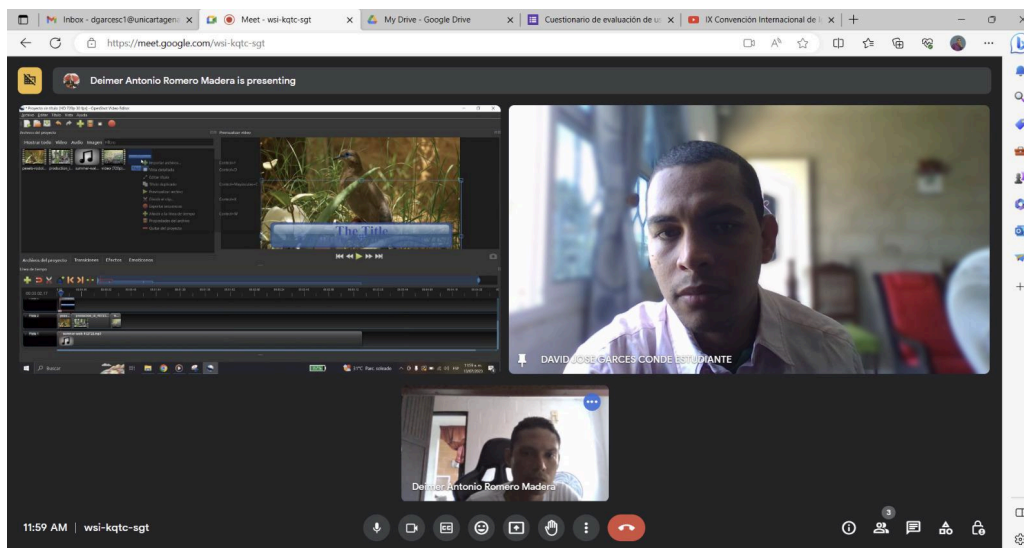
Figura 52. Entorno de la prueba de usabilidad del software con la participante 1.



Fuente: elaboración propia.

En la *figura 53* se puede observar al participante 2 en el entorno de la prueba de usabilidad del software.

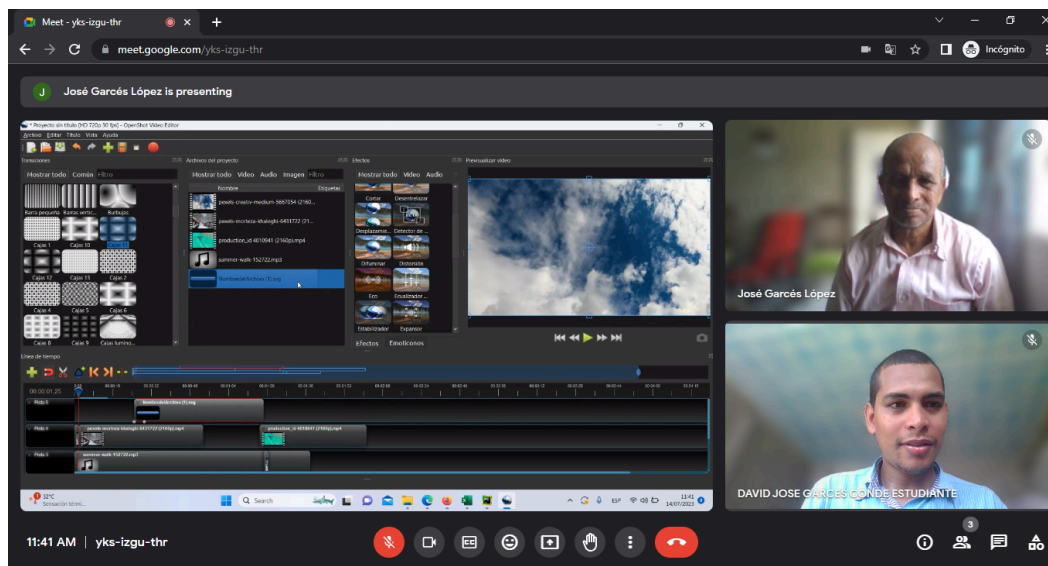
Figura 53. Entorno de la prueba de usabilidad del software con el participante 2.



Fuente: elaboración propia.

En la *figura 54* se puede observar al participante 3 en el entorno de la prueba de usabilidad del software.

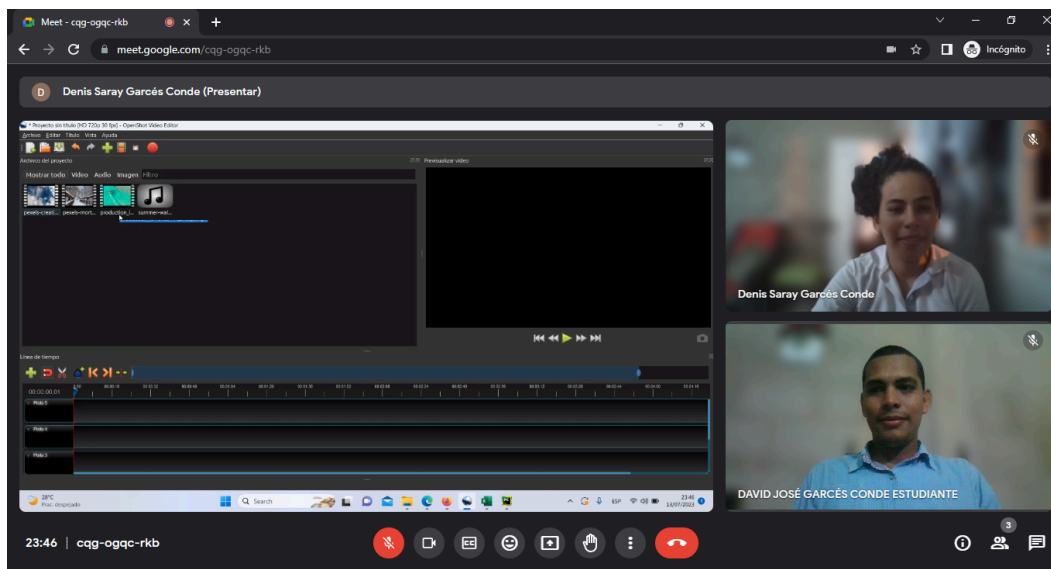
Figura 54. Entorno de la prueba de usabilidad del software con el participante 3.



Fuente: elaboración propia.

En la *figura 55* se puede observar a la participante 4 en el entorno de la prueba de usabilidad del software.

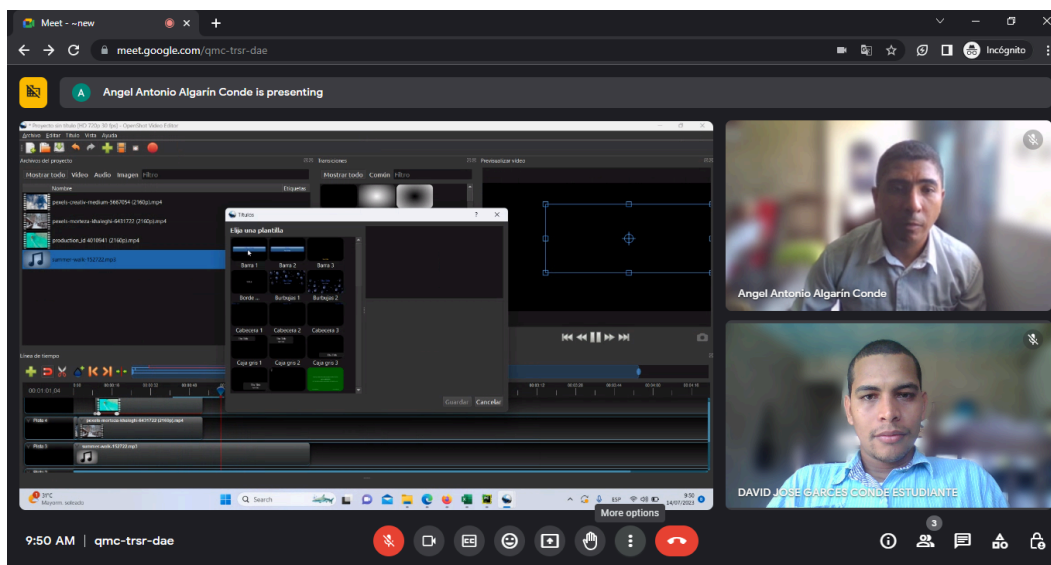
Figura 55. Entorno de la prueba de usabilidad del software con la participante 4.



Fuente: elaboración propia.

En la *figura 56* se puede observar al participante 5 en el entorno de la prueba de usabilidad del software.

Figura 56. Entorno de la prueba de usabilidad del software con el participante 5.



Fuente: elaboración propia.

4.3.4. Resultados de la prueba de usabilidad

En este apartado se muestran los resultados obtenidos de la prueba de usabilidad como normalmente lo hace el evaluador en un estudio de usabilidad, sin uso del aplicativo desarrollado en la presente investigación. Se analizan los tres atributos: eficacia, eficiencia, satisfacción (por preguntas cerradas, y abiertas).

4.3.4.1. Eficacia

Para calcular la eficacia del sistema, se tomaron en cuenta los resultados presentados en la *tabla 16*, la cual muestra la cantidad de subtareas completadas por cada usuario en cada tarea. Estos datos se compararon con los valores de referencia, que corresponden al total de subtareas de cada tarea.

Tabla 16. Cantidad de subtareas totales y cantidad de subtareas completadas por usuario.

Tarea	Tarea 1	Tarea 2	Tarea 3
Cantidad subtareas	3	7	5
Usuario 1	3	4	4
Usuario 2	3	5	4
Usuario 3	2	3	3
Usuario 4	3	5	4
Usuario 5	2	4	3

Fuente: elaboración propia.

Según la medición de la eficacia definida previamente en esta investigación, y con base en las fórmulas establecidas en la metodología, se obtuvo un valor de **72,89 % de eficacia** al realizar los cálculos correspondientes, tal como se ve a continuación, en la *tabla 17*.

Esto demuestra que el software OpenShot alcanza un grado de eficacia del 73 % en la realización de las tareas evaluadas, como se observa en la *tabla 17*, lo que indica un nivel moderado de desempeño en cuanto a la eficacia del sistema para cumplir con los objetivos establecidos en la prueba de usabilidad. Este resultado podría explicarse por la presencia de tareas con un grado de dificultad mayor, errores de interfaz o funcionalidades poco intuitivas que afectaron la correcta finalización de algunas actividades por parte de los usuarios.

Tabla 17. Porcentajes de eficacia.

Tarea	Tarea 1	Tarea 2	Tarea 3	PROMEDIO
Cantidad subtareas	3	7	5	
Usuario 1	100%	57,14%	80%	79,05%
Usuario 2	100%	71,43%	80%	83,81%
Usuario 3	66,67%	42,86%	60%	56,51%
Usuario 4	100%	71,43%	80%	83,81%
Usuario 5	66,67%	57,14%	60%	61,27%
PROMEDIO	86,67%	60%	72%	72,89%

Fuente: elaboración propia.

4.3.4.2. Eficiencia

Para medir la eficiencia del sistema, se compararon los tiempos registrados en la *tabla 18* con los tiempos de referencia, que corresponden a la duración esperada para completar cada tarea.

Tabla 18. Tiempos estimados para cada tarea y duración en completarlas por usuario.

Tarea	Tarea 1	Tarea 2	Tarea 3
Tiempo estimado	2 min	10 min	5 min
Usuario 1	2 min	15 min	4 min
Usuario 2	2 min	11 min	5 min
Usuario 3	3 min	19 min	8 min
Usuario 4	4 min	12 min	6 min
Usuario 5	2 min	16 min	7 min

Fuente: elaboración propia.

De acuerdo con la medición de la eficiencia definida en esta investigación y utilizando las fórmulas descritas en la metodología, se obtuvo un resultado del 81,00 % de eficiencia, como se muestra en la *tabla 19*. Este valor representa un buen nivel de eficiencia según los criterios de referencia establecidos en el marco teórico (ISO 9241-11, 2018), donde se considera adecuado cuando la mayoría de los usuarios completan las tareas dentro del tiempo esperado.

Sin embargo, no alcanza niveles óptimos (superiores al 90 %) posiblemente por la existencia de tareas con mayor complejidad cognitiva o por la limitada familiaridad de los usuarios con la interfaz, lo que impactó el rendimiento temporal de manera moderada.

Tabla 19. Porcentajes de eficiencia.

Tarea	Tarea 1	Tarea 2	Tarea 3	PROMEDIO
Tiempo estimado	2 min	10 min	5 min	-
Usuario 1	100,00%	66,67%	125,00%	97,22%
Usuario 2	100,00%	90,91%	100,00%	96,97%
Usuario 3	66,67%	52,63%	62,50%	60,60%
Usuario 4	50,00%	83,33%	83,33%	72,22%
Usuario 5	100,00%	62,50%	71,43%	77,98%
PROMEDIO	83,33%	71,21%	88,45%	81,00%

Fuente: elaboración propia.

4.3.4.3. *Satisfacción de los usuarios a partir de preguntas cerradas*

Para calcular la satisfacción del sistema, se consideraron los resultados de las preguntas cerradas, donde se utilizó una escala del 1 al 5 para cada una. Estos resultados están detallados en la *tabla 20*.

Tabla 20. Respuesta de cada usuario a las preguntas cerradas.

Tarea	1	2	3	4	5	6	7	8	9	10
Peso	10%	10%	10%	10%	10%	10%	10%	10%	10%	10%
Usuario 1	4	4	4	4	4	4	3	4	4	4
Usuario 2	5	5	4	4	4	5	5	5	5	5
Usuario 3	1	2	2	2	2	2	2	2	2	2

Usuario 4	2	2	3	2	2	4	2	4	2	3
Usuario 5	3	4	3	2	3	3	2	3	4	4

Fuente: elaboración propia.

El cálculo manual de la satisfacción, a partir de las preguntas cerradas, se realizó con base en las fórmulas establecidas en la metodología. Como resultado, se obtuvo un valor porcentual de **65 % de satisfacción**, tal como se ve en la siguiente *tabla 21*:

Entonces podemos deducir que el 65 % de satisfacción obtenido indica un nivel moderado de aceptación del sistema por parte de los usuarios, sugiriendo áreas de mejora para optimizar la experiencia general.

Tabla 21. Porcentaje de satisfacción por usuarios en cada pregunta cerrada.

Tarea	1	2	3	4	5	6	7	8	9	10	$\bar{x}$
Peso	10%	10%	10%	10%	10%	10%	10%	10%	10%	10%	10%
Usuario 1	80%	80%	80%	80%	80%	80%	60%	80%	80%	80%	78%
Usuario 2	100%	100%	80%	80%	80%	100%	100%	100%	100%	100%	94%
Usuario 3	20%	40%	40%	40%	40%	40%	40%	40%	40%	40%	38%
Usuario 4	40%	40%	60%	40%	40%	80%	40%	80%	40%	60%	52%
Usuario 5	60%	80%	60%	40%	60%	60%	40%	60%	80%	80%	62%
PROM	60%	68%	64%	56%	60%	72%	56%	72%	68%	72%	64,8%

Fuente: elaboración propia.

4.3.4.4. Satisfacción de los usuarios a partir de preguntas abiertas

En este apartado, el coordinador de la prueba analizó los comentarios de los usuarios y extrajo conclusiones basadas en su percepción de las respuestas, con el fin de obtener una

visión más clara del nivel de satisfacción con el software. Este análisis es crucial en la medición de la usabilidad, aunque, como se señala en la investigación, es susceptible de errores o sesgos, ya que se basa en una interpretación cualitativa, por lo que representa un punto crítico. En la *tabla 22* se pueden ver resumidos los comentarios de los usuarios.

Tabla 22. Resumen de los comentarios de los usuarios en la prueba de usabilidad.

Usuario	Lo que más le gustó del software OpenShot	Razones para recomendarlo	Recomendaciones para mejorar
1	El software es fácil de usar.	Tiene todas las herramientas necesarias para una buena experiencia de edición de video y está disponible de manera gratuita.	Sería bueno aplicar más opciones de animación de fondo.
2	Le gustaron las líneas de tiempo, la forma de llevar los elementos a la línea y la facilidad para exportar videos.	Lo recomienda por ser fácil de usar y tener una curva de aprendizaje baja.	Mejorar la apariencia visual para que sea más atractiva.
3	Posibilidad de mezclar fotografías para hacer videos cortos.	Lo recomienda porque es gratuito y permite crear videos.	Reorganizar la interfaz para reducir la complejidad.
4	Facilidad para encontrar opciones, interfaz sencilla, buena organización de zonas	Lo recomienda porque, aun siendo principiante, pudo crear un video fácilmente.	Hacer los elementos visuales más accesibles e incluir instrucciones.

	de reproducción, estabilidad y rapidez.		
5	Facilidad para crear videos desde el computador, edición con línea de tiempo. Desea incluir letras en los videos como cantautor.	Lo recomienda por su comodidad de uso.	Aumentar el tamaño de la fuente para facilitar la lectura de opciones en pantalla.

Fuente: elaboración propia.

El coordinador de la prueba concluyó que el software cuenta con una interfaz simple y accesible, lo que lo hace ideal para principiantes. Las herramientas básicas, como la línea de tiempo y la exportación fácil, fueron bien valoradas. No obstante, se sugiere incorporar más opciones de animación y efectos, así como actualizar la interfaz visual para hacerla más atractiva. Además, se recomienda aumentar el tamaño del texto y mejorar su legibilidad, así como hacer los elementos visuales más fáciles de encontrar. En resumen, el software es una excelente opción para quienes buscan una herramienta gratuita y fácil de usar, pero requiere mayor personalización y un diseño más moderno.

4.3.4.5. Grado de usabilidad del software evaluado

Como se muestra en la tabla 22, el grado de usabilidad del software se determinó a partir del promedio aritmético de los tres atributos principales establecidos por la norma ISO 9241-11: eficacia, eficiencia y satisfacción. Este cálculo se realizó con base en los datos objetivos recopilados durante la prueba de usabilidad, sin incorporar cuantitativamente las opiniones subjetivas de los usuarios.

El resultado obtenido para el software evaluado, OpenShot, fue un porcentaje de usabilidad del 72,9 %. Este valor refleja un nivel aceptable de desempeño general, resultado del equilibrio entre los valores de eficacia (73 %), eficiencia (81 %) y satisfacción (65 %) obtenidos mediante preguntas cerradas. El valor más alto en eficiencia sugiere que el

software permite ejecutar las tareas en tiempos adecuados, sin demoras significativas. Sin embargo, la eficacia, ligeramente inferior, indica que algunos usuarios enfrentaron dificultades puntuales al intentar cumplir con ciertos objetivos funcionales, lo cual afecta el cumplimiento total de las tareas.

Por su parte, la satisfacción obtuvo el valor más bajo entre los tres atributos, lo cual señala que, si bien el sistema es funcional y rápido, no genera una experiencia de uso plenamente positiva. Este resultado puede estar relacionado con aspectos como el diseño visual de la interfaz, la organización de herramientas, o la facilidad para aprender a usar funciones específicas, elementos mencionados por los usuarios en sus comentarios abiertos. Estos factores funcionales influyen directamente en la percepción general del sistema y limitan su capacidad de ser considerado altamente usable.

En conjunto, el valor de 72,9 % indica que OpenShot cumple en buena medida con los criterios establecidos por la norma, pero también evidencia áreas de mejora concretas que podrían abordarse para optimizar la experiencia del usuario final.

Tabla 23. Resultados para el software evaluado.

Atributo	Porcentaje
Eficacia	72,89%
Eficiencia	81,00%
Satisfacción del usuario	64,80%
Usabilidad del software evaluado	72,90%

Fuente: elaboración propia.

4.3.5. Cálculo de la usabilidad mediante el uso del aplicativo

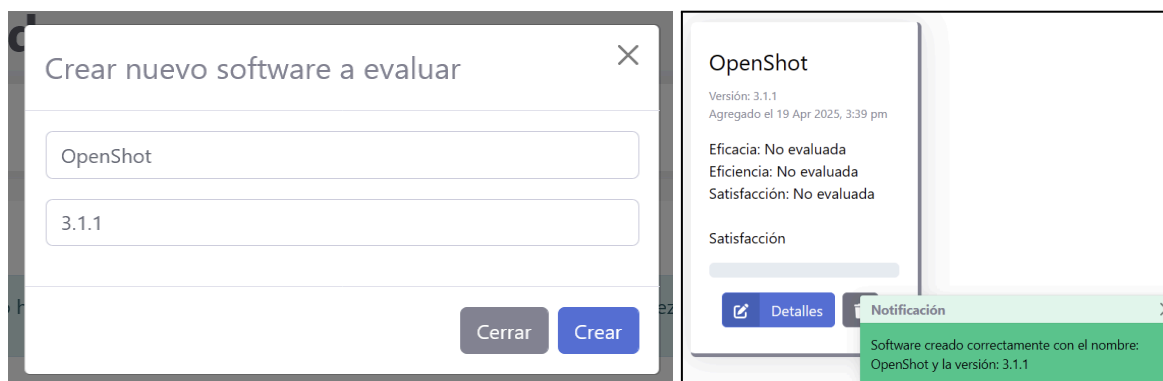
En este apartado se presentan los resultados obtenidos de la prueba de usabilidad, utilizando el nuevo aplicativo “evaluador de usabilidad” desarrollado en esta investigación. Se analizan en detalle los tres atributos fundamentales: eficacia, eficiencia y satisfacción, a

partir de las respuestas a preguntas cerradas y abiertas, mediante un exhaustivo proceso de minería de opiniones.

En primer lugar, se ejecutaron los aplicativos correspondientes: el del lado del servidor, eval-us-api, desarrollado en Python con el framework Flask (las instrucciones para abrirlo se encuentran en el archivo README.md del proyecto, en el [Anexo A](#)), y el del lado del cliente, eval-us-app, construido en JavaScript con el framework ReactJS (también con instrucciones detalladas en el README.md del proyecto, en el [Anexo A](#)).

Una vez abierto el aplicativo evaluador de usabilidad, se procedió a crear un nuevo software a evaluar. Se hizo clic en el botón Nuevo y luego se llenó el cuadro de diálogo correspondiente, como se muestra en la *figura 57* a continuación.

Figura 57. Creación del software a evaluar.



Fuente: elaboración propia.

Posteriormente, se cargaron los datos correspondientes a cada atributo de usabilidad: la cantidad de subtarefas completadas por usuario (Eficacia), el tiempo empleado por cada participante en completar las tareas (Eficiencia), las respuestas a preguntas cerradas (Satisfacción basada en puntajes), y las respuestas a preguntas abiertas (Satisfacción basada en comentarios). Esta información se podía ingresar manualmente o mediante la carga de archivos .xlsx o .csv, utilizando la plantilla proporcionada por el sistema.

Una vez completado el ingreso de datos en cada tabla, estos se guardaban desde la pestaña Datos correspondiente a cada ítem. La *figura 58* muestra un ejemplo con los datos cargados

para el atributo Eficacia, procedimiento que se repitió de forma similar para los demás atributos.

Figura 58. Carga de datos en el aplicativo evaluador de usabilidad.

O cargue los datos mediante un archivo CVS o Excel:

Browse... tareas(7).xlsx Descargar ejemplo

La fila **REF** corresponde la cantidad de subtareas que tiene cada tarea.
Debe ser el valor más alto de la columna.

	T1	T2	T3
REF	3	7	5
U1	3	4	4
U2	3	5	4
U3	2	3	3
U4	3	5	4
U5	2	4	3

Guardar datos

Notificación X

Archivo cargado correctamente.

Fuente: elaboración propia.

4.3.5.1. Eficacia

Una vez finalizado el proceso de carga y almacenamiento de los datos, se revisó el reporte generado automáticamente por el sistema, disponible en la pestaña Reporte, dentro del ítem Eficacia. En dicho informe se muestra el porcentaje de eficacia alcanzado, tanto por cada usuario como por cada tarea, de forma visual e interactiva, como se observa en la *figura 59*. Para consultar los valores específicos representados en la gráfica, basta con posicionar el cursor sobre cada barra.

Figura 59. Reporte de la eficacia por usuarios y por tareas.

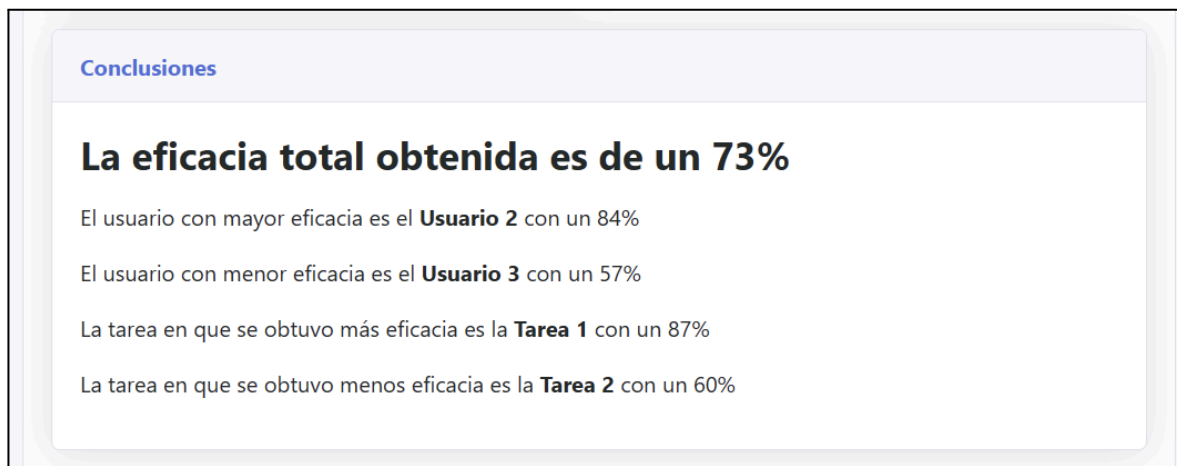


Fuente: elaboración propia.

El resultado obtenido fue de un 73 %, como se ve en la *figura 60*, valor que coincide con el cálculo realizado de forma manual a partir de los datos recolectados, lo cual representa un indicio de la precisión y confiabilidad del sistema evaluador de usabilidad.

Además, el reporte destaca de manera automática los usuarios con los niveles más alto y más bajo de eficacia, así como la tarea con mayor y menor porcentaje de eficacia, tal como se muestra en la *figura 60*.

Figura 60. Reporte de la eficacia del software evaluado.



Fuente: elaboración propia.

La eficacia fue calculada como el porcentaje de subtareas completadas exitosamente por cada usuario. El resultado promedio fue del 73 %, evidenciando que, si bien la mayoría de los usuarios logró finalizar las tareas asignadas, existieron puntos de fricción en la interfaz que afectaron el cumplimiento total.

En términos funcionales, la tarea que presentó mayores dificultades fue la Tarea 2, relacionada con la aplicación de efectos y transiciones a clips en la línea de tiempo. Los usuarios reportaron dificultades para identificar los efectos ya que no cuentan con nombres visibles una vez aplicados, cómo se evidencia en la *figura 61*. Además, no existe una vista previa de la transición aplicada, y su activación requiere reproducir el segmento involucrado, lo cual afectó tanto la comprensión del cambio como la confirmación de su ejecución.

Figura 61. Interfaz de OpenShot con el menú contextual sobre una transición.

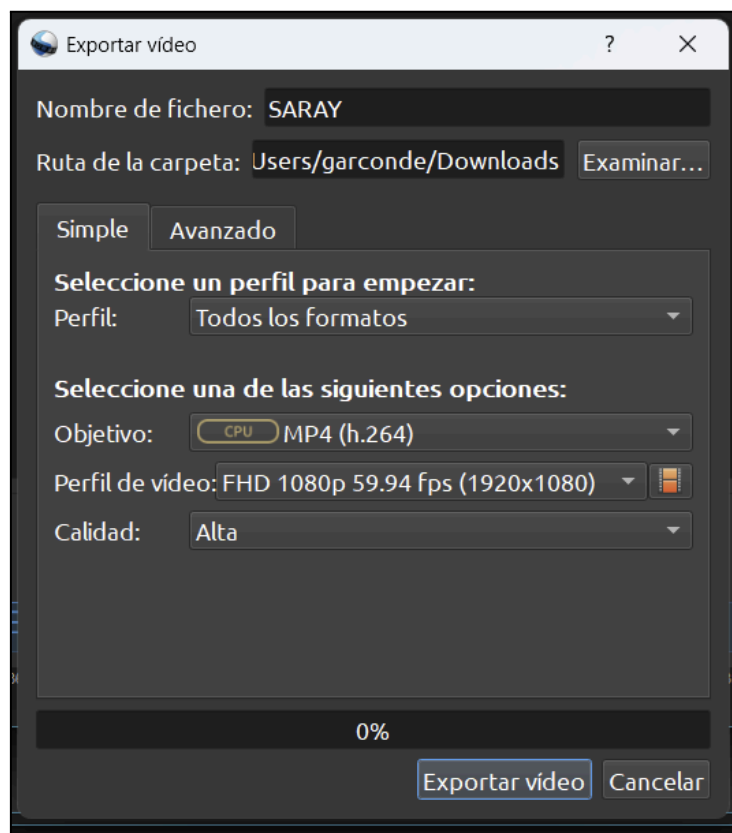


Fuente: Elaboración propia.

Este resultado se ve reflejado también en las respuestas cerradas: el usuario 3, que indicó “muy difícilmente” en la pregunta “¿Pudo completar las tareas planteadas?”, también calificó con 2 su satisfacción respecto a la edición y aplicación de transiciones. Esto se alinea con su baja tasa de eficacia. En contraste, los usuarios más experimentados (usuarios 1 y 2) reportaron haber completado las tareas “fácilmente” y otorgaron puntuaciones de 4 y 5 en los mismos ítems, lo cual muestra una clara relación entre experiencia previa y efectividad operativa.

La tarea que menos afectó la eficacia fue la Tarea 5, correspondiente a la exportación del video, como se ve en la *figura 62*. La mayoría de los usuarios completó esta tarea sin inconvenientes, gracias a un panel claro y a la posibilidad de elegir perfiles de exportación predefinidos, como “para YouTube” o “para dispositivo móvil”. Esto fue reflejado tanto en comentarios como “la forma de exportar el video, debido a que es muy fácil” como en respuestas cerradas con puntuaciones de 4 y 5 en la pregunta: “Su grado de satisfacción en cuanto a la facilidad de exportar un video”.

Figura 62. Interfaz de OpenShot para la exportación de videos.



Fuente: Elaboración propia.

4.3.5.2. Eficiencia

Siguiendo un procedimiento similar al del ítem anterior, el aplicativo evaluador de usabilidad generó automáticamente el reporte correspondiente al atributo Eficiencia, el cual se encuentra disponible en la pestaña Reporte. En este se presenta el porcentaje de eficiencia alcanzado por el software evaluado, como se observa en la *figura 63*.

Figura 63. Reporte de la eficiencia por usuarios y por tareas.

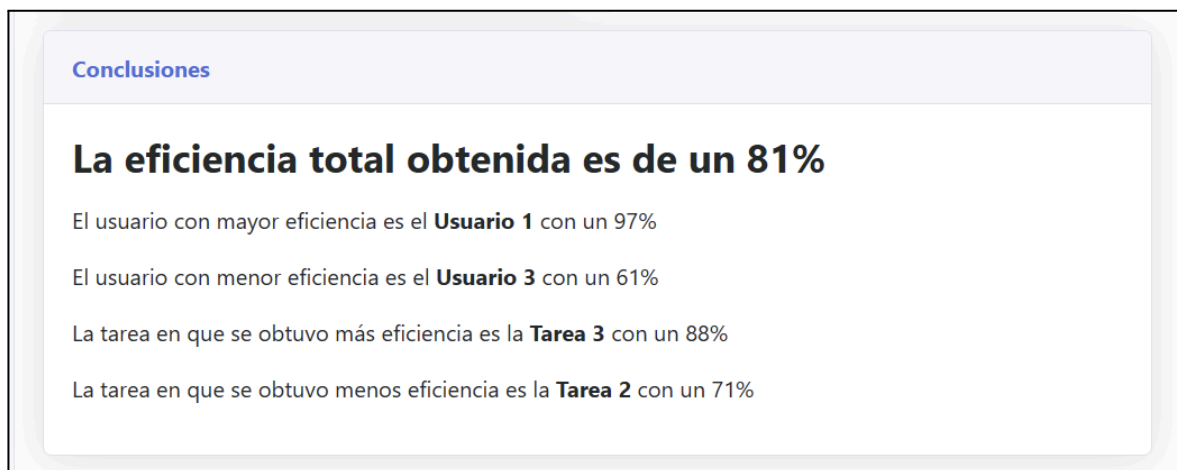


Fuente: elaboración propia.

El resultado fue un 81 % de eficiencia, valor que coincide con el cálculo realizado manualmente a partir de los datos recolectados. Esta concordancia refuerza la precisión y confiabilidad del sistema evaluador en la medición de este atributo, tal como se muestra en la *figura 64*.

Adicionalmente, el reporte identifica de forma automática a los usuarios con el mayor y menor nivel de eficiencia, así como las tareas que registraron los porcentajes más altos y más bajos en este aspecto, información que se presenta también en la *figura 64*.

Figura 64. Reporte de la eficiencia del software evaluado.



Fuente: elaboración propia.

La eficiencia fue medida con base en el tiempo que tardaron los usuarios en completar las tareas, ponderado por la cantidad de tareas completadas correctamente. El resultado obtenido fue del 81 %, indicando que, en general, las tareas se realizaron en un tiempo razonable. Sin embargo, algunas funcionalidades específicas aumentaron el tiempo requerido para ciertos usuarios.

Particularmente en la Tarea 3, que implicaba modificar la duración de un clip, se observó un aumento de tiempo por parte de los usuarios menos experimentados. Esta acción fue desconocida para al menos tres participantes, lo que se refleja en respuestas como “la vista no es muy agradable” y “los elementos deben dar instrucciones”. Estas observaciones coinciden con respuestas cerradas de 2 y 3 en preguntas sobre navegación y facilidad de encontrar funciones.

En contraste, tareas como agregar música o emoticones a la línea de tiempo (parte de la Tarea 1) fueron completadas rápidamente. La mayoría de usuarios señaló que estos elementos eran fáciles de añadir gracias a la estructura intuitiva del panel de archivos y a la herramienta tipo navaja para cortes rápidos. Estos aspectos positivos fueron también destacados en las respuestas abiertas: “me gustó la línea de tiempo y la forma de llevar los elementos a ella”.

4.3.5.3. Satisfacción de los usuarios a partir de preguntas cerradas

En la pestaña Reporte del ítem Satisfacción basada en puntajes, se presenta un desglose detallado de los niveles de satisfacción, ya sea por usuario individual o por cada pregunta evaluada. Esta información se visualiza a través de gráficas intuitivas que muestran el porcentaje de satisfacción asociado a cada elemento, tal como se ilustra en la *figura 65*.

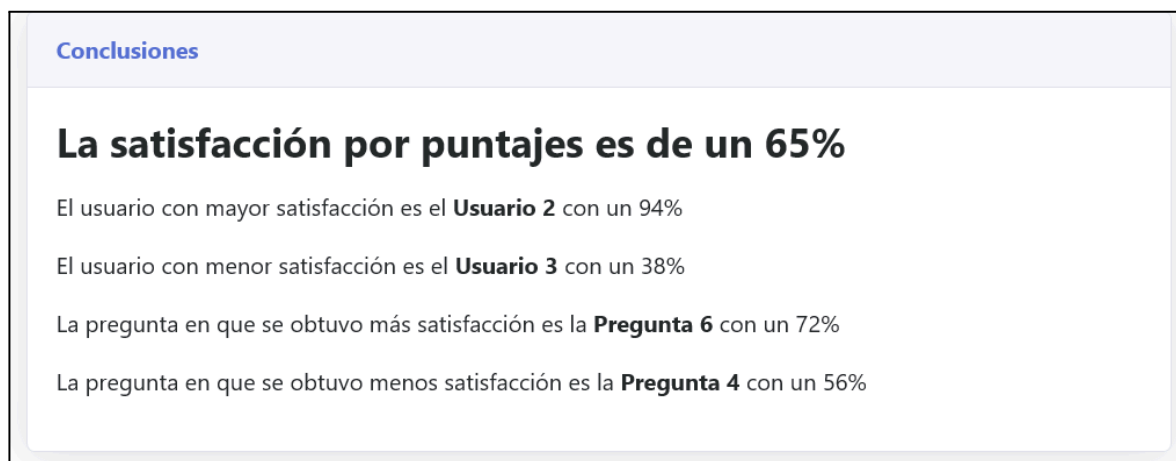
Figura 65. Reporte satisfacción en preguntas cerradas por usuarios y por preguntas.



Fuente: elaboración propia.

Asimismo, la *figura 66* muestra un resumen general del grado de satisfacción basado en puntajes, expresado en porcentaje, destacando al usuario con mayor y menor nivel de satisfacción, así como la pregunta que obtuvo la puntuación más alta y la que recibió la más baja. Cabe resaltar que el porcentaje de satisfacción coincide con el cálculo realizado manualmente en el análisis anterior, con un porcentaje del 65 %, lo cual reafirma la precisión y confiabilidad del sistema evaluador en la medición de este atributo.

Figura 66. Reporte de la satisfacción basada en puntajes.



Fuente: elaboración propia.

4.3.5.4. *Satisfacción de los usuarios a partir de preguntas abiertas*

En esta etapa se abordó el análisis de la satisfacción del usuario a partir de las respuestas a preguntas abiertas, donde los participantes expresaron libremente sus opiniones sobre el software evaluado.

Tal como se expone en la sección [Análisis de sentimientos](#), se aplicaron técnicas análisis de sentimientos para transformar estas respuestas cualitativas en datos cuantitativos. Específicamente, se utilizó el modelo VADER Sentiment, el cual permite calcular la polaridad de cada opinión textual.

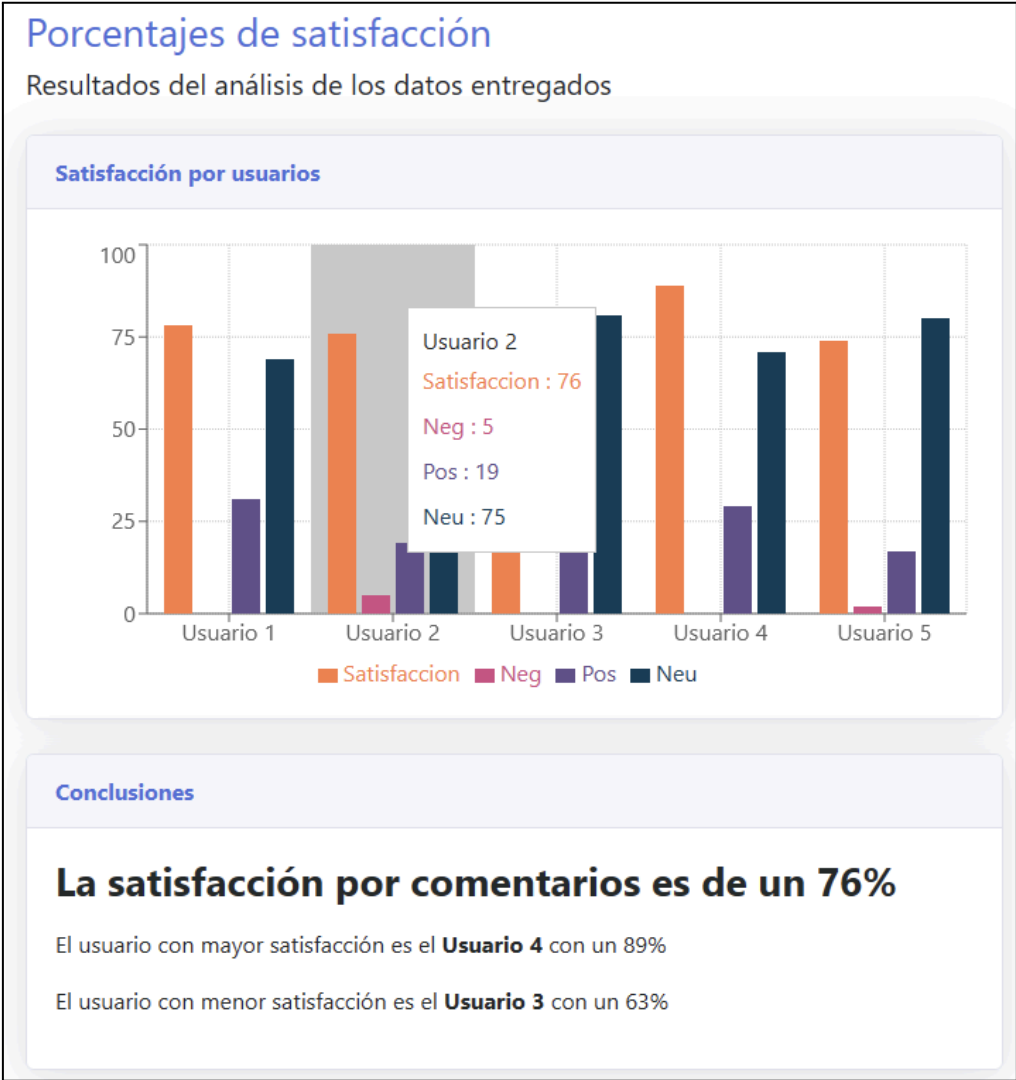
Cada opinión fue analizada para obtener una polaridad compuesta (compound), cuyo valor varía entre -1 (totalmente negativa) y +1 (totalmente positiva), siendo 0 neutro. Estas polaridades se convirtieron en porcentajes, permitiendo visualizar en una gráfica la proporción de polaridad positiva, neutra y negativa por usuario. El valor compuesto, transformado en porcentaje, representa el nivel de satisfacción individual.

Como se muestra en la *figura 67*, los comentarios escritos por los usuarios fueron procesados mediante análisis de sentimientos, obteniendo valores de polaridad negativa, positiva y neutra. En el caso del usuario 2, por ejemplo, se identificó un 5 % de polaridad negativa, un 19 % de polaridad positiva y un 75 % de polaridad neutra.

A partir de estos valores, se calculó la polaridad compuesta (compound), la cual fue transformada a porcentaje, arrojando un 76 % de satisfacción específica. Este valor representa de manera cuantitativa la percepción general del usuario 2 frente al software evaluado, a partir del análisis automático de sus opiniones escritas. De manera similar ocurre con los demás usuarios.

El análisis de satisfacción basado en preguntas abiertas arrojó un **76 % de satisfacción**, un valor diferente al obtenido con las preguntas cerradas. Esta diferencia valida que el análisis de sentimientos en opiniones abiertas ofrece una perspectiva más rica y objetiva de la experiencia del usuario, complementando las valoraciones numéricas y reduciendo el sesgo de juicios subjetivos. La *figura 67* muestra este reporte, incluyendo la deducción del usuario con mayor y menor satisfacción.

Figura 67. Reporte del atributo satisfacción basada en comentarios.

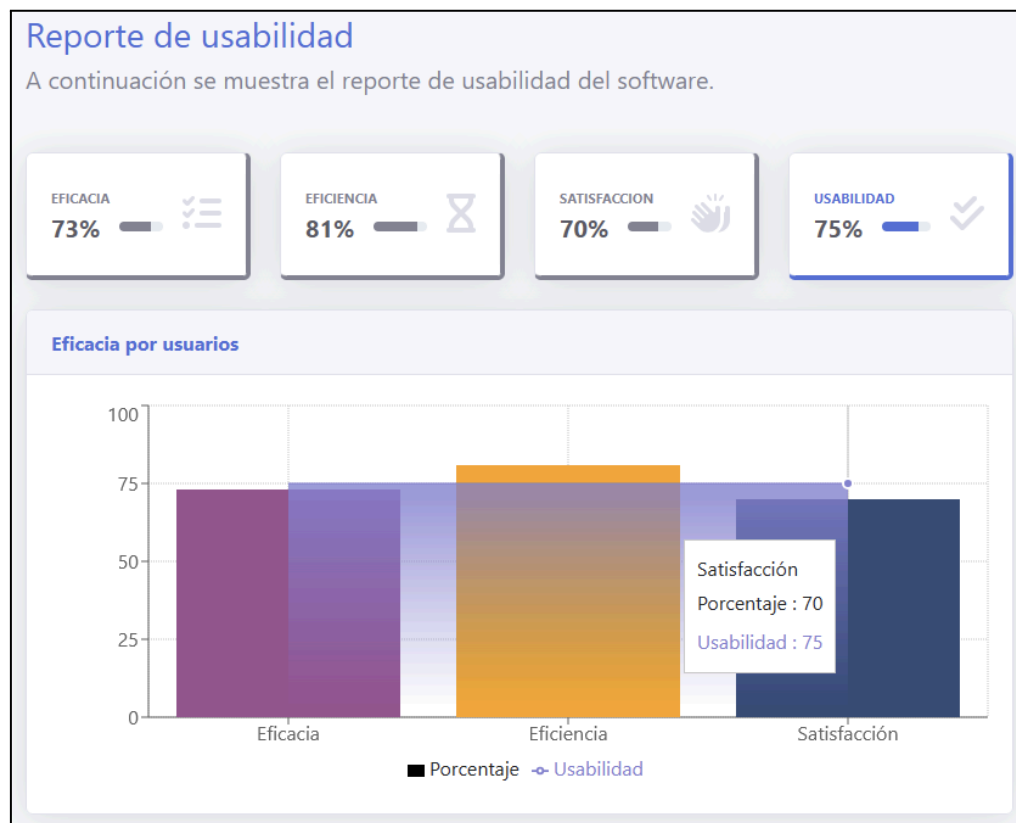


Fuente: elaboración propia.

4.3.5.5. Estimación del atributo Satisfacción general

A partir del cálculo de la satisfacción del usuario, tanto mediante puntajes obtenidos en respuestas cerradas como a través del análisis de opiniones en respuestas abiertas, se obtuvo un promedio general del **70 % para el atributo satisfacción** en relación con el software evaluado durante la presente prueba de usabilidad, como se muestra en la *figura 68*.

Figura 68. Reporte de usabilidad del software evaluado.



Fuente: elaboración propia.

La satisfacción se evaluó a través de un enfoque combinado: puntuaciones otorgadas en preguntas cerradas y análisis de sentimientos aplicados a comentarios abiertos. El valor global obtenido fue del 70 %, resultado del promedio ponderado entre la satisfacción basada en puntuaciones (65 %) y la satisfacción calculada mediante análisis de opiniones (76 %).

Desde las respuestas cerradas, se observó que los usuarios más familiarizados con herramientas de edición otorgaron calificaciones más altas: por ejemplo, el usuario 2 seleccionó “muy satisfactorio” en preguntas sobre facilidad de edición, importación y exportación. Mientras tanto, los usuarios sin experiencia previa (usuarios 3 y 4) manifestaron baja satisfacción, con puntuaciones de 2 en los ítems relativos a facilidad de uso, transiciones y orientación en la interfaz. Estas valoraciones se corresponden con sus

observaciones cualitativas, en las que expresaron que “la letra debería ser más amplia”, “es difícil de leer” y “la vista no es muy agradable”.

Por su parte, el análisis de sentimientos de los comentarios abiertos arrojó un 76 % de satisfacción positiva, con énfasis en la facilidad para importar archivos, la línea de tiempo amigable y la exportación intuitiva. Los comentarios como “me gustó la forma de exportar”, “se pueden mezclar fotos para hacer un video corto” y “muy fácil de ejecutar” fueron clasificados como altamente positivos. Este contraste entre percepción emocional y valoración cerrada sugiere que el análisis cualitativo aporta matices que los formularios estructurados no logran capturar por completo.

4.3.5.6. Grado de usabilidad del software evaluado

De acuerdo con la norma ISO 9241-11, la estimación del grado de usabilidad del software OpenShot Video Editor se realizó calculando el promedio de los tres atributos fundamentales: eficacia (73 %), eficiencia (81 %) y satisfacción del usuario (70 %), tal como se muestra en la *tabla 24*.

Tabla 24. Resultados del evaluador de usabilidad para el software evaluado.

Atributo	Porcentaje
Eficacia	73%
Eficiencia	81%
Satisfacción del usuario	70%
Usabilidad del software evaluado	75%

Fuente: elaboración propia.

El resultado general fue de 75 %, superando ligeramente el 72,9 % obtenido mediante métodos tradicionales basados exclusivamente en métricas cuantitativas. La incorporación del análisis de sentimientos sobre respuestas abiertas permitió integrar percepciones más

matizadas, lo que aportó mayor validez al proceso evaluativo, alineándose con las recomendaciones metodológicas propuestas por Monedero, Cebrián y Desenne (2015).

Este valor refleja un desempeño aceptable en términos generales, con resultados sólidos en tareas como importar archivos al proyecto y exportar el video final, donde la mayoría de los usuarios completaron exitosamente las acciones sin requerir asistencia adicional. Estas tareas estuvieron asociadas a las calificaciones más altas en las preguntas cerradas relacionadas con la utilidad de las funciones y la facilidad de uso general del sistema. La pregunta 8 (“¿Cuál es su grado de satisfacción en cuanto a la facilidad de exportar un video en un formato o perfil específico?”) fue una de las mejor valoradas, con varios usuarios seleccionando 4 (satisfactorio) y 5 (muy satisfactorio). Estas tareas también coinciden con los comentarios positivos en los que los participantes destacaron lo intuitivo del asistente de exportación y la amplia variedad de perfiles de salida disponibles.

En contraste, las tareas que implicaron una interacción más compleja con la interfaz generaron una disminución en la eficacia y eficiencia. Específicamente, la Tarea 2, relacionada con la aplicación de transiciones y efectos, presentó mayores dificultades. Las transiciones deben arrastrarse desde una pestaña lateral poco visible y, una vez ubicadas en la línea de tiempo, no muestran su nombre, ni ofrecen una vista previa individual, lo que obliga a reproducir la secuencia completa para verificar su efecto. La falta de retroalimentación visual clara y la profundidad de navegación para opciones como quitar volumen a un clip o añadir subtítulos personalizados contribuyeron al abandono o incorrecta ejecución de acciones por parte de algunos participantes.

Estas observaciones se reflejan también en la distribución de respuestas en las preguntas cerradas. En particular, las preguntas 7 y 9 —referidas a la facilidad para editar un video y agregar transiciones, y al uso general del software— obtuvieron puntuaciones de 2 o 3 por parte de los usuarios sin experiencia, revelando un área de mejora. Las tareas que menos se relacionaron con estos descensos fueron la importación de archivos (pregunta 6) y la edición simple con la herramienta tipo navaja, ambas mejor valoradas.

Desde una perspectiva de orientación y navegación, algunas funcionalidades clave están ocultas en menús contextuales (por ejemplo, previsualizar un archivo antes de añadirlo a la línea de tiempo) o requieren una secuencia poco intuitiva de clics. Estas limitaciones afectaron especialmente a los participantes sin experiencia previa con OpenShot, quienes expresaron menor grado de orientación en preguntas como la número 3 (“¿Se ha sentido orientado dentro del software?”) y la número 4 (“¿Considera que es fácil navegar a través de las partes del software?”). Esto concuerda con lo planteado por Ramirez-Acosta (2017), quien advierte que la falta de visibilidad de las funciones o de rutas claras de acción puede comprometer gravemente la usabilidad percibida, incluso cuando las funcionalidades están disponibles técnicamente.

Pese a estas dificultades, OpenShot presentó ventajas notables en otros aspectos, como la precisión del previsualizador de video, la facilidad para añadir marcadores y pistas, y la rapidez al cortar clips con la herramienta navaja, elementos que fueron reconocidos como positivos tanto en los formularios cerrados como en los comentarios abiertos. Estos aspectos técnicos contribuyeron directamente a los buenos resultados en eficiencia, especialmente entre usuarios con alguna familiaridad con programas de edición.

Este enfoque integral permite comprender cómo las funcionalidades puntuales del software —y no solo sus atributos generales— condicionan el desempeño del usuario. Además, valida el valor agregado de incluir variables subjetivas tratadas con métodos automáticos y rigurosos.

4.4. Resultados inesperados

Un hallazgo inesperado durante el desarrollo de la herramienta fue que VADER, al analizar comentarios en español, mostró un 46 % de satisfacción y altos niveles de neutralidad, lo que no reflejaba con precisión las opiniones de los usuarios. Este comportamiento se debe a que VADER, diseñado principalmente para el idioma inglés (Hutto y Gilbert, 2014), utiliza un analizador léxico basado en jerarquías y valores propios de este idioma.

Sin embargo, al integrar la librería de traducción automática ArgosTranslate para traducir los comentarios al inglés antes de analizarlos, los resultados mejoraron notablemente, alcanzando un grado de satisfacción de los usuarios del 76 %, reduciendo los niveles de neutralidad, como se vio en la *figura 69*. Es importante destacar que los comentarios se traducen únicamente para su análisis, pero se mantienen en español para su almacenamiento y tratamiento, conforme al alcance del proyecto.

Este hallazgo evidenció la necesidad de ajustar el análisis de sentimientos para otros idiomas, lo que llevó a una mejora crucial en el rendimiento de la herramienta.

Figura 69. Reporte satisfacción basado en comentarios netamente en español.



Fuente: elaboración propia.

4.5. Discusión de los resultados

Las respuestas abiertas en los cuestionarios de usabilidad permiten obtener una valoración más precisa de la satisfacción del usuario, pero su evaluación es compleja debido a su subjetividad (Castro, 2016; Retnani et al., 2017). Tradicionalmente, interpretar estas respuestas de manera manual es laborioso y propenso a errores.

Este proyecto ha utilizado análisis de sentimientos para automatizar el proceso, lo cual se alinea con investigaciones previas sobre la minería de opiniones (Balahur et al., 2013) y su aplicación en la evaluación de la usabilidad (Mendes et al., 2013).

El análisis de sentimientos en los comentarios dio un 70 % de satisfacción, mientras que el cálculo manual alcanzó un 65 %. La similitud entre ambos resultados valida la eficacia del análisis automatizado, confirmando que las opiniones cualitativas son consistentes con los puntajes de las preguntas cerradas y reflejan de manera precisa la satisfacción del usuario.

El uso de esta técnica ha mejorado la precisión en la medición de la satisfacción del usuario. Mientras que el cálculo manual dio un 72,9 % de satisfacción, el análisis automatizado alcanzó un 75 %, un pequeño aumento que refleja la mejora en la exactitud sin distorsionar los resultados. Esto respalda lo indicado por Cuervo et al. (2017), mostrando que la automatización no solo incrementa la precisión, sino también la escalabilidad de las evaluaciones de usabilidad.

Los estudios previos en Colombia, como el de Claro y Collazos (2006), subrayan la falta de herramientas automatizadas para evaluar la usabilidad de sitios web del gobierno, identificando falencias en la presentación y navegación, pero sin abordar el análisis de las opiniones de los usuarios. En cambio, el presente proyecto introduce una herramienta que automatiza la estimación de la satisfacción a través del análisis de sentimientos, mejorando así la precisión en la medición de la usabilidad.

Ocampo et al. (2014) proponen un sistema basado en árboles de decisión para evaluar métricas de usabilidad, pero enfrentan dificultades para adaptarlo a diferentes sitios. Su enfoque no incluye la valoración de los usuarios, una carencia que el análisis de

sentimientos resuelve al proporcionar una evaluación más directa y objetiva de las opiniones subjetivas.

El trabajo de Delgado et al. (2018), que mide el comportamiento emocional mediante expresiones faciales, aporta una perspectiva interesante sobre la usabilidad, pero no integra el análisis de las opiniones de los usuarios. El análisis de sentimientos que utiliza este proyecto va más allá al abordar las valoraciones subjetivas, ofreciendo una visión más completa de la satisfacción del usuario.

En cuanto a las herramientas propuestas por Chanchí y colaboradores (2019, 2020, 2022), si bien ofrecen avances en la automatización y el análisis de usabilidad, su aplicación está limitada a contextos específicos como videojuegos, y no permiten realizar pruebas remotas o adaptarse a diversos tipos de software. Este proyecto, por el contrario, desarrolla una solución flexible y escalable que cubre una gama más amplia de necesidades.

En comparación con las herramientas comerciales como Maze, Userback y Usabilla, que se enfocan en la recolección de comentarios y la retroalimentación visual, ninguna de ellas automatiza la conversión de las opiniones de los usuarios en datos cuantitativos. La solución aquí presentada, que emplea análisis de sentimientos, permite una medición más precisa y objetiva, llenando un vacío que estas herramientas no cubren.

El proyecto aquí desarrollado confirma que el análisis automatizado de sentimientos es una herramienta valiosa para mejorar la fiabilidad en la estimación de la satisfacción del usuario en pruebas de usabilidad.

En este contexto, los resultados del presente proyecto refuerzan la idea de que el análisis automatizado de sentimientos no sólo complementa, sino que puede superar en varios aspectos a los métodos tradicionales de evaluación de usabilidad. Mientras que instrumentos cerrados como el cuestionario SUS, ampliamente utilizado por su sencillez y fiabilidad (Serrano y Cebrián, 2014), arrojaron un valor de satisfacción del 65 %, el análisis aplicado a comentarios abiertos alcanzó un 76 %, lo que evidencia una interpretación más rica y matizada de la experiencia del usuario. Este hallazgo se alinea con los planteamientos

de Thüning y Mahlke (2007), Champney y Stanney (2007) y Sauer y Sonderegger (2009), quienes destacan la influencia de factores emocionales y estéticos en la valoración de los sistemas.

Además, la automatización del análisis redujo significativamente el tiempo requerido para procesar la información, en contraste con los enfoques manuales habituales. Frente a propuestas como las de Delgado et al. (2018), centradas en expresiones faciales, o las herramientas diseñadas por Chanchí et al. (2019, 2020, 2022), limitadas a contextos como los videojuegos, esta solución amplía el espectro de análisis al integrar directamente los comentarios de los usuarios, permitiendo una evaluación más precisa, flexible y aplicable a diversos tipos de software.

4.6. Cumplimiento de los objetivos

El desarrollo del aplicativo permitió alcanzar el primer objetivo específico relacionado con la caracterización de los procesos de evaluación de usabilidad en un laboratorio especializado. Esto se logró mediante la recopilación y documentación de información clave sobre los métodos empleados, la definición de conceptos asociados, el diseño de flujos de proceso y la selección de tecnologías apropiadas para el desarrollo del sistema (PT1).

En cuanto al segundo objetivo, se cumplió al diseñar e implementar los módulos funcionales del aplicativo web, a partir de la caracterización previa. Este proceso incluyó la definición de requisitos, el diseño del modelo de negocio (diagramas de casos de uso, clases, base de datos y paquetes), el desarrollo e integración del prototipo y su posterior despliegue en un entorno web funcional (PT2).

El tercer objetivo también fue alcanzado, ya que se verificó la funcionalidad del sistema mediante una prueba de usabilidad aplicada a un software específico. Para ello, se diseñaron instrumentos de evaluación, se configuró un entorno controlado, se recolectaron datos cuantitativos y cualitativos, y se analizaron los resultados, incluyendo el uso de análisis de sentimientos sobre opiniones abiertas (PT3).

Como resultado global, se evidencia el cumplimiento del objetivo general de esta investigación: desarrollar un aplicativo web que apoye la automatización del proceso de estimación del atributo satisfacción del usuario en pruebas de usabilidad, integrando técnicas computacionales como el análisis de sentimientos.

5. CONCLUSIONES Y RECOMENDACIONES

Este proyecto desarrolló una herramienta web para automatizar la estimación de la usabilidad en pruebas de software, integrando análisis de sentimientos a partir de respuestas abiertas. Su aplicación permitió una evaluación más precisa y enriquecida, respondiendo de forma satisfactoria a la pregunta de investigación: ¿es posible optimizar la estimación de la satisfacción del usuario en pruebas de usabilidad mediante el análisis de sentimiento?

Los objetivos planteados se cumplieron: se caracterizaron los procesos de evaluación de usabilidad en un laboratorio académico; se diseñó e implementó el aplicativo con tecnologías libres (ReactJS y Flask), garantizando flexibilidad, escalabilidad y bajo costo; y se validó su funcionamiento con una prueba práctica sobre el software OpenShot, cargando datos reales y generando reportes automáticos.

El atributo de eficacia fue estimado en 73 %, resultado del análisis de tareas completadas. Este valor, visualizado mediante reportes interactivos por usuario y tarea, evidenció que la herramienta permite identificar puntos críticos del software evaluado de forma clara y objetiva. En cuanto a la eficiencia, se obtuvo un 81 %, lo cual sugiere un desempeño aceptable en el tiempo de ejecución, aunque mejorable. Esta cifra refleja diferencias entre tareas y usuarios, mostrando que el sistema cumple con su propósito al ayudar a detectar ineficiencias operativas.

El atributo de satisfacción arrojó un 65 % mediante preguntas cerradas y un 76 % a través de análisis de sentimientos en comentarios abiertos. Esta diferencia demuestra que los enfoques cualitativos ofrecen una comprensión más rica y matizada de la experiencia del usuario. El sistema, al combinar ambas fuentes, aporta una medición más representativa y menos sesgada, alineándose con diversos autores citados, quienes destacan la necesidad de integrar múltiples perspectivas en la evaluación de la usabilidad.

El grado total de usabilidad fue estimado en 75 % al combinar eficacia, eficiencia y satisfacción (incluyendo análisis de opiniones abiertas), superando el 72,9 % obtenido por el método tradicional. Esta mejora evidencia que la herramienta ofrece una evaluación más

precisa al considerar fuentes adicionales de información, fortaleciendo la validez del proceso.

La arquitectura del sistema, separada en cliente y servidor, facilitó su escalabilidad y mantenimiento. Además, el uso de tecnologías libres permitió reducir costos y ofrecer una solución adaptable a distintos contextos sin restricciones de licenciamiento.

Un hallazgo importante fue la baja precisión del modelo VADER al analizar textos en español, pues este fue diseñado originalmente para inglés. Se resolvió integrando un traductor automático interno que convirtió los textos antes del análisis, lo que mejoró la coherencia de los resultados hasta en un 11%. Este ajuste demuestra la viabilidad del enfoque propuesto, aunque subraya la necesidad de modelos nativos más precisos para otros idiomas.

En cuanto a las limitaciones del estudio, se identificó que el sistema fue aplicado en una única prueba, con un solo software y en un entorno académico controlado, lo cual restringe la generalización de los resultados. Aunque se integró un traductor automático para mitigar la baja precisión del modelo VADER en español, esto no reemplaza la necesidad de modelos nativos más precisos. Además, no se incluyó una validación externa ni se compararon los resultados con otras metodologías cualitativas, lo que limita el alcance y la robustez de la evaluación.

Entre las principales lecciones aprendidas se destaca la importancia de seleccionar tecnologías adecuadas desde el inicio del proyecto. La elección de ReactJS y Flask permitió un desarrollo modular, escalable y de bajo costo, lo cual facilitó la implementación de funciones como la carga de datos, la generación de reportes y la integración del análisis de sentimientos. Se comprobó que automatizar la estimación de la satisfacción del usuario es técnicamente viable, aunque fue necesario resolver limitaciones idiomáticas del modelo VADER mediante traducción automática. También se evidenció que combinar métricas cuantitativas y cualitativas proporciona una visión más rica de la experiencia del usuario. Finalmente, el desarrollo del sistema permitió comprender mejor cómo adaptar

herramientas evaluativas al entorno académico y dejó una base sólida para extender el análisis hacia otros atributos de experiencia de usuario en futuras investigaciones.

La relevancia de este trabajo radica en la automatización de la evaluación de la usabilidad mediante enfoques cuantitativos y cualitativos, lo que representa un aporte significativo a la línea de investigación en Interacción Humano-Computador. El sistema desarrollado mejora la calidad de los procesos evaluativos en entornos académicos y profesionales, y queda disponible como base para futuras investigaciones. En este sentido, el enfoque propuesto sienta las bases para extenderse hacia la estimación de otros atributos de la experiencia de usuario (UX), como la utilidad percibida, la accesibilidad, la credibilidad y la estética, los cuales también pueden inferirse a partir de comentarios subjetivos de los usuarios, ampliando así el alcance y la aplicabilidad del sistema.

Con base en estos hallazgos, se proponen las siguientes recomendaciones para optimizar y escalar el sistema:

- ❖ Optimizar el análisis de sentimientos en español mediante modelos entrenados específicamente o bibliotecas multilingües más avanzadas.
- ❖ Escalar la infraestructura usando servicios en la nube y contenedores para soportar más usuarios y mayor volumen de datos.
- ❖ Ampliar las fuentes de retroalimentación incluyendo opiniones de redes sociales, foros o plataformas externas.
- ❖ Implementar soporte para pruebas de usabilidad remotas, permitiendo la participación geográficamente distribuida.
- ❖ Fortalecer la protección de datos personales conforme a la normativa colombiana (Ley 1581 de 2012) y sus decretos reglamentarios, garantizando el tratamiento adecuado de la información y la confianza del usuario.

6. REFERENCIAS BIBLIOGRÁFICAS

- Ab Hamid, S. H., Husin, W. H. W., & Nasir, M. H. N. M. (2007). The Design of Text to Use Case Diagram Tool. *T2 Software Design & Development*, 1.
- Aelenei, L., Smyth, M., Platzer, W., Norton, B., Kennedy, D., Kalogirou, S., & Maurer, C. (2016). Solar Thermal Systems–Towards a systematic characterization of building integration. *Energy Procedia*, 91, 897-906.
- Agarwal, A., & Meyer, A. (2009). Beyond usability: evaluating emotional response as an integral part of the user experience. In *CHI'09 Extended Abstracts on Human Factors in Computing Systems* (pp. 2919-2930).
- Alarcón, H. F., Hurtado, A. M., Pardo, C., Collazos, C. A., & Pino, F. J. (2007). Integración de técnicas de usabilidad y accesibilidad en el proceso de desarrollo de software de las MiPyMEs. *Avances en Sistemas e Informática*, 4(3).
- Arndt, T. (2018). Big Data and software engineering: prospects for mutual enrichment. *Iran journal of computer science*, 1(1), 3-10.
- Baeza-Yates, Ricardo; Rivera-Loaiza, Cuauhtémoc; Velasco-Martín, Javier. (2004). “Arquitectura de la información y usabilidad en la web”. El profesional de la información, v. 13, n. 3, pp. 169-178. <http://www.elprofesionaldelainformacion.com/contenidos/2004/mayo/1.pdf><http://dx.doi.org/10.1080/13866710412331291886>
- Balahur, A., Steinberger, R., Kabadjov, M., Zavarella, V., Van Der Goot, E., Halkia, M., ... & Belyaeva, J. (2013). Sentiment analysis in the news. *arXiv preprint arXiv:1309.6202*.
- Bevan, N., & Macleod, M. (1994). Usability measurement in context. *Behaviour & information technology*, 13(1-2), 132-145.
- Bohmann, K. (12 de Octubre de 2001). Definition of usability. Recuperado el 20 de Agosto de 2010, de <http://www.bohmann.dk/observations/2001oct12.html>
- Bordens, K., y Abbott, B. (2018). *Research Design and Methods: A Process Approach*. McGraw-Hill Education.
- Bourque, P., Dupuis, R., Abran, A., Moore, J. W., & Tripp, L. (1999). The guide to the software engineering body of knowledge. *IEEE software*, 16(6), 35-44.

- Bowman, D. A., Gabbard, J. L., & Hix, D. (2002). A survey of usability evaluation in virtual environments: classification and comparison of methods. *Presence: Teleoperators & Virtual Environments*, 11(4), 404-424.
- Cadavid, A. N., Martínez, J. D. F., & Vélez, J. M. (2013). Revisión de metodologías ágiles para el desarrollo de software. *Prospectiva*, 11(2), 30-39.
- Cancio, L. P., & Bergues, M. M. (2013). Usabilidad de los sitios Web, los métodos y las técnicas para la evaluación. *Revista Cubana de Información en Ciencias de la Salud (ACIMED)*, 24(2), 176-194.
- Castro, A. D. (2016). Herramienta de soporte para la evaluación subjetiva de la usabilidad mediante SUS-System Usability Scale (Bachelor's thesis). Universidad Autónoma de Madrid.
- Castro, Y., Solarte, G., & Muñoz, L. (2019). Planning, Management and Control of Software Quality. *Scientia et Technica*, 24(4), 2344-7214.
- Cayola, L., & Macías, J. A. (2018). Systematic guidance on usability methods in user-centered software development. *Information and Software Technology*, 97, 163-175.
- Champney, R. K., & Stanney, K. M. (2007, October). Using emotions in usability. In *Proceedings of the Human Factors and Ergonomics Society Annual Meeting* (Vol. 51, No. 17, pp. 1044-1049). Sage CA: Los Angeles, CA: SAGE Publications.
- Chanchi, G. E. G., Campo, W. Y. M., & Sierra, L. M. M. (2019). Estudio del atributo satisfacción en pruebas de usabilidad, mediante técnicas de análisis de sentimientos. *Revista Ibérica de Sistemas e Tecnologias de Informação*, (E 23), 340-352.
- Chanchí, G. E. G., Álvarez, M. C. G., & Campo, W. Y. M. (2020). Criterios de usabilidad para el diseño e implementación de videojuegos. *Revista Ibérica de Sistemas e Tecnologias de Informação*, (E26), 461-474.
- Chanchí-Golondrino, G. E., Ospina-Alarcón, M. A., & Campo-Muñoz, W. Y. (2022). Herramienta para el Análisis de Evaluaciones Heurísticas de Usabilidad Mediante Lógica Difusa. *INGENIERÍA Y COMPETITIVIDAD*, (00).
- Claros, I., & Collazos, C. (2006, November). Propuesta metodológica para la evaluación de la usabilidad en sitios web: Experiencia Colombiana. In *Armenia (Colombia): Documento presentado en el 7º Congreso Internacional de interacción*

- Persona-Ordenador, Asociación para la Interacción Persona-Ordenador, AIPO* (pp. 165-174).
- Clavel, C., & Callejas, Z. (2015). Sentiment analysis: from opinion mining to human-agent interaction. *IEEE Transactions on affective computing*, 7(1), 74-93.
- Couoh Novelo, M. A. (2021). Evaluación de usabilidad en herramientas de aprendizaje colaborativo en dispositivos móviles para ambientes virtuales educativos. *RIDE. Revista Iberoamericana para la Investigación y el Desarrollo Educativo*, 11(22).
- Cuervo, M. C., Aldana, A. C. A., y Álvarez-Carreño, A. M. (2017). Modelos de calidad del software, un estado del arte. *Entramado*, 13(1), 236-250.
- Davis, R., Gardner, J., & Schnall, R. (2020). A review of usability evaluation methods and their use for testing eHealth HIV interventions. *Current HIV/AIDS reports*, 17(3), 203.
- Debdi, O., Paredes Velasco, M., & Velázquez Iturbide, Á. (2013). Un Análisis de Correlación entre Motivación y Usabilidad con GreedExCol.
- Delgado, D. M., Girón, D. F., Chanchí, G. E., y Márceles, K. (2018). Propuesta de una herramienta para la estimación de la satisfacción en pruebas de usuario, a partir del análisis de expresión facial. *Revista Colombiana De Computación*, 19(2), 6-15. <https://doi.org/10.29375/25392115.3438>
- Devi, T. R., & Reddy, V. S. (2012). Work breakdown structure of the project. *Int J Eng Res Appl*, 2(2), 683-686.
- Díaz, E., & Valderrama, C. (2018). Evaluación de la usabilidad de los EVA (entornos virtuales de aprendizaje) a partir de la experiencia de usuarios aplicando lógica difusa. *Revista Vínculos: Ciencia, tecnología y sociedad*, 15(2), 150-159.
- Dijkstra, E. W. (1970). Notes on structured programming.
- Dingli, A., & Cassar, S. (2014). An intelligent framework for website usability. *Advances in Human-Computer Interaction*, 2014.
- Doll, W. J., & Torkzadeh, G. (1988). The measurement of end-user computing satisfaction. *MIS quarterly*, pp. 259-274.
- Downey, L. L. (2007). Group usability testing: Evolution in usability techniques. *Journal of Usability Studies*, 2(3), 133-144.
- Dray, S., & Siegel, D. (2004). Remote possibilities? International usability testing at a distance. *Interactions*, 11(2), 10-17.

- El-Halees, A., & Abu-Zaid, I. M. (2017). Automated Usability Evaluation on University Websites using Data Mining Methods. *Palestinian Journal of Open Education*, 6(11), 13-21.
- Enriquez, J. G., & Casas, S. I. (2013). Usabilidad en aplicaciones móviles. *Informes científicos técnicos-UNPA*, 5(2), 25-47.
- Ferré, X. (2000, November). Principios Básicos de Usabilidad para Ingenieros Software. In *JISBD* (pp. 39-46).
- García López, G., (2022). Sentiment Analysis. Emotional SEO. Disponible en: <https://emotionalseo.com/sentiment-analysis>
- GetFeedback (2021). Customer Experience Platform. Disponible en: <https://www.getfeedback.com/en/>
- González-Sánchez, J. L., Montero-Simarro, F., & Gutiérrez-Vela, F. L. (2012). Evolución del concepto de usabilidad como indicador de calidad del software. *Profesional de la información*, 21(5), 529-536.
- Grau, J. (2007). Pensando en el usuario: la usabilidad. *Anuario thinkEPI*, 1(1), 172-177.
- Guillemette, R. A. (1989). Usability in computer documentation design: Conceptual and methodological considerations. *IEEE transactions on professional communication*, 32(4), 217-229.
- Hernández, R., Fernández, C., & Baptista, P. (2014). *Metodología de la investigación (sexta edición)*. McGraw-Hill Education.
- Hibbeln, M., Jenkins, J. L., Schneider, C., Valacich, J. S., & Weinmann, M. (2017). How is your user feeling? Inferring emotion through human-computer interaction devices. *Mis Quarterly*, 41(1), 1-22.
- Hone, K. S., & Graham, R. (2001). Subjective assessment of speech-system interface usability. In *the Seventh European Conference on Speech Communication and Technology*.
- Hoyos, S. R., Chanchí, G. E., & Acosta-Vargas, P. (2019, December). Software System for the Support of Mouse Tracking Tests. In *Human-Computer Interaction: 5th Iberoamerican Workshop, HCI-Collab 2019, Puebla, Mexico, June 19–21, 2019, Revised Selected Papers* (Vol. 1114, p. 332). Springer Nature.

- Hussain, A. & Mohamed Omar, A. (2020).. A usability evaluation of Lazada mobile application. In *AIP Conference Proceedings (Vol. 1891, No. 1, p. 020059)*. AIP Publishing LLC.
- Hutto, C., & Gilbert, E. (2014, May). Vader: A parsimonious rule-based model for sentiment analysis of social media text. In *Proceedings of the international AAAI conference on web and social media* (Vol. 8, No. 1, pp. 216-225).
- IEEE Standards Coordinating Committee. (1990). IEEE standard glossary of software engineering terminology (IEEE Std 610.12-1990). Los Alamitos. CA: *IEEE Computer Society*, 169, 132.
- IEEE. (1998). IEEE Recommended Practice for Software Requirements Specifications (IEEE Std 830-1998). *Institute of Electrical and Electronics Engineers*.
- ISO/IEC 25010. (2011). Systems and software engineering - systems and software quality requirements and evaluation (SQuaRE) - system and software quality models (ISO 25010). <https://iso25000.com/index.php/en/iso-25000-standards/iso-25010>
- ISO/IEC 9126-1. Software engineering – product quality – Part 1: Quality Model, first ed.: 2001-06-15.
- ISO 9241-11. (2018). Ergonomics of human-system interaction — Part 11: Usability: Definitions and concepts (ISO 25010). <https://www.iso.org/obp/ui/#iso:std:iso:9241:-11:en>
- Kaur, H., & Mangat, V. (2017, February). A survey of sentiment analysis techniques. In *2017 International Conference on I-SMAC (IoT in Social, Mobile, Analytics and Cloud)(I-SMAC)* (pp. 921-925). IEEE.
- Kit, E., & Finzi, S. (1995). Software testing in the real world: improving the process. *ACM Press/Addison-Wesley Publishing Co*.
- Kruchten, Philippe B. "The 4+ 1 view model of architecture." *IEEE software* 12.6 (2002): 42-50.
- Loranger, H. (2016, April 17). Checklist for Planning Usability Studies. *Nielsen Norman Group*. Retrieved October 27, 2022, from <https://www.nngroup.com/articles/usability-test-checklist/>
- Leavy, P. (2017). *Research Design: Quantitative, Qualitative, Mixed Methods, Arts-Based, and Community-Based Participatory Research Approaches*. The Guilford Press.

- Li, X., Liu, Z., & Jifeng, H. (2004, April). A formal semantics of UML sequence diagram. In *2004 Australian Software Engineering Conference. Proceedings.* (pp. 168-177). IEEE.
- Liu, W., Yang, J., Song, Y., Yu, X., & Zhao, S. (2020, January). Research on Software Quality Evaluation Method Based on Process Evaluation and Test Results. In *2019 6th International Conference on Dependable Systems and Their Applications (DSA)* (pp. 480-483). IEEE.
- Llana, L., Martín Martín, E., Pareja Flores, C., & Velázquez Iturbide, Á. (2013). Una evaluación de usabilidad de FLOP.
- Malhotra, R., & Jain, J. (2019). Analysis of refactoring effect on software quality of object-oriented systems. In the *International Conference on Innovative Computing and Communications* (pp. 197-212). Springer, Singapore.
- Marcos, M. C., & Rovira, C. (2005). Evaluación de la usabilidad en sistemas de información web municipales: metodología de análisis y desarrollo. In *La dimensió humana de l'organització del coneixement* (pp. 415-432). Facultat de Biblioteconomia i Documentació.
- Marcos, Mari-Carmen. (2004). "Percibir, procesar y memorizar". El profesional de la información, 2004, v. 13, n. 3, pp. 197-202. <http://www.elprofesionaldelainformacion.com/contenidos/2004/mayo/4.pdf><http://dx.doi.org/10.1080/13866710412331291916>
- Martínez, O. (2015) Metodología de la gestión del alcance de la oferta para los proyectos en el sector de la construcción. *Universidad Militar Nueva Granada*.
- Mascheroni, M. A., Greiner, C. L., Dapozo, G. N., & Estayno, M. G. (2013, June). Automatización de la evaluación de la Usabilidad del Software. In *XV Workshop de Investigadores en Ciencias de la Computación*.
- Maze. (2021). User testing designed for product teams. Disponible en: <https://maze.co/about-us/>
- Medhat, W., Hassan, A., & Korashy, H. (2014). Sentiment analysis algorithms and applications: A survey. *Ain Shams engineering journal*, 5(4), 1093-1113.
- Mendes, M. S., Furtado, E. S., Theophilo, F., & Franklin, M. (2013, December). A Study about the usability evaluation of Social Systems from messages in Natural Language. In the *Latin American Conference on Human Computer Interaction* (pp. 59-62). Springer, Cham.

- Mera, J. (2016). Análisis del proceso de pruebas de calidad de software. *Ingeniería solidaria*, 12(20), 163-176.
- Mishev, K., Gjorgjevikj, A., Vodenska, I., Chitkushev, L. T., & Trajanov, D. (2020). Evaluation of sentiment analysis in finance: from lexicons to transformers. *IEEE Access*, 8, 131662-131682.
- Molich, R., Thomsen, A. D., Karyukina, B., Schmidt, L., Ede, M., van Oel, W., & Arcuri, M. (1999, May). Comparative evaluation of usability tests. In *CHI'99 extended abstracts on Human factors in computing systems* (pp. 83-84).
- Monedero, J., Cebrián, D., & Desenne, P. (2015). Usabilidad y satisfacción en herramientas de anotaciones multimedia para MOOC. *Comunicar: revista científica iberoamericana de comunicación y educación*.
- Montero, Y. H. (2006). Factores del diseño web orientado a la satisfacción y no-frustración de uso. *Revista española de documentación científica*, 29(2), 239-257.
- Najjar, A., Mualla, Y., Singh, K. D., Picard, G., Calvaresi, D., Malhi, A., ... & Främling, K. (2021). One-to-Many Negotiation QoE Management Mechanism for End-User Satisfaction. *IEEE Access*, 9, 59231-59243.
- Namoun, A., Alrehaili, A., & Tufail, A. (2021, July). A Review of Automated Website Usability Evaluation Tools: Research Issues and Challenges. In the *International Conference on Human-Computer Interaction* (pp. 292-311). Springer, Cham.
- Nielsen, J. (1990), Evaluating the thinking aloud technique for use by computer scientists, in Hartson, H. & Hix, D (Eds.) *Advances in Human Computer Interaction: Volume III*, Ablex Publishing Corp.
- Nielsen, J. (1994). *Usability engineering*. Morgan Kaufmann.
- Norman, Donald A. The invisible computer: why good products can fail, the personal computer is so complex, and information appliances are the solution. *MIT press*, 1998.
- Ocampo, R. B., Henao, N. H., Betancourth, C. R., Angarita, L. B., & Taborda, A. P. (2014). Árboles De Decisión Para La Generación De Métrica Cuantitativa En Términos De Calidad En Usabilidad Para Aplicativos Web. *REVISTA COLOMBIANA DE TECNOLOGÍAS DE AVANZADA (RCTA)*, 1(23), 47-53.
- Ojeda, J. C., & Fuentes, M. D. C. G. (2012). Taxonomía de los modelos y metodologías de desarrollo de software más utilizados. *Universidades*, (52), 37-47.

- Ortigosa-Hernández, J., Rodríguez, J. D., Alzate, L., Lucania, M., Inza, I., & Lozano, J. A. (2012). Approaching sentiment analysis by using semi-supervised learning of multi-dimensional classifiers. *Neurocomputing*, 92, 98-115.
- Osada, K., Muke, P. Z., Piwowarczyk, M., Telec, Z., & Trawiński, B. (2020). Comparative Usability Analysis of Selected Data Entry Methods for Web Systems. *Cybernetics and Systems*, 51(2), 192-213.
- Pal, D., & Vanijja, V. (2020). Perceived usability evaluation of Microsoft Teams as an online learning platform during COVID-19 using system usability scale and technology acceptance model in India. *Children and youth services review*, 119, 105535.
- Palinko, O., Kun, A. L., Shyrovkov, A., & Heeman, P. TES INTEGRANTES SEMILLERO DE INVESTIGACIÓN “LABORATORIO DE USABILIDAD”. In. Rincón, O. (2015). SEMILLERO DE INVESTIGACIÓN “Laboratorio de Usabilidad”. In. *SEMINARIO INTERNACIONAL DE INVESTIGACIÓN EN DISEÑO*, 322.
- Planning, S. (2002). The economic impacts of inadequate infrastructure for software testing. *National Institute of Standards and Technology*, 1.
- Puertos, O. L., Cerdán, M. A., & De La Mora, M. P. R. (2020). LOS CASOS DE USO Y EL MODELO DE DOMINIO PARA GENERAR PRUEBAS TEMPRANAS EN EL DESARROLLO DE SOFTWARE. *Miscelánea Científica en México*, 772.
- Ramírez-Acosta, K. (2017). Interfaz y experiencia de usuario: parámetros importantes para un diseño efectivo. *Revista tecnología en marcha*, 30, 49-54.
- Retnani, W. E. Y., Prasetyo, B., Prayogi, Y. P., Nizar, M. A., & Abdul, R. M. (2017, August). Usability testing to evaluate the library's academic web site. In *2017 4th International Conference on Computer Applications and Information Processing Technology (CAIPT)* (pp. 1-4). IEEE.
- Ribera, Mireia; Térmens, Miquel; García-Martín, Maika (2008). “Cómo realizar tests de usabilidad con personas ciegas”. *El profesional de la información*, v. 17, n. 1, pp. 99-105. <http://www.elprofesionaldelainformacion.com/conteni-dos/2008/enero/17.pdfhttp://dx.doi.org/10.3145/epi.2008.ene.12>
- Romero, Y. R., Tame, H., Holzhausen, Y., Petzold, M., Wyszynski, J. V., Peters, H., ... & Dittmar, M. (2021). Design and usability testing of an in-house developed performance feedback tool for medical students. *BMC Medical Education*, 21(1), 1-9.

- Saariluoma, P., & Jokinen, J. P. (2014). Emotional dimensions of user experience: A user psychological analysis. *International Journal of Human-Computer Interaction*, 30(4), 303-320.
- Sánchez Upegüi, A. (2010). Introducción: ¿qué es caracterizar? *Medellín, fundación universitaria católica del norte*.
- Sauer, J., & Sonderegger, A. (2009). The influence of prototype fidelity and aesthetics of design in usability tests: Effects on user behaviour, subjective evaluation and emotion. *Applied ergonomics*, 40(4), 670-677.
- Sauer, J., Sonderegger, A., & Schmutz, S. (2020). Usability, user experience and accessibility: towards an integrative model. *Ergonomics*, 63(10), 1207-1220.
- Schmidt, T., Schlindwein, M., Lichtner, K., & Wolff, C. (2020). Investigating the Relationship Between Emotion Recognition Software and Usability Metrics. *i-com*, 19(2), 139-151.
- Serrano Angulo, J., & Cebrián Robles, D. (2014). Usabilidad y satisfacción de la e-rúbrica. *REDU. Revista De Docencia Universitaria*, 12(1), 177-195.
- Shen, P., Ding, X., Ren, W., & Yang, C. (2018, June). Research on software quality assurance based on software quality standards and technology management. In *2018 19th IEEE/ACIS International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD)* (pp. 385-390). IEEE.
- Thüring, M., & Mahlke, S. (2007). Usability, aesthetics and emotions in human–technology interaction. *International journal of psychology*, 42(4), 253-264.
- Tonella, P., & Potrich, A. (2005). Package diagram. *Reverse Engineering of Object Oriented Code*, 133-154.
- TryMyUI (2021). Get the user's view. Disponible en: <https://trymyui.com/>
- Tullis, T., & Albert, B. (2013). Measuring the user experience: Collecting, analyzing, and presenting usability metrics (2.^a ed.). *Elsevier*.
- Usabilla (2021). Optimize your digital channels with customer feedback. Disponible en: <https://usabilla.com/>
- Useberry (2021). Get answers before decisions. Disponible en: <https://www.useberry.com/>

- Userback (2021). Userback is the #1 visual feedback tool for software teams. Disponible en: <https://www.userback.io/>
- Vilares, D., Alonso, M. A., & Gómez-Rodríguez, C. (2013). Clasificación de polaridad en textos con opiniones en español mediante análisis sintáctico de dependencias. *Procesamiento del lenguaje natural*, 50, 13-20.
- Vinodhini, G., & Chandrasekaran, R. M. (2012). Sentiment analysis and opinion mining: a survey. *International Journal*, 2(6), 282-292.
- Zahra, F., Hussain, A., & Mohd, H. (2017, October). Usability evaluation of mobile applications; where do we stand?. In *AIP Conference Proceedings* (Vol. 1891, No. 1, p. 020056). AIP Publishing LLC.
- Zvarevashe, K., & Olugbara, O. O. (2018, March). A framework for sentiment analysis with opinion mining of hotel reviews. In *2018 Conference on information communications technology and society (ICTAS)* (pp. 1-4). IEEE.

ANEXOS

Anexo A. Enlaces de acceso al repositorio del aplicativo software desarrollado y Manual Del Usuario.

Enlace al frontend: <https://github.com/garconde/eval-us-app>

Enlace al backend: <https://github.com/garconde/eval-us-api>

Enlace al Manual Del Usuario: <https://github.com/garconde/trabajo-grado-2025>

Pueden ser escaneados los siguientes códigos QR para acceder a los repositorios frontend, backend y Manual De Usuario:

		
App	Api	Manual De Usuario

Anexo B. Acuerdo de confidencialidad.

Cuestionario de evaluación de usabilidad

dgarcesc1@unicartagena.edu.co

Switch account

 Not shared

Estimado(a) colaborador(a):

Usted participará en una prueba para evaluar el grado de usabilidad del software **OpenShot Video Editor**. La prueba tiene por objetivo detectar la existencia de problemas en el uso de la plataforma, en el marco de un estudio de usabilidad, a fin de mejorar la experiencia del usuario.

SE ESTÁ EVALUANDO UN SOFTWARE, NO EL DESEMPEÑO DE USTED COMO USUARIO, POR LO TANTO, ¡NO SE PREOCUPE SI COMETE ALGÚN ERROR!

Toda la información que usted nos proporciona es absolutamente confidencial y muy relevante para nuestro estudio, por lo cual le agradecemos su cooperación. La prueba tiene 3 etapas.

SI TIENE ALGUNA DUDA DURANTE EL DESARROLLO DE CUALQUIER ETAPA DE LA PRUEBA, USTED PUEDE PONERSE EN CONTACTO CON EL EVALUADOR.

Next

Page 1 of 4

Clear form

Anexo C. Cuestionario pre-test.

Cuestionario de evaluación de usabilidad

dgarcesc1@unicartagena.edu.co [Switch account](#)

Not shared

* Indicates required question

Cuestionario pre-test

En esta primera etapa usted deberá completar un breve cuestionario con preguntas relativas a su experiencia y contexto habitual de uso de programas de edición de video.

Nombre *

Your answer

Sexo *

☐ Masculino

☐ Femenino

Edad *

Your answer

Nivel de educación completado más recientemente *

☐ Primaria

☐ Bachillerato

☐ Técnico

☐ Tecnólogo

☐ Licenciado

☐ Profesional universitario

☐ Postgrado

Ocupación actualmente *

Your answer

¿Con qué frecuencia usa programas o apps de edición de video? *

☐ Nunca

☐ Muy poco

☐ Algunas veces

☐ Casi siempre

☐ Siempre

¿Alguna vez ha usado el software **OpenShot Video Editor**?

☐ Sí

☐ No

¿Cuáles de estos programas o apps de edición de video ha usado antes? *

Selecciones una o varias.

☐ Adobe Premiere Pro

☐ Final Cut Pro

☐ iMovie

☐ Adobe After Effects

☐ Sony Vegas Pro

☐ Filmora

☐ Other:

Back

Next

Page 2 of 4

Clear form

Anexo D. Cuestionario post-test.

Cuestionario de evaluación de usabilidad

dgarcesc1@unicartagena.edu.co [Switch account](#)

Not shared

* Indicates required question

Cuestionario post-test

En esta tercera etapa usted deberá completar un breve cuestionario que tiene por objetivo obtener la percepción general sobre su experiencia en el uso del software **OpenShot**.

Preguntas cerradas

1. ¿Pudo completar las tareas planteadas? *

☐ 1. Muy difícilmente

☐ 2. Difícilmente

☐ 3. Neutral

☐ 4. Fácilmente

☐ 5. Muy fácilmente

2. ¿Considera que la información requerida en la prueba ha sido fácil de encontrar? *

☐ 1. Muy difícilmente

☐ 2. Difícilmente

☐ 3. Neutral

☐ 4. Fácilmente

☐ 5. Muy fácilmente

3. ¿Se ha sentido orientado dentro del software **OpenShot**? *

- ☐ 1. Muy en desacuerdo
- ☐ 2. En desacuerdo
- ☐ 3. Neutral
- ☐ 4. De acuerdo
- ☐ 5. Completamente de acuerdo

4. ¿Considera que es fácil navegar a través de los las partes del software **OpenShot**? *

- ☐ 1. Muy difícilmente
- ☐ 2. Difícilmente
- ☐ 3. Neutral
- ☐ 4. Fácilmente
- ☐ 5. Muy fácilmente

5. ¿Le parecen útiles opciones para edición de video que ofrece el software **OpenShot**? *

- ☐ 1. Muy difícilmente
- ☐ 2. Difícilmente
- ☐ 3. Neutral
- ☐ 4. Fácilmente
- ☐ 5. Muy fácilmente

6. Su grado de satisfacción en cuanto a la facilidad de importar archivos al proyecto es: *

- ☐ 1. Totalmente insatisfactorio
- ☐ 2. Poco satisfactorio
- ☐ 3. Neutral
- ☐ 4. Satisfactorio
- ☐ 5. Muy satisfactorio

7. Su grado de satisfacción en cuanto a la facilidad de editar un video *
y agregarle transiciones es:

- ☐ 1. Totalmente insatisfactorio
- ☐ 2. Poco satisfactorio
- ☐ 3. Neutral
- ☐ 4. Satisfactorio
- ☐ 5. Muy satisfactorio

8. Su grado de satisfacción en cuanto a la facilidad de exportar un video en un formato o perfil específico es: *

- ☐ 1. Totalmente insatisfactorio
- ☐ 2. Poco satisfactorio
- ☐ 3. Neutral
- ☐ 4. Satisfactorio
- ☐ 5. Muy satisfactorio

9. Su grado de satisfacción en cuanto a la facilidad de uso general del software **OpenShot** es: *

- ☐ 1. Totalmente insatisfactorio
- ☐ 2. Poco satisfactorio
- ☐ 3. Neutral
- ☐ 4. Satisfactorio
- ☐ 5. Muy satisfactorio

10. ¿Volvería a usar el software **OpenShot**? *

- ☐ 1. Muy en desacuerdo
- ☐ 2. En desacuerdo
- ☐ 3. Neutral
- ☐ 4. De acuerdo
- ☐ 5. Completamente de acuerdo

Preguntas abiertas

Siendo lo más descriptivo posible, **¿Qué fue lo que más LE GUSTÓ del software OpenShot?** ^{*} Proporcione detalles en su respuesta.

Your answer

Siendo lo más descriptivo posible, **Si recomienda este software ¿cuáles serían las razones?** ^{*} Proporcione detalles en su respuesta.

Your answer

Siendo lo más descriptivo posible, **¿Qué recomendaciones puede dar para el software OpenShot?** ^{*} Proporcione detalles en su respuesta.

Your answer

Muchas gracias, estimado(a) usuario(a).

Puede finalizar este cuestionario.

[Back](#)

[Submit](#)

Page 4 of 4

[Clear form](#)

Anexo E. Respuestas del cuestionario pre-test.

Nombre	Sexo	Edad	Nivel de educación completado más recientemente	Ocupación actualmente	¿Con qué frecuencia usa programas o apps de edición de video?	¿Alguna vez ha usado el software OpenShot Video Editor?	¿Cuáles de estos programas o apps de edición de video ha usado antes?
Lilia Mozo Herrera	Femenino	21	Profesional universitario	Aprendiz Logística	Algunas veces	No	iMovie
Deimer Romero Madera	Masculino	28	Profesional universitario	Frontend Developer	Algunas veces	No	Adobe Premiere Pro, Adobe After Effects, Sony Vegas Pro, Filmora
José Garcés López	Masculino	56	Bachillerato	Pastor	Nunca	No	Ninguno
DENIS SARAY GARCES CONDE	Masculino	24	Profesional universitario	ASESORA DE VENTAS Y RESERVAS	Nunca	No	Final Cut Pro
Ángel Algarín Conde	Masculino	42	Bachillerato	Obrero de construcciones civiles	Nunca	No	Ninguno

Anexo F. Respuestas del cuestionario post-test.

	A	B	C	D	E	F	G	H	I	J
1	1. ¿Pudo completar las tareas planteadas?	2. ¿Considera que la información requerida en la prueba ha sido fácil de encontrar?	3. ¿Se ha sentido orientado dentro del software OpenShot?	4. ¿Considera que es fácil navegar a través de los las partes del software OpenShot?	5. ¿Le parecen útiles opciones para edición de video que ofrece el software OpenShot?	6. Su grado de satisfacción en cuanto a la facilidad de importar archivos al proyecto es:	7. Su grado de satisfacción en cuanto a la facilidad de editar un video y agregarle transiciones es:	8. Su grado de satisfacción en cuanto a la facilidad de exportar un video en un formato o perfil específico es:	9. Su grado de satisfacción en cuanto a la facilidad de uso general del software OpenShot es:	10. ¿Volvería a usar el software OpenShot?
2	4. Fácilmente	4. Fácilmente	4. De acuerdo	4. Fácilmente	4. Fácilmente	4. Satisfactorio	3. Neutral	4. Satisfactorio	4. Satisfactorio	4. De acuerdo
3	5. Muy fácilmente	5. Muy fácilmente	4. De acuerdo	4. Fácilmente	4. Fácilmente	5. Muy satisfactorio	5. Muy satisfactorio	5. Muy satisfactorio	5. Muy satisfactorio	5. Completamente de acuerdo
4	1. Muy difícilmente	2. Difícilmente	2. En desacuerdo	2. Difícilmente	2. Difícilmente	2. Poco satisfactorio	2. Poco satisfactorio	2. Poco satisfactorio	2. Poco satisfactorio	2. En desacuerdo
5	2. Difícilmente	2. Difícilmente	3. Neutral	2. Difícilmente	2. Difícilmente	4. Satisfactorio	2. Poco satisfactorio	4. Satisfactorio	2. Poco satisfactorio	3. Neutral
6	3. Neutral	4. Fácilmente	3. Neutral	2. Difícilmente	3. Neutral	3. Neutral	2. Poco satisfactorio	3. Neutral	4. Satisfactorio	4. De acuerdo

	A	B	C
1	Siendo lo más descriptivo posible, ¿Qué fue lo que más LE GUSTÓ del software OpenShot? Proporcione detalles en su respuesta.	Siendo lo más descriptivo posible, Si recomienda este software ¿cuáles serían las razones? Proporcione detalles en su respuesta.	Siendo lo más descriptivo posible, ¿Qué recomendaciones puede dar para el software OpenShot? Proporcione detalles en su respuesta.
2	El software es fácil de usar.	Tiene todas las herramientas necesarias para una buena experiencia de edición de video y está disponible de manera gratuita.	Sería bueno aplicar más opciones de animación de fondo.
3	Me gustó las líneas de tiempo y la forma de llevar los elementos a la línea de tiempo. Además, me gustó la forma de exportar el video, debido a que es muy fácil.	Sí lo recomendaría. Es fácil de usar y cualquier persona sin conocimientos podría aprender a usarlo muy rápido, debido a que su curva de aprendizaje es baja.	La vista no es muy agradable a la vista, por tanto, se podrían realizar mejoras visuales que sea llamativo a los usuarios.
4	Se pueden mezclar fotografías para hacer un video corto.	Porque es gratis y se pueden crear videos.	Organizarla para que sea menos compleja.
5	Me parece que el programa tiene algunas opciones muy fácil de encontrar, me gusta lo sencillo del programa, la manera que tiene organizada las zonas de reproducción de video, es decir me gusta la vista y lo fácil de manejo, no se traba, es rápido y fácil de ejecutar.	Sí, lo recomendaría, sinceramente siendo la primera vez que debo hacer un video pude lograr hacerlo muy bien.	Los elementos visuales deben estar más fáciles de encontrar y debe dar instrucciones.
6	Me gusto por la facilidad de hacer un video desde un computador, usar la linea de tiempo en la edición estuvo bastante chévere. Como canta-autor me gustaría colocarle letras a mis videos con la aplicación.	Lo recomendaría a mis conocidos por la comodidad para el usuario que maneja el software de edición.	La letra debería ser mas amplia, es un poco pequeña y se me dificultaba leer las opciones en pantalla.